

Mémoire de fin d'études

Pour l'obtention du diplôme d'Ingénieur d'État en Informatique

Option : Systèmes d'Information et Technologies (ST)

Coordination inter-nœuds pour une défense collaborative contre les attaques DDoS : *conception d'un protocole distribué basé sur des API sécurisées*

Réalisée par :

SI SABER Rania

Organisme d'accueil :

École nationale supérieure d'informatique

Encadré par :

M. ANANE Mohamed
M. NAÏT-ABDESSELAM Farid

Promotion : 2025/2026

Dédicace

*À mes très chers parents, qui ont toujours été les piliers de ma vie,
guidant mes pas avec une patience infinie et un amour incommensurable.*

*À ma mère, qui a cru en moi avant même que je ne sache ce qu'était un rêve.
Pour tes prières quotidiennes, ta tendresse de chaque instant
et ton soutien inconditionnel qui ont été mon refuge
et ma plus grande source de motivation tout au long de ce parcours universitaire.*

*À mon père, pour m'avoir inculqué le goût de l'effort,
la valeur du travail et la persévérance.
Tes sacrifices silencieux, tes précieux conseils et ta confiance absolue
en mes capacités m'ont donné le courage de me dépasser
et de mener à bien ce projet d'études.*

*À mes amis, mes compagnons de route,
qui ont partagé mes doutes, mes fous rires et mes longues nuits de révision.
Merci pour votre présence rassurante, vos encouragements constants
et pour avoir rendu ces années de formation si précieuses et inoubliables.*

*Enfin, à toutes les personnes qui ont cru en moi et m'ont soutenu de près ou de loin.
Que ce mémoire soit le fruit de votre bienveillance,
le reflet de vos efforts combinés aux miens,
et l'expression de ma profonde et éternelle gratitude.*

Rania

Remerciements

Je tiens à exprimer ma sincère gratitude à mon encadrant, M. ANANE Mohamed, et à mon promoteur, M. NAÏT-ABDESSELAM Farid, pour leur disponibilité, leurs conseils avisés et leur accompagnement tout au long de ce travail.

Je remercie les membres du jury d'avoir accepté d'évaluer ce mémoire. Leur expertise et le temps qu'ils y consacrent sont pour moi un véritable honneur.

Je tiens aussi à remercier l'École Nationale Supérieure d'Informatique pour ces cinq années de formation enrichissante, tant sur le plan académique qu'humain. Les connaissances acquises, les rencontres faites et les expériences vécues au sein de cet établissement ont profondément marqué mon parcours. Je remercie également mes camarades de promotion pour la solidarité et l'entraide qui ont marqué ces années d'études.

Enfin, j'adresse mes remerciements à ma famille et à mes amis pour leur soutien et leurs encouragements tout au long de ce parcours.

Résumé

Les attaques par déni de service distribué (Distributed Denial of Service (DDoS)) constituent l'une des menaces les plus critiques pour la disponibilité des services numériques modernes. Face à l'augmentation exponentielle de leur volume et de leur sophistication, les approches de mitigation centralisées montrent leurs limites : points de défaillance uniques, coûts élevés et absence de coordination entre acteurs indépendants.

Ce mémoire a pour objectif de concevoir et d'implémenter un système distribué de mitigation collaborative des attaques DDoS, reposant sur une coalition pair-à-pair de nœuds de protection hétérogènes. Pour ce faire, le système intègre un mécanisme de confiance fondé sur le modèle PeerTrust avec crédibilité historique par session, un algorithme de sélection multicritère des pairs basé sur le Weighted Sum Model avec pondération Analytic Hierarchy Process (AHP), ainsi qu'un protocole de coordination P2P structuré en quatre phases permettant à un nœud victime de solliciter, négocier et clore une session d'aide sans coordinateur central.

Le prototype a été développé en Node.js avec PostgreSQL et évalué sur une coalition simulée à différentes échelles. Les tests confirment la tenue en charge du système ainsi que la gestion correcte des cycles d'aide de bout en bout, depuis la détection de l'attaque jusqu'au recalcul des scores de confiance.

Mots-clés : DDoS, mitigation collaborative, pair-à-pair, confiance, PeerTrust, Weighted Sum Model, mTLS, JWT, sécurité réseau.

Abstract

Distributed Denial of Service (DDoS) attacks represent one of the most critical threats to the availability of modern digital services. As their volume and sophistication grow exponentially, centralized mitigation approaches show their limitations : single points of failure, high costs, and lack of coordination between independent actors.

This thesis aims to design and implement a distributed collaborative DDoS mitigation system based on a peer-to-peer coalition of heterogeneous protection nodes. The system incorporates a trust mechanism based on the PeerTrust model with per-session historical credibility, a multi-criteria peer selection algorithm using the Weighted Sum Model with AHP weighting, and a P2P coordination protocol structured in four phases enabling a victim node to request, negotiate and close a help session without any central coordinator.

The prototype was developed in Node.js with PostgreSQL and evaluated on a simulated coalition at various scales. Tests confirm the system's ability to handle load and correctly manage end-to-end help cycles, from attack detection through trust score recalculation.

Keywords : DDoS, collaborative mitigation, peer-to-peer, trust, PeerTrust, Weighted Sum Model, mTLS, JWT, network security.

Table des matières

Dédicace	I
Remerciements	II
Résumé	III
Abstract	IV
Table des matières	IX
Liste des abréviations	X
Liste des figures	XIII
Liste des tableaux	XIV
Introduction générale	1
Contexte	2
Problématique	2
Objectifs	2
Périmètre du travail	3
Organisation du rapport	3
1 Contexte et concepts fondamentaux	5
1.1 Introduction	5
1.2 Les attaques par déni de service distribué (DDoS)	5
1.2.1 Définition	5
1.2.2 Architecture d'une attaque DDoS	6
1.2.3 Classification des attaques DDoS	6
1.2.3.1 Attaques volumétriques	6

TABLE DES MATIÈRES

1.2.3.2	Attaques sur les protocoles réseau	7
1.2.3.3	Attaques applicatives (couche 7)	7
1.2.4	Impact et données statistiques	8
1.3	Infrastructure de mitigation DDoS	8
1.3.1	Les <i>scrubbing centers</i> : définition et rôle	8
1.3.2	Les réseaux de diffusion de contenu (CDN) comme bouclier anti-DDoS	9
1.3.3	Les pare-feu et systèmes de prévention d'intrusion (IPS)	9
1.3.3.1	Pare-feu traditionnels et nouvelle génération (NGFW)	9
1.3.3.2	Systèmes de prévention d'intrusion (IPS)	10
1.3.4	Le <i>blackholing</i> et le RTBH (<i>Remote Triggered Black Hole</i>)	10
1.3.5	Les services de mitigation cloud (<i>Cloud-based DDoS Protection</i>)	11
1.3.6	Techniques de redirection du trafic	12
1.3.7	Synthèse et comparaison des infrastructures de mitigation	13
1.3.8	Limites des approches centralisées	13
1.4	Architecture pair-à-pair pour la coordination inter-nœuds	14
1.4.1	Définition et pertinence du modèle pair-à-pair	14
1.4.2	Topologies et protocoles P2P applicables à la mitigation	15
1.4.3	Défis du P2P appliqué à la sécurité réseau	15
1.5	Conclusion	16
2	Étude bibliographique	17
2.1	Introduction	17
2.2	DOTS — DDoS Open Threat Signaling	17
2.2.1	Contexte et motivation	17
2.2.2	Architecture et fonctionnement	17
2.2.2.1	Canal de signalisation (<i>Signal Channel</i>)	18
2.2.2.2	Canal de données (<i>Data Channel</i>)	18
2.2.3	Limites identifiées	19
2.3	DDoS Clearing House — Partage collaboratif de <i>fingerprints</i>	19
2.3.1	Contexte et motivation	19
2.3.2	Architecture et fonctionnement	20
2.3.2.1	Le <i>DDoS Fingerprint</i> : structure et contenu	20
2.3.2.2	Architecture logicielle	20
2.3.3	Limites identifiées	21
2.4	NaWas / NBIP — Centre national de <i>scrubbing</i> mutualisé	22

TABLE DES MATIÈRES

2.4.1	Contexte et architecture	22
2.4.1.1	Mécanisme de redirection BGP	22
2.4.2	Limites identifiées	23
2.5	DefCOM — <i>Distributed Cooperative Overlay Defense</i>	23
2.5.1	Contexte et architecture	23
2.5.1.1	Topologie et rôles des nœuds	24
2.5.1.2	Mécanisme de <i>Rate Limiting</i> coopératif	24
2.5.1.3	Protocole de communication overlay	24
2.5.2	Limites identifiées	25
2.6	PeerTrust — Modèle de confiance pour communautés pair-à-pair	26
2.6.1	Contexte et motivation	26
2.6.2	Paramètres de confiance et métrique générale	26
2.6.3	Architecture d'implémentation distribuée	27
2.6.4	Limites identifiées dans le contexte DDoS	28
2.7	Synthèse comparative	28
2.8	Conclusion	29
3	Conception et Architecture du système	30
3.1	Introduction	30
3.2	Conception préliminaire	30
3.2.1	Rappel du périmètre et des exigences	30
3.2.2	Architecture générale	31
3.2.2.1	Topologie de la coalition	31
3.2.2.2	Cycle de vie d'une session de mitigation	31
3.2.3	Périmètre du système	32
3.2.4	Diagrammes de cas d'utilisation	32
3.2.4.1	Protocole de mitigation collaborative	33
3.2.4.2	Moteur de confiance PeerTrust	34
3.2.4.3	Rôle du nœud pair dans la coalition	35
3.2.5	Documentation des cas d'utilisation	38
3.3	Conception technique détaillée	45
3.3.1	Modèle de données	45
3.3.1.1	Diagrammes de classes	45
3.3.1.2	Diagramme de classes — Configuration du nœud local	49
3.3.1.3	Diagramme de classes — Gestion des pairs et de la confiance	50

3.3.1.4	Identifiants et intégrité référentielle	52
3.3.1.5	Diagramme de classes — Sessions de mitigation et traçabilité	52
3.3.1.6	Passage au modèle relationnel	54
3.3.2	Modèle de confiance : PeerTrust adapté	55
3.3.2.1	Motivation du choix de PeerTrust	55
3.3.2.2	Formule $T(u)$ et ses paramètres	55
3.3.2.3	Niveaux de confiance	57
3.3.3	Algorithme de sélection des pairs : WSM	57
3.3.3.1	Critères de sélection	57
3.3.3.2	Pondération AHP	57
3.3.3.3	Répartition proportionnelle du flux	58
3.3.4	Diagramme d'activités : session de mitigation	59
3.3.5	Diagrammes de séquence	61
3.3.5.1	DS1 — Authentification JWT RS256 (CU1)	61
3.3.5.2	DS2 — Session de mitigation collaborative (CU4 + CU5 + CU6)	63
3.3.5.3	DS3 — Recalcul du score PeerTrust (CU7)	65
3.3.6	Mécanismes de sécurité	67
3.3.6.1	Transport sécurisé : mTLS	67
3.3.6.2	Authentification applicative : JWT RS256	67
3.3.7	Interface API REST	67
3.3.7.1	Style architectural	67
3.3.7.2	Endpoints	68
3.4	Conclusion	68
4	Implémentation et Évaluation	70
4.1	Introduction	70
4.2	Environnement technique	70
4.2.1	Langages de programmation	70
4.2.2	Outils de développement	70
4.2.3	Environnement de déploiement	71
4.2.4	Architecture de déploiement	71
4.2.5	Outils de collaboration	72
4.2.6	Structure du projet	73
4.3	Implémentation du gestionnaire de confiance	73

TABLE DES MATIÈRES

4.3.1	Calcul de $T(u)$ avec Cr historique	73
4.3.2	Capture de la crédibilité à la clôture	74
4.3.3	Persistance et sanctions automatiques	75
4.4	Implémentation de la sélection des pairs	75
4.4.1	Calcul du score WSM	75
4.4.2	Plan de répartition proportionnelle	75
4.4.3	Distribution des paquets par Round-Robin pondéré	76
4.5	Mécanismes de sécurité	77
4.5.1	JWT RS256 : génération et vérification	77
4.5.2	mTLS : configuration côté serveur	78
4.6	Protocole de communication inter-nœuds	78
4.6.1	Authentification	78
4.6.2	Heartbeat	79
4.6.3	Détection d'une attaque	79
4.6.4	Demande et réponse d'aide	80
4.6.5	Redirection du trafic et clôture	81
4.7	Tests et évaluation	82
4.7.1	Protocole de test	82
4.7.2	Validation du calcul PeerTrust	82
4.7.3	Résultats des tests de scalabilité	83
4.7.4	Analyse des résultats	84
4.7.5	Résultats des tests de sécurité	85
4.7.6	Démonstration sur le tableau de bord de supervision	85
4.8	Conclusion	89
Conclusion générale		91
Bibliographie		93

Liste des abréviations

Abréviation	Signification
ACL	Access Control List
AHP	Analytic Hierarchy Process
API	Application Programming Interface
AS	Autonomous System
BGP	Border Gateway Protocol
CBOR	Concise Binary Object Representation
CoAP	Constrained Application Protocol
DDoS	Distributed Denial of Service
DHT	Distributed Hash Table
DoS	Denial of Service
DOTS	DDoS Open Threat Signaling
DPI	Deep Packet Inspection
DTLS	Datagram Transport Layer Security
FAI	Fournisseur d'Accès Internet
GRE	Generic Routing Encapsulation
IoT	Internet of Things
JWT	JSON Web Token
mTLS	Mutual Transport Layer Security
NLRI	Network Layer Reachability Information
ORM	Object-Relational Mapping
P2P	Pair-à-Pair
REST	Representational State Transfer
RSA	Rivest-Shamir-Adleman
RTT	Round-Trip Time
SPOF	Single Point of Failure

Abréviation	Signification
TLS	Transport Layer Security
UUID	Universally Unique Identifier
WAF	Web Application Firewall
WSM	Weighted Sum Model

Table des figures

1.1	Architecture générale d'une attaque DDoS.	6
1.2	Processus de mitigation via un <i>scrubbing center</i>	8
1.3	Principe de protection anti-DDoS par un CDN avec Anycast.	9
1.4	Mécanisme de RTBH via BGP : la victime annonce un préfixe avec une communauté RTBH, et le FAI redirige le trafic correspondant vers <code>null0</code>	11
2.1	Architecture de référence du protocole DDoS Open Threat Signaling (DOTS) : séparation entre le canal de signalisation (CoAP/DTLS/UDP) et le canal de données (RESTCONF/HTTPS/TCP).	18
2.2	Architecture du <i>DDoS Clearing House</i> : génération d'empreintes DDoS par le composant Dissector, stockage dans la base DDoS DB, puis utilisation par l'équipe opérationnelle pour générer des règles de mitigation.	21
2.3	Architecture simplifiée du service NaWas : redirection du trafic vers un centre de scrubbing mutualisé via annonce BGP et analyse des flux avant application des mécanismes de mitigation.	23
2.4	Arbre de limitation de débit coopératif dans DefCOM : propagation de la demande de la victime vers les sources.	25
3.1	Exemple de coalition : sept nœuds hétérogènes interconnectés en overlay Pair-à-Pair (P2P) plat non structuré.	31
3.2	Architecture globale du système ShieldNet : périmètre applicatif et redirection réseau hors périmètre.	32
3.3	Diagramme de cas d'utilisation : Protocole de mitigation collaborative.	33
3.4	Diagramme de cas d'utilisation : Moteur de confiance PeerTrust.	35
3.5	Diagramme de cas d'utilisation : Rôle du nœud pair dans la coalition (12 cas d'usage).	37
3.6	Vue globale du diagramme de classes — les quatorze entités du système.	48
3.7	Diagramme de classes — Configuration du nœud local.	49
3.8	Diagramme de classes — Gestion des pairs et de la confiance.	50
3.9	Diagramme de classes — Sessions de mitigation et traçabilité.	53

3.10	Distribution du trafic selon l'algorithme WSM : répartition proportionnelle de l'overflow entre les pairs sélectionnés.	59
3.11	Diagramme d'activités d'une session de mitigation collaborative (deux couloirs : nœud victime et nœud pair).	60
3.12	DS1 — Diagramme de séquence : Authentification JWT RS256 (CU1). . .	62
3.13	DS2 — Diagramme de séquence : Session de mitigation collaborative (CU4 + CU5 + CU6).	64
3.14	DS3 — Diagramme de séquence : Recalcul du score PeerTrust (CU7). . . .	66
4.1	Diagramme de déploiement du prototype.	72
4.2	Structure des répertoires du projet.	73
4.3	Distribution des paquets par Round-Robin pondéré : le nœud victime N_v distribue n paquets vers m pairs en cycles successifs, chaque pair N_{p_i} recevant $alloc_i$ paquets par cycle.	77
4.4	Temps de réponse en fonction du nombre de nœuds simulés (enregistrement, heartbeat et sélection WSM).	84
4.5	Vue d'ensemble du tableau de bord : état de la coalition avant le lancement du scénario.	86
4.6	Tableau des pairs de la coalition triés par score PeerTrust décroissant. . . .	86
4.7	Configuration de l'attaque simulée et début du tableau de sélection WSM. . .	87
4.8	Tableau WSM complet : classement des 38 pairs éligibles avec scores et allocations.	87
4.9	Mitigation distribuée active : 52 tunnels GRE, 15,7 Gbps absorbés, recalcul PeerTrust sur 52 pairs.	88
4.10	Journal d'audit : événements TRUST_LEVEL_CHANGED à la clôture de l'attaque. .	88
4.11	Messages P2P : HELP_ACCEPT et TRAFFIC_REDIRECT échangés durant la mitigation.	89
4.12	Panel Heartbeats : battements périodiques des nœuds réels reçus par node-university.	89

Liste des tableaux

1.1	Synthèse comparative des catégories d'attaques DDoS.	7
1.2	Comparaison des principaux services de mitigation cloud.	12
1.3	Comparaison des infrastructures de mitigation DDoS.	13
2.1	Comparaison des systèmes existants et positionnement de la contribution. .	28
3.1	Identification des cas d'utilisation.	38
3.2	Documentation du cas d'utilisation <i>Obtenir un token JWT RS256</i>	39
3.3	Documentation du cas d'utilisation <i>Envoyer un heartbeat</i>	40
3.4	Documentation du cas d'utilisation <i>Envoyer une requête d'aide (sélection WSM)</i>	41
3.5	Documentation du cas d'utilisation <i>Accepter ou rejeter une session d'aide</i> . .	42
3.6	Documentation du cas d'utilisation <i>Clôturer l'attaque</i>	43
3.7	Documentation du cas d'utilisation <i>Recalculer le score PeerTrust</i>	44
3.8	Description des tables de la base de données.	46
3.9	Classes — Configuration du nœud local.	49
3.10	Classes — Gestion des pairs et de la confiance.	51
3.11	Classes — Sessions de mitigation et traçabilité.	54
3.12	Principales clés étrangères et politiques de suppression.	55
3.13	Facteur contextuel $D(u, i)$ selon la sévérité de l'attaque.	56
3.14	Niveaux de confiance dans le système.	57
3.15	Matrice AHP de comparaison des critères de sélection des pairs.	58
3.16	Endpoints principaux de l'API REST.	68
4.1	Scénarios de validation du calcul de $T(u)$	83
4.2	Résultats des tests de scalabilité (20 à 100 nœuds).	83
4.3	Tests de validation du mécanisme d'authentification	85

Introduction générale

La transformation numérique massive des organisations, l'essor du cloud et de l'Internet des objets (Internet of Things (IoT)) ont profondément renforcé la dépendance aux infrastructures réseau et aux systèmes d'information et technologie. La disponibilité continue des services numériques est devenue une exigence fondamentale, tant pour les acteurs économiques que pour les institutions et les utilisateurs. Dans cet environnement fortement interconnecté, toute interruption de service peut engendrer des conséquences opérationnelles, financières et réputationnelles significatives.

Les attaques par déni de service distribué (DDoS) constituent aujourd'hui l'une des principales menaces affectant la disponibilité des services numériques. Ces attaques consistent à générer un volume massif de trafic malveillant, distribué à partir de multiples sources, dans le but de saturer les ressources d'une cible et d'entraîner une indisponibilité partielle ou totale du service. Leur simplicité conceptuelle contraste avec leur efficacité opérationnelle, notamment lorsqu'elles exploitent des infrastructures distribuées à grande échelle et des botnets composés de milliers, voire de millions de dispositifs compromis.

Selon le rapport Cloudflare sur les menaces DDoS au premier trimestre 2025, 20,5 millions d'attaques ont été identifiées, soit une augmentation de 358 % par rapport à l'année précédente (CLOUDFLARE, 2025). Certains incidents récents ont atteint des volumes dépassant plusieurs téraoctets par seconde, illustrant la capacité des attaquants à mobiliser des botnets massifs et des infrastructures distribuées à grande échelle.

Les secteurs les plus ciblés incluent notamment la finance, les télécommunications, les services en ligne à forte disponibilité, ainsi que les infrastructures publiques et les plateformes de commerce électronique. Les attaques multi-vecteurs, combinant saturation volumétrique, exploitation de protocoles réseau et ciblage applicatif (couche 7), représentent désormais une part significative des incidents observés, rendant les mécanismes de défense traditionnels plus difficiles à maintenir efficaces.

Les solutions actuelles reposent majoritairement sur des architectures centralisées, telles que les *scrubbing centers* ou les services de mitigation cloud. Bien que ces solutions offrent une capacité de traitement élevée, elles présentent des limites structurelles : dépendance à un point unique de décision, latence due à la redirection du trafic, coûts élevés et absence de coordination dynamique entre acteurs indépendants.

Face à ces limites, une nouvelle approche émerge : la **défense collaborative distribuée**. Plutôt que de traiter les attaques de manière isolée, plusieurs entités autonomes peuvent coopérer afin de partager des informations, synchroniser leurs actions et renforcer leur résilience collective. C'est dans ce cadre que s'inscrit ce projet de fin d'études, visant à concevoir une infrastructure de coordination inter-nœuds pour la défense collaborative contre les attaques DDoS.

Contexte

Les solutions classiques de mitigation DDoS reposent majoritairement sur des architectures centralisées, telles que des centres de nettoyage du trafic (*scrubbing centers*) ou des services cloud spécialisés. Ces approches permettent d'absorber de forts volumes de trafic malveillant et d'appliquer des mécanismes de filtrage avancés. Toutefois, elles présentent plusieurs limites structurelles.

D'une part, la centralisation introduit des points de concentration critiques, susceptibles de devenir eux-mêmes des goulots d'étranglement ou des cibles d'attaque. D'autre part, les coûts associés à ces services peuvent être significatifs, ce qui limite leur accessibilité pour certaines organisations, notamment les PME et les collectivités. Enfin, la latence induite par la redirection du trafic vers des infrastructures distantes peut impacter la qualité de service.

Parallèlement, l'évolution des menaces complexifie la détection et la coordination des réponses. Les attaques modernes exploitent des botnets massifs, des vulnérabilités applicatives et des techniques d'évasion sophistiquées. Elles peuvent évoluer dynamiquement en fonction des contre-mesures mises en place. Face à ces attaques adaptatives, une simple réaction locale et isolée montre rapidement ses limites.

Dans ce cadre, l'idée d'une défense collaborative et distribuée — reposant sur la coordination entre plusieurs nœuds de protection capables d'échanger des informations en temps réel et de mutualiser leurs ressources — apparaît comme une piste pertinente et innovante.

Problématique

Les attaques DDoS atteignent aujourd'hui des volumes et une complexité qui dépassent les capacités de mitigation d'infrastructures isolées. Face à cette menace, les organisations utilisent principalement des solutions centralisées : *scrubbing centers* ou services cloud de mitigation.

Ces architectures présentent plusieurs limitations techniques : dépendance aux prestataires externes, coûts élevés pour les PME et collectivités, latence due à la redirection du trafic, points uniques de défaillance (Single Point of Failure (SPOF)) et absence de mutualisation des ressources de défense.

De plus, la défense reste individuelle. Les mécanismes actuels se limitent au partage d'informations, sans mutualisation opérationnelle des capacités de mitigation ni coordination des contre-mesures. En cas d'attaque distribuée touchant plusieurs cibles simultanément, chaque organisation réagit de son côté, ce qui réduit l'efficacité globale.

Objectifs

L'objectif principal de ce projet est de concevoir et d'implémenter un mécanisme de coordination inter-nœuds destiné à renforcer la défense collaborative contre les attaques DDoS.

Pour atteindre cet objectif, le travail consiste à :

1. Réaliser une étude approfondie des attaques DDoS, des mécanismes de mitigation existants et des approches collaboratives en environnement distribué.
2. Analyser les limites des solutions centralisées actuelles et identifier les besoins en matière de coordination et de partage de ressources entre nœuds autonomes.
3. Définir les principaux cas d'usage de coordination entre nœuds de protection, notamment l'alerte, l'entraide et la redirection de trafic.
4. Concevoir un protocole distribué d'échange inter-nœuds précisant les règles de communication et de prise de décision.
5. Développer un prototype permettant l'échange sécurisé de métadonnées d'attaque et la synchronisation des actions de mitigation entre plusieurs nœuds.
6. Mettre en place un environnement de test simulant un réseau de nœuds coopérants afin d'évaluer la robustesse, la latence et la cohérence des mécanismes proposés.
7. Réaliser une documentation technique détaillant l'architecture et le fonctionnement du système développé.

Périmètre du travail

Ce travail se concentre exclusivement sur les **échanges inter-nœuds** au sein d'une coalition de défense déjà constituée. Plus précisément, le périmètre couvre la conception et l'implémentation de l'**API REST** exposée par chaque nœud pour la coordination collaborative, le **protocole d'aide** entre nœuds (signalement d'attaque, demande et acceptation de sessions de mitigation, redirection de trafic, clôture), le **mécanisme de confiance** fondé sur PeerTrust (calcul des scores, détection des violations, exclusion automatique) ainsi que la **sécurisation des communications** inter-nœuds par mTLS et JWT RS256.

Les aspects suivants sont explicitement **hors périmètre** : la formation du réseau P2P (bootstrap, découverte initiale des pairs, protocoles de routage overlay), la détection technique des attaques DDoS au niveau réseau (analyse de trafic, deep packet inspection), le filtrage et le nettoyage effectif du trafic (scrubbing) au sein de chaque nœud, ainsi que la gestion des tunnels de redirection au niveau infrastructure (configuration GRE, BGP Flowspec).

Organisation du rapport

Le présent mémoire est structuré en quatre chapitres. **Le premier chapitre** présente les fondements théoriques des attaques DDoS, la taxonomie des vecteurs d'attaque et les mécanismes de mitigation existants (*scrubbing centers*, CDN, pare-feu, RTBH, services cloud), ainsi que les principes des architectures pair-à-pair applicables à la coordination inter-nœuds. **Le deuxième chapitre** analyse les principales approches collaboratives et distribuées de défense contre les attaques DDoS : le protocole de signalisation DOTS, la DDoS Clearing House, la mutualisation opérationnelle NaWas/NBIP et l'overlay coopératif DefCOM ; une synthèse comparative identifie les lacunes motivant la contribution de ce mémoire. **Le troisième chapitre** décrit les décisions de conception du système : topologie

P2P de la coalition, modèle de données, mécanisme de confiance fondé sur PeerTrust, algorithme de sélection des pairs par Weighted Sum Model (WSM) avec pondération AHP, protocoles de sécurité mTLS et JWT RS256, et interface Application Programming Interface (API) Representational State Transfer (REST). **Le quatrième chapitre** présente la réalisation concrète du prototype : pile technologique, implémentation du gestionnaire de confiance et du sélecteur de pairs, configuration de la sécurité, et résultats des tests de scalabilité conduits sur une coalition simulée jusqu'à 100 nœuds.

Enfin, une **conclusion générale** synthétise les contributions du travail et propose des perspectives d'évolution.

Chapitre 1

Contexte et concepts fondamentaux

1.1 Introduction

La sécurité des infrastructures réseau est aujourd’hui l’un des enjeux majeurs de l’informatique moderne. Parmi les menaces les plus dévastatrices, les attaques par déni de service distribué (DDoS) occupent une place prépondérante, en raison de leur capacité à paralyser des services critiques en saturant les ressources d’un ou plusieurs systèmes cibles.

Face à l’augmentation en volume et en sophistication de ces attaques, les approches de mitigation centralisées montrent rapidement leurs limites. Il devient alors indispensable d’envisager des mécanismes de défense collaborative et distribuée, où plusieurs nœuds de protection — notamment des *scrubbing centers* — coordonnent leurs actions en temps réel pour détecter, analyser et contrer ces menaces de manière collective.

Ce chapitre a pour objectif de poser les bases conceptuelles nécessaires à la compréhension du travail présenté dans ce mémoire. J’aborde successivement la nature et la classification des attaques DDoS, les mécanismes d’infrastructure de mitigation existants, puis les principes des systèmes distribués appliqués à la coordination inter-nœuds.

1.2 Les attaques par déni de service distribué (DDoS)

1.2.1 Définition

Une attaque par déni de service (Denial of Service (DoS)) désigne toute action malveillante visant à rendre un service informatique indisponible pour ses utilisateurs légitimes, en épuisant les ressources du système cible (bande passante, mémoire, CPU, connexions). Lorsque cette attaque est menée simultanément depuis un grand nombre de machines distribuées, on parle d’attaque DDoS (MIRKOVIC & REIHER, 2004).

Mirkovic et Reiher définissent une attaque DDoS comme : « une tentative explicite d’un attaquant de nier aux utilisateurs légitimes l’utilisation d’un service réseau, en submergeant la cible ou son infrastructure intermédiaire avec du trafic illégitime » (MIRKOVIC & REIHER, 2005).

1.2.2 Architecture d’une attaque DDoS

Une attaque DDoS repose sur une architecture à plusieurs niveaux impliquant différents acteurs (voir Figure 1.1). **L’attaquant (*handler*)** est l’entité malveillante qui orchestre l’attaque depuis une machine maîtresse. Il s’appuie sur un **réseau de commande et contrôle (C&C)**, infrastructure permettant de coordonner les agents d’attaque à distance, souvent dissimulée via des protocoles légitimes (IRC, HTTP, P2P). Les **agents (*zombies* / *bots*)** sont des machines compromises — ordinateurs, serveurs, objets connectés IoT — qui exécutent les instructions reçues du C&C et génèrent le trafic malveillant. Enfin, **la cible (*victim*)** est le système dont on cherche à épuiser les ressources : serveur web, infrastructure DNS, routeurs de transit, etc. Specht et Lee (SPECHT & LEE, 2004) soulignent que la compromission de milliers voire de millions de machines forme des botnets dont la puissance agrégée dépasse largement la capacité de réponse d’un seul nœud de défense ; le botnet Mirai (ANTONAKAKIS et al., 2017) en constitue l’exemple emblématique, ayant recruté plus de 600 000 objets connectés IoT pour lancer des attaques dépassant 1 Tbps, justifiant ainsi l’intérêt d’une défense distribuée et collaborative.

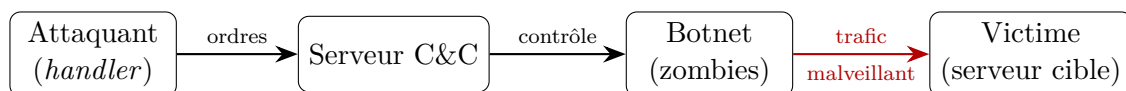


FIGURE 1.1 – Architecture générale d’une attaque DDoS.

1.2.3 Classification des attaques DDoS

La littérature propose plusieurs taxonomies des attaques DDoS (MIRKOVIC & REIHER, 2004 ; ZARGAR et al., 2013). J’adopte ici la classification en trois grandes catégories, largement répandue dans les travaux académiques et industriels. Le Tableau 1.1 en offre une synthèse comparative.

1.2.3.1 Attaques volumétriques

Ces attaques visent à saturer la bande passante du lien réseau de la victime ou de son Fournisseur d’Accès Internet (FAI). Elles constituent la catégorie la plus répandue et la plus facile à générer avec un botnet. **L’UDP Flood** repose sur l’envoi massif de paquets UDP vers des ports aléatoires de la cible ; l’effet principal réside dans la saturation de la bande passante par le volume brut du trafic entrant (MIRKOVIC & REIHER, 2004). **L’ICMP Flood (*Ping Flood*)** procède de façon similaire par saturation via des requêtes ICMP Echo Request en volume. Les **attaques par amplification/réflexion** exploitent quant à elles des serveurs tiers (DNS, NTP, memcached) pour amplifier le volume du trafic : l’attaquant envoie des requêtes à adresse source *spoofée* (celle de la victime) vers des serveurs ouverts, qui répondent avec des paquets de taille bien supérieure. Selon Rossow (ROSSOW, 2014), le facteur d’amplification DNS peut atteindre $28\times$ à $54\times$, celui de NTP jusqu’à $556\times$, et les attaques via memcached ont démontré des facteurs dépassant $50\,000\times$ (CHUA, 2018).

1.2.3.2 Attaques sur les protocoles réseau

Ces attaques exploitent des faiblesses dans les protocoles de communication (TCP, ICMP) pour épuiser les ressources de traitement des équipements réseau intermédiaires. Le **SYN Flood** exploite le mécanisme de *handshake* TCP en trois voies : l'attaquant envoie des paquets SYN avec des adresses source *spoofées*, forçant le serveur à maintenir en mémoire des connexions semi-ouvertes jusqu'à épuisement de la table d'état (*backlog*). Eddy (EDDY, 2007) décrit en détail les vulnérabilités du protocole TCP ainsi exploitées. La **Smurf Attack** procède différemment : des requêtes ICMP *broadcast* sont envoyées avec l'adresse source *spoofée* de la victime, générant une avalanche de réponses vers cette dernière.

Note historique : Le *Ping of Death*, qui consistait à envoyer des paquets ICMP de taille supérieure au maximum autorisé, a constitué une menace dans les années 1990 mais est aujourd'hui corrigé.

1.2.3.3 Attaques applicatives (couche 7)

Ces attaques ciblent la couche application du modèle OSI et sont particulièrement difficiles à distinguer du trafic légitime, car elles utilisent des requêtes HTTP, HTTPS, DNS valides du point de vue syntaxique (ZARGAR et al., 2013). Le **L'HTTP Flood** consiste en l'envoi massif de requêtes HTTP GET ou POST vers un serveur web, saturant sa capacité de traitement sans nécessiter un volume de trafic important. **Slowloris** adopte une approche plus subtile : il maintient un maximum de connexions HTTP ouvertes en envoyant des en-têtes partiels à faible débit, empêchant ainsi le serveur d'accepter de nouvelles connexions légitimes. Le **DNS Query Flood** procède par envoi massif de requêtes DNS syntaxiquement valides pour épuiser les ressources des serveurs de noms.

TABLE 1.1 – Synthèse comparative des catégories d'attaques DDoS.

Caractéristique	Volumétrique	Protocole	Applicative (L7)
Couche OSI ciblée	3–4 (Réseau /Transport)	3–4 (Réseau /Transport)	7 (Application)
Objectif principal	Saturer la bande passante	Épuiser les tables d'état	Épuiser les ressources applicatives
Volume requis	Très élevé (Gbps–Tbps)	Modéré	Faible à modéré
Difficulté de détection	Faible (anomalie volumétrique)	Moyenne	Élevée (trafic légitime en apparence)
Exemples	UDP Flood, Amplification	SYN Flood, Smurf	HTTP Flood, Slowloris
Mitigation typique	Filtrage, <i>blackholing</i>	SYN cookies, <i>rate limiting</i>	Analyse comportementale, WAF

1.2.4 Impact et données statistiques

Les attaques DDoS engendrent des pertes économiques considérables. Selon le rapport NETSCOUT Threat Intelligence Report 2023, plus de 7,9 millions d’attaques DDoS ont été enregistrées sur la seule première moitié de l’année 2023, soit une moyenne de plus de 44 000 attaques par jour (NETSCOUT SYSTEMS, 2023). Zargar et al. (ZARGAR et al., 2013) estiment que le coût moyen d’une attaque DDoS pour une entreprise peut dépasser 50 000 dollars par heure d’interruption de service, incluant les pertes commerciales directes, les coûts de mitigation et les dommages réputationnels.

1.3 Infrastructure de mitigation DDoS

La mitigation des attaques DDoS repose sur un ensemble d’infrastructures et de techniques déployées à différents niveaux du réseau. Cette section présente les principales solutions existantes, leurs mécanismes de fonctionnement et leurs limites respectives.

1.3.1 Les *scrubbing centers* : définition et rôle

Un *scrubbing center* (ou centre de nettoyage du trafic) est une infrastructure spécialisée dont la mission est d’analyser le trafic réseau entrant, de filtrer les paquets malveillants, puis de renvoyer uniquement le trafic légitime vers la destination finale. Ce mécanisme est souvent désigné sous le terme de « *clean pipe* » (MAHAJAN et al., 2002).

Le processus de mitigation via *scrubbing center* se déroule en plusieurs étapes distinctes (voir Figure 1.2) :

1. **Détection** : une anomalie dans le trafic déclenche une alerte (analyse NetFlow/sFlow, seuils de bande passante, corrélation multi-sources).
2. **Redirection** : le trafic destiné à la victime est redirigé vers le *scrubbing center* via des mécanismes de routage (Border Gateway Protocol (BGP), DNS).
3. **Analyse et filtrage** : le centre analyse le trafic en appliquant un pipeline de traitement multi-couches (empreintes, comportement, limitation de débit).
4. **Réinjection** : le trafic nettoyé est retransmis à la victime via un tunnel Generic Routing Encapsulation (GRE) ou MPLS pour éviter les boucles de routage.

Parmi les principaux fournisseurs de *scrubbing centers*, on trouve des acteurs comme Arbor Networks (Sightline/TMS), Akamai (Prolexic), Radware (DefensePro) et A10 Networks (Thunder TPS). Ces solutions offrent des capacités de traitement allant de quelques dizaines de Gbps à plusieurs Tbps selon l’infrastructure déployée.

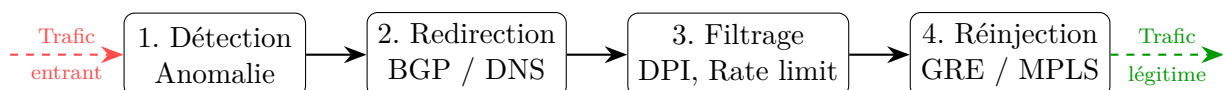


FIGURE 1.2 – Processus de mitigation via un *scrubbing center*.

1.3.2 Les réseaux de diffusion de contenu (CDN) comme bouclier anti-DDoS

Les **CDN** (*Content Delivery Networks*) tels que Cloudflare, Akamai et AWS CloudFront constituent une autre ligne de défense majeure contre les attaques DDoS. Leur principe repose sur la **distribution géographique** du contenu et du traitement du trafic à travers un réseau mondial de points de présence (PoP).

Le mécanisme de protection d'un CDN fonctionne comme suit :

1. **Interception par DNS** : les enregistrements DNS de la victime sont modifiés pour pointer vers les serveurs du CDN. Tout le trafic entrant transite donc par le réseau du CDN avant d'atteindre le serveur d'origine.
2. **Absorption volumétrique** : grâce à la technique **Anycast**, le même préfixe IP est annoncé depuis des centaines de PoP répartis mondialement. Le trafic est automatiquement dirigé vers le PoP le plus proche géographiquement, ce qui **distribue naturellement** la charge d'une attaque volumétrique sur l'ensemble du réseau.
3. **Filtrage en bordure** : chaque PoP applique des règles de filtrage (limitation de débit, WAF, challenges JavaScript/CAPTCHA) pour bloquer le trafic malveillant avant qu'il n'atteigne le serveur d'origine.
4. **Transmission du trafic légitime** : seul le trafic validé est transmis au serveur d'origine via un canal sécurisé (HTTPS).

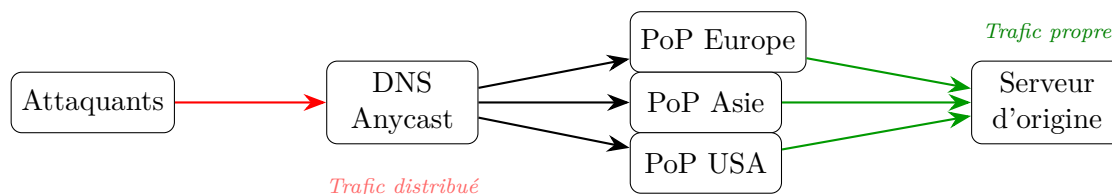


FIGURE 1.3 – Principe de protection anti-DDoS par un CDN avec Anycast.

Cependant, les CDN présentent des limites notables. Leur protection est principalement cantonnée à la **couche applicative (HTTP/HTTPS)** : les attaques ciblant d'autres protocoles (DNS autoritaire, VoIP, jeux en ligne, VPN) n'en bénéficient pas. Par ailleurs, l'ensemble du trafic transitant par l'infrastructure du CDN crée une **dépendance forte à un fournisseur tiers**, avec les risques de confidentialité associés. Enfin, la tarification au volume peut s'avérer prohibitive lors d'attaques massives prolongées.

1.3.3 Les pare-feu et systèmes de prévention d'intrusion (IPS)

Les **pare-feu** (*firewalls*) et les **systèmes de prévention d'intrusion** (*Intrusion Prevention Systems* — IPS) constituent la première ligne de défense déployée en périphérie du réseau de l'organisation.

1.3.3.1 Pare-feu traditionnels et nouvelle génération (NGFW)

Les pare-feu filtrent le trafic réseau en fonction de règles prédéfinies basées sur les adresses IP, les ports et les protocoles. Les **pare-feu de nouvelle génération** (NGFW)

— *Next-Generation Firewall*) ajoutent des capacités d'inspection applicative. **L'inspection à états (*stateful inspection*)** assure le suivi des connexions TCP pour détecter les anomalies telles que les SYN Flood ou les connexions incomplètes. **L'inspection applicative (couche 7)** analyse le contenu des paquets pour identifier les requêtes malveillantes (injections SQL, requêtes HTTP anormales). Enfin, la **limitation de débit (*rate limiting*)** restreint le nombre de connexions acceptables par source IP et par unité de temps.

1.3.3.2 Systèmes de prévention d'intrusion (IPS)

Les IPS analysent le trafic en temps réel pour détecter et bloquer les comportements malveillants selon trois approches complémentaires. La première repose sur la **correspondance de signatures** : le trafic est comparé à une base de signatures d'attaques connues, à l'image des antivirus. La deuxième procède par **détection d'anomalies** : tout écart significatif par rapport à un profil de trafic normal (*baseline*) déclenche une alerte. La troisième mobilise des **heuristiques comportementales** combinant plusieurs indicateurs pour identifier des attaques inconnues (*zero-day*).

Limites face aux attaques DDoS : les pare-feu et IPS sont dimensionnés pour le volume de trafic *normal* de l'organisation. Lors d'une attaque volumétrique, le pare-feu lui-même devient un goulot d'étranglement : sa capacité de traitement (mesurée en paquets par seconde — pps) est rapidement dépassée, ce qui peut provoquer l'effondrement de l'ensemble du périmètre de sécurité. De plus, les pare-feu à états consomment de la mémoire pour chaque connexion suivie, les rendant particulièrement vulnérables aux attaques d'épuisement de tables d'état (SYN Flood, TCP connection exhaustion).

1.3.4 Le *blackholing* et le RTBH (*Remote Triggered Black Hole*)

Le ***blackholing*** est une technique de dernier recours qui consiste à rediriger l'intégralité du trafic destiné à une adresse IP attaquée vers une route nulle (`null0` / *bit bucket*), supprimant ainsi tout le trafic — légitime et malveillant — vers cette destination.

Le mécanisme **RTBH** (*Remote Triggered Black Hole*, RFC 5635 (KUMARI & MCPHERSON, 2009)) permet d'automatiser ce processus via BGP :

1. Le réseau victime annonce le préfixe attaqué avec une **communauté BGP spéciale** (ex. : `AS:666`) à ses fournisseurs de transit.
2. Les routeurs des fournisseurs de transit, configurés pour reconnaître cette communauté, redirigent le trafic vers `null0`.
3. Le trafic d'attaque est ainsi éliminé *en amont*, avant qu'il ne sature les liens du réseau victime.

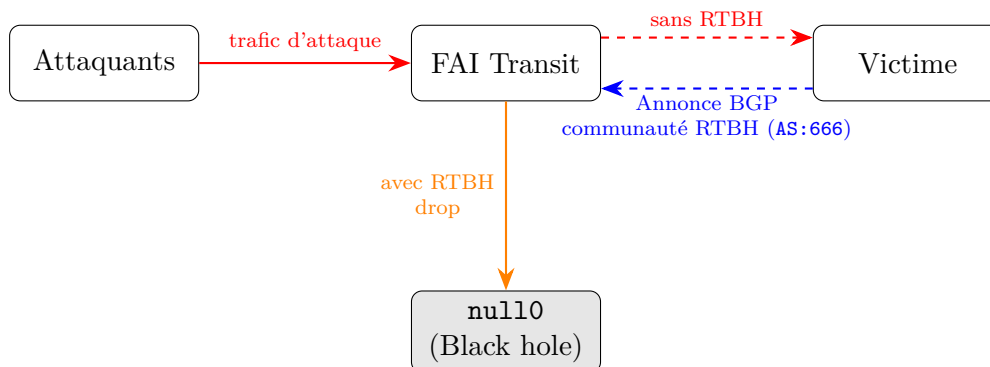


FIGURE 1.4 – Mécanisme de RTBH via BGP : la victime annonce un préfixe avec une communauté RTBH, et le FAI redirige le trafic correspondant vers `null0`.

Avantage : le *blackholing* est extrêmement efficace pour protéger l'infrastructure réseau (les liens ne sont plus saturés).

Inconvénient majeur : il constitue une **victoire de l'attaquant**. En supprimant tout le trafic vers la cible — y compris le trafic légitime —, le *blackholing* produit exactement le résultat recherché par l'attaque : le déni de service. C'est pourquoi cette technique n'est utilisée qu'en dernier recours, lorsque le sacrifice d'un service est préférable à l'effondrement de l'infrastructure complète.

1.3.5 Les services de mitigation cloud (*Cloud-based DDoS Protection*)

Les **services de mitigation cloud** sont proposés par des fournisseurs spécialisés (Cloudflare, AWS Shield, Azure DDoS Protection, Google Cloud Armor) qui disposent d'une infrastructure mondiale massive pour absorber les attaques DDoS.

Le fonctionnement repose sur le même principe que les *scrubbing centers*, mais avec deux différences majeures. D'une part, leur **capacité élastique** est sans commune mesure : les fournisseurs cloud disposent de capacités de traitement pouvant dépasser 100 Tbps en cumulé sur l'ensemble de leurs PoP, ce qui leur permet d'absorber même les attaques les plus volumétriques. D'autre part, le **modèle *as-a-Service*** propose la mitigation sous forme d'abonnement, évitant à l'organisation tout investissement dans une infrastructure physique dédiée.

Le Tableau 1.2 compare les principaux services de mitigation cloud disponibles sur le marché.

TABLE 1.2 – Comparaison des principaux services de mitigation cloud.

Service	Capacité	Couches protégées	Activation
Cloudflare (CLOUDFLARE, 2025)	348 Tbps (réseau global)	L3/L4 + L7 (WAF intégré)	Always-on (automatique)
AWS Shield Advanced (AMAZON WEB SERVICES, 2024)	Élastique (infra AWS)	L3/L4 + L7 (avec WAF)	Always-on + sur demande
Azure DDoS Protection (MICROSOFT AZURE, 2024)	Élastique (infra Azure)	L3/L4	Always-on
Google Cloud Armor (GOOGLE CLOUD, 2024)	Élastique (infra Google)	L7 (HTTP/HTTPS)	Always-on
Akamai Prolexic	20+ Tbps (scrubbing dédié)	L3/L4 + L7	On-demand (BGP redirect)

Ces services présentent néanmoins plusieurs limites importantes. En confiant l'intégralité de son trafic à un tiers, l'organisation crée une **dépendance au fournisseur** qui soulève des questions de confidentialité, de souveraineté des données et de *vendor lock-in*. Les **coûts récurrents** peuvent par ailleurs s'avérer prohibitifs pour les petites structures : AWS Shield Advanced, par exemple, est facturé 3 000 \$/mois auxquels s'ajoutent les coûts de transfert de données. Le reroutage du trafic via le réseau du fournisseur introduit également une **latence additionnelle** (typiquement 5 à 30 ms selon la distance au PoP le plus proche). Enfin, et c'est peut-être la limite la plus structurelle, chaque service opère de manière isolée : il n'existe à ce jour aucun mécanisme standardisé permettant à des acteurs comme Cloudflare, AWS et Akamai de coordonner leurs actions face à une attaque ciblant simultanément plusieurs de leurs clients.

1.3.6 Techniques de redirection du trafic

La redirection du trafic vers une infrastructure de mitigation (qu'il s'agisse d'un *scrubbing center*, d'un CDN ou d'un service cloud) est une opération critique. Plusieurs techniques sont utilisées en pratique. Le **BGP Blackholing** et **BGP Flowspec** constituent la première approche : le protocole BGP (RFC 4271 (REKHTER et al., 2006)) permet d'annoncer des préfixes IP pour rediriger le trafic, tandis que le BGP Flowspec (RFC 5575 (MARQUES et al., 2009)) étend cette capacité en distribuant des règles de filtrage fin (correspondance sur les champs source/destination IP, port, protocole, taille de paquet, fragment) sous forme de Network Layer Reachability Information (NLRI) multi-champs, permettant aux routeurs de bordure d'appliquer des actions (**discard**,

rate-limit, **redirect** vers VRF) sans intervention manuelle. La **DNS Redirection** consiste à modifier les enregistrements DNS de la victime pour les faire pointer vers l'infrastructure de mitigation ; c'est la technique privilégiée par les CDN (Cloudflare, Akamai) et les services cloud. L'**Anycast** repose sur le déploiement du même préfixe IP sur plusieurs nœuds distribués géographiquement, permettant une absorption distribuée naturelle : le trafic est dirigé vers le nœud le plus proche selon la topologie BGP. Enfin, les **tunnels GRE/MPLS** permettent d'encapsuler le trafic nettoyé pour le renvoyer à la victime via un chemin distinct (*clean-return path*), évitant ainsi les boucles de routage.

1.3.7 Synthèse et comparaison des infrastructures de mitigation

Le Tableau 1.3 compare les différentes infrastructures de mitigation présentées dans cette section selon plusieurs critères clés.

TABLE 1.3 – Comparaison des infrastructures de mitigation DDoS.

Solution	Couches	Capacité	Avantage principal	Limite principale
Scrubbing center	L3/L4/L7	Élevée (Tbps)	Filtrage fin multi-couches	Coût, latence de redirection
CDN (Anycast)	L7 (HTTP)	Très élevée	Distribution géographique	Limité au trafic web
Pare-feu / IPS	L3/L4/L7	Limitée (Gbps)	Proximité, contrôle local	Goulot d'étranglement
RTBH Blackholing	L3	Illimitée	Protège l'infrastructure	Supprime le trafic légitime
Service cloud	L3/L4/L7	Élastique	Capacité massive, as-a-Service	Dépendance, coût, confidentialité

1.3.8 Limites des approches centralisées

Bien que les solutions présentées ci-dessus offrent chacune des capacités de mitigation significatives, elles partagent un trait commun : elles opèrent de manière **centralisée et isolée**. Chaque organisation déploie ses propres défenses ou souscrit à un service individuel, sans mécanisme de coordination avec les autres entités potentiellement touchées par la même attaque.

Cette approche présente plusieurs limites fondamentales (DOULIGERIS & MITROKOTSA, 2004). Toute architecture centralisée introduit d'abord un **point de défaillance unique (SPOF)** : que ce soit le *scrubbing center*, le contrôleur SDN ou le fournisseur cloud, la défaillance de ce composant central compromet l'ensemble de la protection. S'y ajoute une **latence de redirection** inhérente au reroutage du trafic vers une infrastructure distante (10 à 50 ms), qui dégrade la qualité de service pour les applications sensibles telles que la VoIP, les jeux en ligne ou le trading haute fréquence. La **capacité finie** de ces solutions constitue une autre vulnérabilité : même les plus grandes infrastructures atteignent

un plafond, au-delà duquel la solution centralisée est elle-même submergée. Par ailleurs, chaque solution ne dispose que d'une **visibilité locale** sur le trafic qui lui est redirigé, sans possibilité de corrélérer ses observations avec celles d'autres infrastructures pour détecter des attaques coordonnées ciblant simultanément plusieurs victimes. L'**absence de mutualisation** aggrave encore cette situation : les ressources de mitigation de différentes organisations ne sont pas partagées, si bien qu'un *scrubbing center* sous-utilisé ne peut pas venir en aide à un voisin surchargé. Enfin, les **coûts prohibitifs** du dimensionnement d'une infrastructure capable d'absorber les attaques les plus volumétriques (multi-Tbps) la rendent inaccessible aux PME et aux petites collectivités.

Ces limitations convergent vers un constat : **la défense isolée ne suffit plus**. La réponse aux attaques DDoS modernes nécessite une approche **collaborative et distribuée**, où plusieurs entités autonomes coordonnent leurs actions en temps réel pour mutualiser leurs capacités de détection, d'analyse et de filtrage. C'est cette approche que je développe dans ce mémoire.

1.4 Architecture pair-à-pair pour la coordination inter-nœuds

1.4.1 Définition et pertinence du modèle pair-à-pair

Une architecture P2P est un modèle de système distribué dans lequel les nœuds (ou pairs) sont logiquement équivalents et peuvent assumer simultanément les rôles de client et de serveur. Contrairement aux architectures centralisées traditionnelles, aucun nœud ne joue le rôle d'autorité de contrôle unique (TANENBAUM & VAN STEEN, 2017). Chaque pair peut ainsi fournir des services aux autres tout en consommant leurs ressources.

Dans le contexte de la mitigation des attaques DDoS, ce modèle apparaît particulièrement pertinent pour répondre aux limites des architectures centralisées identifiées précédemment (cf. section 1.3.8). En structurant différents mécanismes de défense — tels que les *scrubbing centers*, les pare-feu ou les systèmes de détection — au sein d'un réseau logique superposé (*overlay network*), l'architecture P2P permet de répartir les capacités de détection et de filtrage entre plusieurs entités autonomes.

Une telle organisation repose sur trois principes fondamentaux. La **décentralisation** d'abord : l'absence de point de contrôle central élimine les points de défaillance uniques (SPOF) et renforce la résilience globale de l'infrastructure de défense. Le principe d'**autonomie et de coopération** ensuite : chaque nœud conserve la maîtrise de ses ressources locales tout en participant à un mécanisme de défense collectif, ce qui permet de mutualiser les capacités de détection et de mitigation. Enfin, la **symétrie des rôles** garantit qu'un même pair peut à la fois signaler une attaque lorsqu'il en est victime et contribuer à la mitigation en appliquant des règles de filtrage ou en relayant des informations vers d'autres pairs.

Cette approche ouvre la voie à des mécanismes de défense collaborative où plusieurs organisations peuvent coordonner leurs actions afin de limiter l'impact d'attaques distribuées à grande échelle.

1.4.2 Topologies et protocoles P2P applicables à la mitigation

Les systèmes P2P peuvent adopter différentes formes d'organisation, dont l'efficacité varie selon les objectifs et les contraintes du système distribué (TANENBAUM & VAN STEEN, 2017). Dans le cadre d'une infrastructure de défense collaborative, plusieurs modèles sont particulièrement pertinents.

Dans le modèle **P2P non structuré**, les nœuds établissent des connexions de manière ad hoc sans structure globale prédéfinie. La recherche d'information ou la diffusion d'alertes s'effectue généralement par inondation (*flooding*). Bien que cette approche soit très robuste face à la disparition de pairs, elle entraîne une surcharge réseau importante et devient difficilement scalable dans des environnements comportant un grand nombre de participants.

Les **architectures structurées (P2P avec Distributed Hash Table (DHT))** reposent quant à elles sur une topologie logique déterministe. Des systèmes tels que Chord (STOICA et al., 2001) permettent de localiser une information en $O(\log N)$ sauts. Dans le contexte de la défense DDoS, une DHT peut être utilisée pour maintenir une base de données distribuée contenant des indicateurs de compromission, des signatures d'attaques ou des listes d'adresses IP malveillantes partagées entre participants.

Les **architectures hybrides** introduisent la notion de *super-pairs* : certains nœuds disposant de capacités importantes (par exemple des *scrubbing centers* ou des nœuds cœur de réseau d'un FAI) jouent un rôle de coordination ou d'agrégation, tandis que des nœuds périphériques participent à l'échange d'informations avec des capacités plus limitées. Ce modèle permet d'améliorer l'efficacité du routage des messages tout en conservant un degré de décentralisation.

Les **protocoles épidémiques (*gossip*)** reposent sur une diffusion probabiliste de l'information : chaque nœud transmet périodiquement son état ou une alerte à un sous-ensemble aléatoire de ses voisins. Cette approche offre une forte tolérance aux pannes et permet une propagation rapide des informations, généralement en $O(\log N)$ cycles de communication. Dans le cadre d'une défense collaborative, les mécanismes *gossip* sont particulièrement adaptés pour diffuser rapidement des alertes d'attaque ou des règles de mitigation sans saturer le réseau de communication.

1.4.3 Défis du P2P appliqué à la sécurité réseau

Malgré ses avantages, l'application d'un modèle distribué à la coordination d'infrastructures de défense appartenant à des entités indépendantes soulève plusieurs défis majeurs.

La **gestion de la confiance** constitue le premier défi majeur : dans un système décentralisé, l'absence d'autorité centrale complique la validation de l'identité et de la fiabilité des pairs. Un nœud compromis pourrait diffuser de fausses alertes afin de provoquer un blocage injustifié du trafic légitime (*blackholing* offensif) ou créer de multiples identités afin d'influencer les décisions collectives (attaque *Sybil*). La mise en place de mécanismes d'authentification forte ou de systèmes de réputation devient alors indispensable.

La **latence de convergence** pose un problème tout aussi critique : les attaques DDoS volumétriques peuvent saturer les ressources d'une cible en quelques secondes. La

propagation des alertes et la synchronisation des mécanismes de filtrage entre pairs doivent donc être extrêmement rapides afin que la mitigation distribuée puisse être effective avant l’effondrement du service attaqué.

Le **problème du passager clandestin** (*free-rider*) est inhérent à tout système coopératif ouvert : la mitigation d’attaques distribuées nécessite des ressources importantes en bande passante et en capacité de traitement, et certains participants pourraient bénéficier de la protection collective sans contribuer réellement à l’effort de mitigation. Des mécanismes d’incitation à la coopération, tels que des quotas de contribution ou des systèmes de réputation, peuvent être nécessaires pour limiter ce comportement.

Enfin, le **passage à l’échelle sous contrainte réseau** représente un défi opérationnel de premier plan. Lors d’une attaque de grande ampleur, le plan de données du réseau peut subir une congestion importante. Le réseau de coordination P2P — qui constitue le plan de contrôle du système — doit donc rester opérationnel même en présence de pertes de paquets élevées ou de fortes variations de latence.

Ces défis expliquent pourquoi la mise en œuvre de mécanismes de défense véritablement distribués reste complexe dans la pratique. Les approches existantes tentant d’exploiter ces principes seront analysées dans le chapitre suivant à travers l’étude de plusieurs systèmes représentatifs de la littérature.

1.5 Conclusion

Ce chapitre a posé les fondements nécessaires à la compréhension du travail présenté dans ce mémoire. J’ai défini les attaques DDoS et leurs catégories (volumétriques, protocolaires, applicatives), décrit l’infrastructure de mitigation actuelle (*scrubbing centers*) et ses limites centralisées, puis introduit l’architecture P2P comme solution potentielle de coordination entre nœuds de défense.

Le chapitre suivant analyse les principales approches existantes en matière de défense collaborative contre les attaques DDoS.

Chapitre 2

Étude bibliographique

2.1 Introduction

La défense collaborative contre les attaques DDoS a donné lieu à une littérature académique riche ainsi qu'à plusieurs déploiements institutionnels et industriels. Ce chapitre analyse quatre contributions majeures, sélectionnées pour leur représentativité des différentes approches existantes : un standard de signalisation inter-domaines (DOTS), une coalition nationale de partage de métadonnées (DDoS Clearing House), une mutualisation opérationnelle de *scrubbing centers* (NaWas/NBIP), et un overlay coopératif distribué (DefCOM).

Pour chaque travail, je décris le contexte de sa conception, son architecture technique détaillée, ses mécanismes protocolaires et ses limites identifiées. Une synthèse comparative clôt le chapitre et identifie les lacunes que ma contribution vise à combler.

2.2 DOTS — DDoS Open Threat Signaling

2.2.1 Contexte et motivation

L'IETF a initié le groupe de travail DOTS (*DDoS Open Threat Signaling*) pour définir un protocole standardisé permettant à une entité attaquée de notifier automatiquement un prestataire de mitigation. Les exigences sont formalisées dans la RFC 8612 (MORTENSEN et al., 2019) et le canal de signalisation dans la RFC 8782 (REDDY et al., 2020).

2.2.2 Architecture et fonctionnement

DOTS repose sur une architecture client–serveur asymétrique comprenant un **DOTS client** (l'entité sous attaque) et un **DOTS server** (le prestataire de mitigation).

Le protocole s'appuie sur une pile technologique légère adaptée aux environnements contraints et congestionnés, divisée en deux canaux distincts (voir Figure 2.1) :

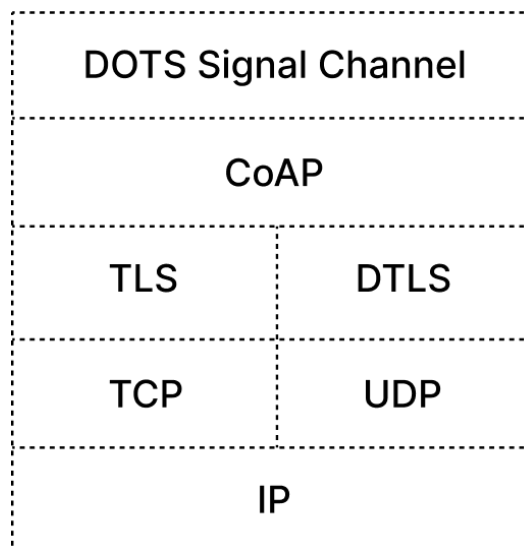


FIGURE 2.1 – Architecture de référence du protocole DOTS : séparation entre le canal de signalisation (CoAP/DTLS/UDP) et le canal de données (RESTCONF/HTTPS/TCP).

2.2.2.1 Canal de signalisation (*Signal Channel*)

Ce canal utilise le protocole Constrained Application Protocol (CoAP) (*Constrained Application Protocol*, RFC 7252 (SHELBY et al., 2014)) sécurisé par Datagram Transport Layer Security (DTLS) 1.2 (RFC 6347 (RESCORLA & MODADUGU, 2012)) sur UDP. Ce choix est stratégique : contrairement à TCP, UDP évite les délais de retransmission et le blocage de tête de ligne (*Head-of-Line blocking*) fréquents lors d’une saturation de lien.

Concrètement, le client émet une requête CoAP PUT sur l’URI :

`/dots/signal/{cuid}/{mid}`

contenant un objet Concise Binary Object Representation (CBOR) (*Concise Binary Object Representation*, RFC 8949 (BORMANN & HOFFMAN, 2020)) qui spécifie les préfixes IP à protéger (`target-prefix`), les numéros de port (`target-port-range`), le protocole (`target-protocol`) ainsi que la durée de mitigation souhaitée (`lifetime`). Le serveur répond avec un code 2.01 **Created** et un identifiant de mitigation (`mid`).

Ce canal gère le cycle de vie de la mitigation via une **machine à états à quatre transitions** :

idle $\longrightarrow$ *mitigation-requested* $\longrightarrow$ *attack-mitigation-in-progress* $\longrightarrow$ *terminated*

Un mécanisme de *heartbeat* (messages CoAP GET périodiques, intervalle configurable via `heartbeat-interval`) vérifie la connectivité entre client et serveur. En l’absence de réponse après un nombre configurable de tentatives (`missing-hb-allowed`), le client déclenche une procédure de reprise (*failover*).

2.2.2.2 Canal de données (*Data Channel*)

Basé sur **RESTCONF** (RFC 8040 (BIERMAN et al., 2017)) via HTTPS (TLS 1.2+, TCP), il permet l’échange de configurations complexes et moins urgentes. Les données

sont modélisées en YANG (RFC 7950 (BJÖRKLUND, 2016)). Ce canal remplit trois fonctions. Il permet d’abord de définir des **alias** (*dots-alias*) regroupant plusieurs préfixes IP, ports et protocoles sous un identifiant unique, réduisant ainsi la taille des messages de signalisation en période d’attaque. Il sert également à pré-configurer des **Access Control Lists (ACLs)** en amont, modélisées selon le module YANG `ietf-dots-data-channel` (RFC 8783), qui spécifient des règles de filtrage au format `ietf-access-control-list` incluant les critères de correspondance (5-tuple : IP source/destination, port source/destination, protocole) et les actions (`permit`, `deny`, `rate-limit`). Enfin, il permet de récupérer les **télémétries** d’attaque (RFC 9244 (BOUCAIR et al., 2022)) : bande passante entrante, nombre de paquets par seconde, distribution des protocoles d’attaque, *top talkers* (sources les plus actives).

2.2.3 Limites identifiées

DOTS souffre tout d’abord d’un **modèle hiérarchique strict** : le protocole est fondamentalement asymétrique (client → serveur) et ne prévoit pas de coordination horizontale entre clients. Un client DOTS ne peut pas signaler une attaque à un pair DOTS ni partager des métadonnées d’attaque avec d’autres victimes potentielles ; le standard ne définit d’ailleurs pas de mécanisme de *peering* entre serveurs DOTS de différents opérateurs.

À cette rigidité s’ajoute une **absence de mutualisation de capacité** : DOTS signale le besoin de mitigation mais ne spécifie pas comment répartir la charge entre plusieurs mitigeurs. Si le serveur DOTS est lui-même saturé, aucun mécanisme de délégation ou d’équilibrage n’est prévu par le standard.

La **confiance** reste par ailleurs **limitée aux relations bilatérales** : l’authentification repose sur des certificats X.509 ou PSK (*Pre-Shared Key*) pré-provisionnés, sans mécanisme dynamique pour évaluer la fiabilité d’un pair inconnu (MORTENSEN et al., 2019), ce qui limite le déploiement à des relations contractuelles préétablies.

Enfin, DOTS **ne permet pas le partage d’intelligence** : il transmet des demandes de mitigation, non des signatures d’attaque. Un serveur DOTS ne redistribue pas les caractéristiques techniques de l’attaque (vecteurs, TTL, taille de paquets) aux autres entités du réseau pour une protection proactive.

2.3 DDoS Clearing House — Partage collaboratif de *fingerprints*

2.3.1 Contexte et motivation

Le projet DDoS Clearing House, développé par SIDN Labs dans le cadre du projet européen CONCORDIA (H2020), vise à briser le cloisonnement des informations sur les attaques DDoS entre opérateurs. Il permet le partage automatisé de *fingerprints* (empreintes) d’attaques entre membres d’une coalition nationale (SIDN LABS, 2021).

2.3.2 Architecture et fonctionnement

2.3.2.1 Le *DDoS Fingerprint* : structure et contenu

Le cœur du système repose sur le **DDoS Fingerprint**, une structure de données standardisée décrivant les caractéristiques techniques observées d'une attaque, *sans inclure de données personnelles identifiables* (PII). Chaque fingerprint contient le **vecteur d'attaque** identifié (ex. : DNS_AMPLIFICATION, NTP_AMPLIFICATION, SYN_FLOOD, UDP_FLOOD), les **caractéristiques réseau** observées (protocole de transport TCP/UDP/ICMP, ports source et destination, préfixes source agrégés et anonymisés en /24 ou /16 pour préserver la vie privée) ainsi que les **caractéristiques des paquets** (taille moyenne et distribution, valeur TTL observée, flags TCP, présence ou absence de fragmentation IP). Ces données structurelles sont complétées par des **métriques volumétriques** (débit crête en pps et bps, durée de l'attaque, nombre estimé de sources uniques) et un **horodatage** UTC de début et fin.

Les fingerprints sont sérialisés au format **JSON** et échangés via une **API REST** (HTTPS) entre les composants du système.

2.3.2.2 Architecture logicielle

Le système suit une architecture à trois composants (voir Figure 2.2) :

1. **Module local de génération (*DDoS DB local*)** : Déployé chez chaque membre participant, ce module se connecte aux outils de détection existants (NetFlow/IPFIX collectors, IDS type Suricata/Snort, ou plateformes DDoS dédiées). Il capture les métadonnées du trafic d'attaque via **DissecTor**, un outil d'analyse de captures PCAP développé par SIDN Labs, qui extrait automatiquement les champs du fingerprint à partir du trafic brut. Le module applique ensuite une **anonymisation** des adresses IP sources (troncature à /24) et supprime tout payload applicatif avant transmission.
2. **Clearing House central (*Hub*)** : Un serveur central reçoit les fingerprints via l'API REST, les **déduplique** (en identifiant les attaques observées par plusieurs membres simultanément grâce à la corrélation temporelle et aux vecteurs communs), les **enrichit** (ajout de métadonnées de réputation des préfixes sources via des bases comme Shodan ou Spamhaus) et les stocke dans une base de données centralisée. Le hub expose une API de consultation permettant aux membres d'interroger l'historique par vecteur d'attaque, période ou préfixe cible.
3. **Module de consommation (*Converter*)** : Chaque membre récupère les fingerprints pertinents et un module de conversion les traduit automatiquement en règles de filtrage opérationnelles adaptées à l'infrastructure locale : règles **iptables/nftables** pour le filtrage Linux, règles BGP Flowspec (RFC 5575 (MARQUES et al., 2009)) pour distribution aux routeurs de bordure, et politiques pour *scrubbing appliances* (ex. : Arbor Sightline/TMS, A10 Thunder TPS).

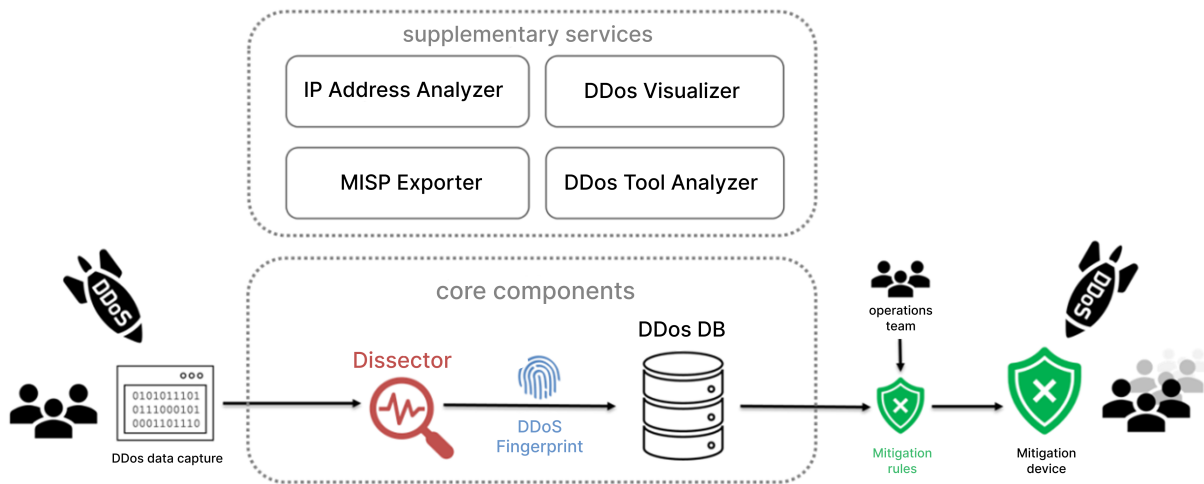


FIGURE 2.2 – Architecture du *DDoS Clearing House* : génération d’empreintes DDoS par le composant Dissector, stockage dans la base DDoS DB, puis utilisation par l’équipe opérationnelle pour générer des règles de mitigation.

2.3.3 Limites identifiées

La principale limite du système tient à son **partage passif sans orchestration** : il se limite à la diffusion d’intelligence (*Threat Intel*) sans coordonner de réponse active. Il n’existe aucun mécanisme permettant à un membre de demander une mitigation distribuée aux autres ; chaque membre applique indépendamment les règles dérivées des fingerprints reçus.

Son **architecture centralisée** introduit par ailleurs un point de défaillance unique (SPOF) : si le hub central est indisponible (panne, attaque ciblée), la dissémination des fingerprints est interrompue, et les opérations de déduplique et d’enrichissement deviennent impossibles.

Le système souffre également d’un **partage *a posteriori*** : la génération d’un fingerprint nécessite l’analyse du trafic d’attaque après sa détection. Le temps de traitement cumulé (capture PCAP → extraction DissecTor → upload API → déduplique → download par les membres → conversion en règles) introduit un délai incompressible de l’ordre de plusieurs minutes (SIDN LABS, 2021), rendant le système inefficace contre les attaques éphémères (*hit-and-run*, durée < 5 min).

Enfin, le modèle repose sur une **confiance implicite** : aucun mécanisme formel ne permet de vérifier la véracité d’un fingerprint soumis. Un membre compromis ou malveillant pourrait théoriquement injecter de faux fingerprints pour provoquer le blocage de trafic légitime chez les autres membres (*poisoning attack*).

2.4 NaWas / NBIP — Centre national de *scrubbing* mutualisé

2.4.1 Contexte et architecture

NaWas (*National Wasstraat*, littéralement « station de lavage nationale ») est une initiative néerlandaise opérée par le NBIP (*Nationale Beheersorganisatie Internet Providers*), un consortium à but non lucratif regroupant des FAIs néerlandais. Elle offre une infrastructure de *scrubbing* mutualisée aux membres participants (NBIP, 2023).

2.4.1.1 Mécanisme de redirection BGP

L’activation de la protection NaWas repose sur le principe de l’**injection de routes plus spécifiques** via BGP. En cas d’attaque (voir Figure 2.3) :

1. Le membre annonce le préfixe IP attaqué (ex. : 203.0.113.0/24) à la plateforme NaWas via une session eBGP dédiée, en y attachant une **communauté BGP spécifique** (ex. : AS:666 ou une communauté étendue propriétaire) qui indique la demande de scrubbing.
2. NaWas **réannonce** ce préfixe vers ses fournisseurs de transit (*upstream providers*) avec un *AS-path* plus court, attirant ainsi le trafic destiné au préfixe attaqué (*traffic attraction* par spécificité de préfixe).
3. Le trafic entrant est traité par des ***scrubbing lanes*** (lignes de nettoyage) composées d’appliances matérielles et logicielles qui appliquent un filtrage basé sur signatures (Deep Packet Inspection (DPI)) pour les vecteurs d’attaque connus, un *rate limiting* adaptatif par source IP et par protocole, une analyse comportementale (détection d’anomalies statistiques sur la distribution des tailles de paquets, TTL, flags TCP), ainsi qu’un challenge cryptographique (SYN cookies, JavaScript challenge pour HTTP) pour valider les connexions légitimes.
4. Le trafic nettoyé (*clean traffic*) est renvoyé au membre via un **tunnel GRE** (RFC 2784) ou un **VLAN dédié** sur une interconnexion privée, contournant ainsi la saturation des liens publics du membre. Le routage de retour utilise un chemin distinct (*clean-return path*) pour éviter les boucles de routage.

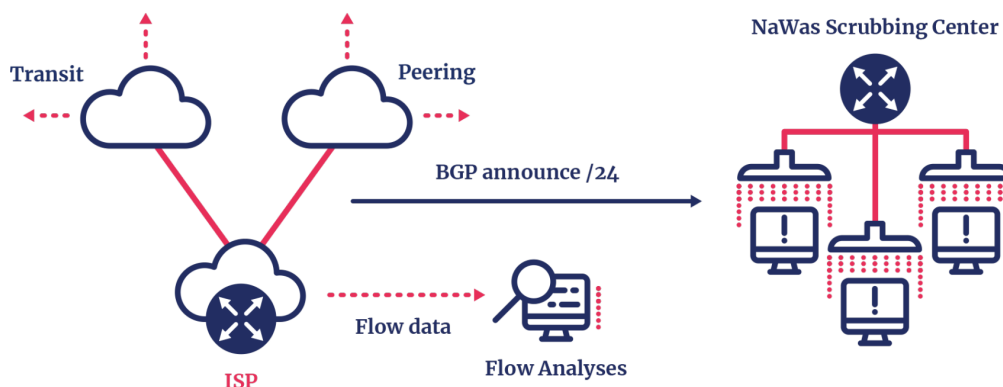


FIGURE 2.3 – Architecture simplifiée du service NaWas : redirection du trafic vers un centre de scrubbing mutualisé via annonce BGP et analyse des flux avant application des mécanismes de mitigation.

2.4.2 Limites identifiées

L'**architecture semi-centralisée** constitue la première limite : bien que mutualisée, l'infrastructure physique de NaWas est concentrée aux Pays-Bas (quelques PoP sur des IXP néerlandais — AMS-IX, NL-ix). Un membre éloigné géographiquement subit une latence de redirection significative, et le *scrubbing center* lui-même peut devenir un SPOF en cas d'attaque ciblant directement l'infrastructure NaWas.

L'**activation semi-manuelle** représente un second obstacle opérationnel : l'injection de la route BGP avec la communauté de scrubbing nécessite typiquement une intervention de l'opérateur du membre (commande CLI sur le routeur de bordure ou via un portail web). L'automatisation complète dépend donc de l'intégration locale chez chaque membre, sans garantie d'homogénéité.

Les membres NaWas n'échangent pas de métadonnées d'attaque entre eux et chacun interagit uniquement avec la plateforme centrale ; cette **absence de coordination horizontale inter-membres** prive le système des avantages d'une intelligence partagée comparable à ce que propose le Clearing House.

Enfin, la **capacité finie** de la plateforme est dimensionnée pour le volume agrégé moyen des attaques ciblant les membres. En cas d'attaque volumétrique massive (> 1 Tbps) ciblant un seul membre, la capacité mutualisée peut s'avérer insuffisante sans mécanisme de débordement (*overflow*) vers des tiers.

2.5 DefCOM — *Distributed Cooperative Overlay Defense*

2.5.1 Contexte et architecture

Proposé par Mirkovic et al. (MIRKOVIC et al., 2003), DefCOM est un système de défense coopérative qui structure les nœuds de défense en un **réseau overlay logique** pour éviter les goulots d'étranglement centraux. C'est l'une des premières propositions

académiques de défense DDoS véritablement distribuée.

2.5.1.1 Topologie et rôles des nœuds

DefCOM définit trois rôles fonctionnels pour les nœuds participants. Les **nœuds *Victim-end* (V-nodes)**, déployés dans le réseau de la victime, détectent l'attaque en observant la surcharge locale (taux d'utilisation CPU, bande passante saturée, taux de paquets rejetés) et initient la réponse coopérative en classifiant les flux entrants comme légitimes (*good*) ou suspects (*bad*) à l'aide de statistiques locales et de listes blanches préétablies. Les **nœuds *Core* (C-nodes)**, positionnés dans les réseaux intermédiaires (FAIs de transit), relayent les informations de contrôle et appliquent le *rate limiting* sur les flux signalés comme suspects ; chaque C-node maintient un classifieur de paquets qui marque les paquets traversants via un champ de l'en-tête IP pour distinguer le trafic légitime du trafic suspect en aval. Enfin, les **nœuds *Source-end* (S-nodes)**, déployés au plus proche des sources d'attaque, appliquent le filtrage le plus agressif sur les flux identifiés comme malveillants en utilisant les informations reçues des V-nodes et C-nodes, avec pour objectif de supprimer le trafic d'attaque au plus tôt dans le chemin réseau, préservant ainsi la bande passante des liens intermédiaires.

2.5.1.2 Mécanisme de *Rate Limiting* coopératif

L'architecture met en œuvre un mécanisme de ***Rate Limiting* coopératif hiérarchique** (voir Figure 2.4). Le protocole fonctionne comme suit :

1. Le V-node détecte une surcharge et calcule un **débit maximal acceptable** (R_{max}) pour le trafic entrant sur le préfixe attaqué.
2. Il envoie un message de type `RATE_LIMIT_REQUEST` à ses C-nodes voisins dans l'overlay, contenant : le préfixe cible, le débit maximal R_{max} , et une liste de préfixes sources classés comme suspects.
3. Chaque C-node recevant ce message **répartit** la limite de débit R_{max} proportionnellement entre ses liens amont :

$$R_i = R_{max} \times w_i \quad (2.1)$$

où w_i est le poids du lien i basé sur le volume de trafic observé, puis propage la demande vers les nœuds plus en amont.

4. Ce processus forme un **arbre de limitation de débit** (*rate-limiting tree*) où la contrainte se propage de la victime vers les sources, chaque nœud intermédiaire appliquant sa part du budget de débit.
5. Les S-nodes, en bout de chaîne, appliquent le **filtrage terminal** : les flux dépassant leur budget alloué R_i sont soit rejetés (*drop*), soit soumis à un marquage probabiliste (*probabilistic drop*) privilégiant les sources connues comme légitimes.

2.5.1.3 Protocole de communication overlay

La communication entre les nœuds se fait via un **protocole de type *gossip*** (épidémique). Chaque nœud maintient une **table de voisinage** (liste de pairs connus) et diffuse

périodiquement ses informations d'état (charge actuelle, débit traité, alertes actives) à un sous-ensemble aléatoire de ses voisins. Ce mécanisme assure une **convergence rapide** — l'information atteint tous les nœuds en $O(\log N)$ rounds de gossip pour N participants — ainsi qu'une **tolérance aux pannes** élevée, la perte d'un nœud intermédiaire n'empêchant pas la propagation grâce à la redondance des chemins de diffusion.

La construction de l'overlay suit un mécanisme de **découverte initiale par traceroute** : chaque nœud participant exécute des **traceroute** vers les autres nœuds connus pour reconstruire la topologie Internet sous-jacente et positionner correctement les C-nodes sur le chemin entre sources et victimes.

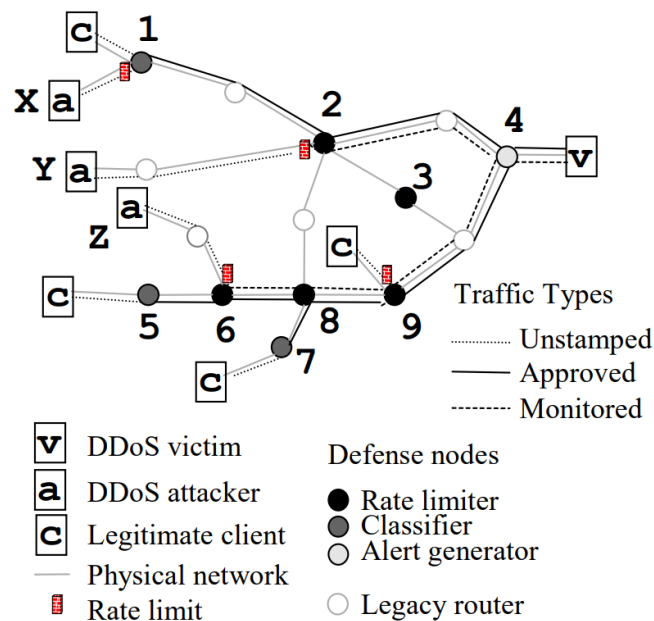


FIGURE 2.4 – Arbre de limitation de débit coopératif dans DefCOM : propagation de la demande de la victime vers les sources.

2.5.2 Limites identifiées

DefCOM présente plusieurs limites qui expliquent son absence de déploiement opérationnel. Il s'agit d'abord d'une **contribution purement académique** : le système est resté au stade de prototype évalué en simulation (ns-2), sur des topologies de quelques dizaines de nœuds, loin des conditions réelles de l'Internet.

L'**absence de standards d'interopérabilité** constitue un obstacle technique majeur : le protocole de communication overlay (format des messages, encodage, machine à états) n'est défini que dans les publications académiques, sans spécification formelle (pas de RFC, pas de schéma YANG, pas d'API standardisée), ce qui rend l'interopérabilité entre implémentations différentes impossible en pratique.

La **vulnérabilité aux nœuds compromis** est particulièrement préoccupante car le modèle de confiance de DefCOM est implicite : tout nœud participant est considéré comme honnête. Un nœud compromis pourrait générer de fausses alertes (`RATE_LIMIT_REQUEST` avec des préfixes légitimes comme cibles) pour provoquer le blocage de trafic légitime, ignorer les demandes de *rate limiting* reçues (*free-riding*), ou manipuler les poids de répartition

pour rediriger le trafic légitime vers des liens saturés. Aucun mécanisme de réputation, de vérification cryptographique des messages ou de détection de comportement byzantin n'est intégré.

L'hypothèse de déploiement incrémental pose également problème : DefCOM suppose un déploiement coopératif par des Autonomous Systems (ASs) multiples, incluant des FAIs de transit. En pratique, l'incitation économique pour un FAI de transit à déployer un nœud DefCOM (qui consomme des ressources pour protéger les clients d'un autre AS) est faible, posant un problème classique d'externalité en économie des réseaux.

Enfin, la scalabilité de la découverte par traceroute est intrinsèquement limitée : la reconstruction de la topologie overlay génère un volume de trafic de sondage quadratique ($O(N^2)$ pour N nœuds) et les résultats sont instables en raison de l'asymétrie de routage, du filtrage ICMP et du load balancing.

2.6 PeerTrust — Modèle de confiance pour communautés pair-à-pair

2.6.1 Contexte et motivation

Le modèle PeerTrust, proposé par Xiong et Liu (XIONG & LIU, 2004), est un cadre de gestion de la confiance basé sur la réputation pour les communautés en ligne pair-à-pair. Bien que conçu initialement dans le contexte du commerce électronique P2P, PeerTrust constitue la référence théorique fondamentale pour tout système de confiance distribué sans autorité centrale. Son inclusion dans cette étude bibliographique est justifiée par le fait qu'il fournit les bases formelles du mécanisme de confiance que nous adaptons dans le système.

2.6.2 Paramètres de confiance et métrique générale

PeerTrust identifie cinq facteurs essentiels pour évaluer la fiabilité d'un pair dans une communauté P2P évolutive.

Le premier paramètre est le **feedback** $S(u, i)$, qui quantifie le degré de satisfaction normalisé que le pair u reçoit lors de sa i -ème transaction, exprimé dans $[0, 1]$. Le deuxième est la **portée du feedback** $I(u)$, soit le nombre total de transactions effectuées par le pair, qui permet de contextualiser la valeur de confiance dans son volume d'activité : un pair honnête mais peu actif ne doit pas être traité comme plus fiable qu'un pair actif ayant un historique solide. Le troisième paramètre est la **crédibilité de la source de feedback** $Cr(p)$: la qualité d'un retour dépend de la fiabilité de son émetteur. PeerTrust propose deux méthodes de calcul : la métrique TVM (Trust Value Metric), qui utilise récursivement la valeur de confiance des évaluateurs, et la métrique PSM (Personalized Similarity Metric), qui mesure la similarité comportementale entre l'évaluateur et le pair évalué. Le quatrième est le **facteur contextuel de transaction** $TF(u, i)$, qui permet de pondérer les transactions selon leur taille, leur catégorie ou leur criticité : des transactions importantes doivent peser davantage dans le calcul final. Le cinquième est le **facteur contextuel de communauté** $CF(u)$, qui capture des caractéristiques spéci-

fiques à la communauté, comme l'incitation à fournir du feedback ou la présence de pairs pré-approuvés.

Ces cinq paramètres sont combinés dans la métrique générale de confiance :

$$T(u) = \alpha \cdot \sum_{i=1}^{I(u)} S(u, i) \cdot Cr(p(u, i)) \cdot TF(u, i) + \beta \cdot CF(u) \quad (2.2)$$

où α et β sont des facteurs de pondération normalisés. En désactivant les facteurs contextuels ($TF = 1$, $\alpha = 1$, $\beta = 0$), on obtient la **métrique de base** :

$$T(u) = \sum_{i=1}^{I(u)} S(u, i) \cdot Cr(p(u, i)) \quad (2.3)$$

Cette forme constitue une somme pondérée du niveau de satisfaction, où chaque feedback est pondéré par la crédibilité de sa source ; cette somme brute n'étant pas normalisée, $T(u)$ n'est pas borné dans $[0, 1]$ à ce stade. La question centrale est alors : comment calculer $Cr(p(u, i))$? PeerTrust propose deux métriques concrètes.

La première, appelée **métrique TVM** (*Trust Value Metric*), utilise récursivement la valeur de confiance du pair évaluateur comme mesure de crédibilité, normalisée par la somme des valeurs de confiance de tous les évaluateurs ayant interagi avec u :

$$T_{TVM}(u) = \sum_{i=1}^{I(u)} S(u, i) \cdot \frac{T(p(u, i))}{\sum_{i=1}^{I(u)} T(p(u, i))} \quad (2.4)$$

Cette normalisation garantit que $T_{TVM}(u) \in [0, 1]$ sans troncature artificielle. Les feedbacks provenant de pairs très fiables pèsent davantage que ceux de pairs peu fiables. La deuxième, la **métrique PSM** (*Personalized Similarity Metric*), mesure la similarité comportementale entre l'évaluateur et le pair évalué via leurs historiques de feedback communs, ce qui la rend robuste aux collusions mais plus coûteuse en calcul.

Lorsqu'un facteur contextuel de transaction $D(u, i)$ est incorporé pour pondérer les transactions selon leur importance opérationnelle (Équation 6 du papier original), la métrique devient :

$$T(u) = \sum_{i=1}^{I(u)} S(u, i) \cdot Cr(p(u, i)) \cdot D(u, i) \quad (2.5)$$

C'est cette forme qui est directement adaptée dans le système, où $D(u, i)$ reflète la sévérité de l'attaque DDoS associée à la session d'aide i (LOW, MEDIUM, HIGH, CRITICAL).

2.6.3 Architecture d'implémentation distribuée

PeerTrust est conçu pour fonctionner sans base de données centrale. Chaque nœud maintient un **gestionnaire de confiance** (Trust Manager) responsable de la soumission

du feedback et du calcul de la fiabilité des pairs, une **base de données locale** stockant une portion des données de confiance globales, et un **localisateur de données** permettant de placer et retrouver ces données sur le réseau via un schéma DHT (P-Grid dans l'implémentation originale). L'évaluation est exécutée de manière dynamique et décentralisée : au lieu d'un serveur central, un pair obtient les données de confiance d'un autre pair auprès du reste du réseau et calcule sa valeur à la demande.

2.6.4 Limites identifiées dans le contexte DDoS

PeerTrust, bien que robuste sur le plan théorique, présente plusieurs limites lorsqu'on envisage son application directe à un système de coordination anti-DDoS. Il a été conçu pour des **transactions e-commerce** où le feedback est fourni après chaque achat ; dans un contexte de mitigation, les sessions d'aide sont des interventions d'urgence avec des contraintes de latence radicalement différentes. La métrique de crédibilité TVM, qui utilise la valeur de confiance des évaluateurs de manière récursive, requiert $O(N)$ sauts réseau pour un calcul dynamique complet, ce qui est prohibitif lors d'une attaque active. PeerTrust ne définit pas non plus de **mécanisme de sélection des pairs** tenant compte de critères opérationnels tels que la capacité disponible, la latence réseau ou la réciprocité des échanges. Enfin, le facteur contextuel $D(u, i)$ est laissé entièrement à la discrétion du concepteur, sans guidance pour le choix des valeurs dans un contexte de sécurité réseau.

2.7 Synthèse comparative

Le Tableau 2.1 compare les travaux analysés selon les critères clés de la coordination distribuée anti-DDoS.

Système	Architecture	P2P	Partage méta.	Sync. actions	Mutualisation	Confiance formalisée
DOTS (RFC 8612)	Client-serveur	×	✓ (<i>partiel</i>)	✓	×	×
DDoS Clearing House	Centralisée	×	✓	×	×	×
NaWas / NBIP	Semi-centralisée	×	×	×	✓	×
DefCOM	P2P overlay	✓	✓ (<i>partiel</i>)	✓ (<i>partiel</i>)	×	×
PeerTrust (XIONG & LIU, 2004)	P2P décentralisée	✓	×	×	×	✓
Le système	P2P plat	✓	✓	✓	✓	✓

TABLE 2.1 – Comparaison des systèmes existants et positionnement de la contribution.

Le Tableau 2.1 positionne le système par rapport aux travaux existants. Comme le montre la dernière ligne, la contribution est la seule à satisfaire simultanément les cinq critères identifiés. Les systèmes anti-DDoS existants couvrent chacun une dimension isolée : DOTS standardise la signalisation mais reste hiérarchique et sans confiance dynamique ; le Clearing House partage l'intelligence en différé et de manière centralisée ; NaWas mutualise les ressources sans coordination horizontale entre membres ; DefCOM distribue la défense mais sans modèle de confiance ni interopérabilité formelle. PeerTrust apporte le modèle de confiance manquant, mais dans un contexte e-commerce sans lien avec la mitigation réseau. le système comble l'ensemble de ces lacunes en combinant une architecture

P2P décentralisée, une coordination en temps réel, un partage de capacités, et un modèle PeerTrust adapté au contexte DDoS avec crédibilité historique persistée.

L'analyse révèle qu'aucun système existant ne combine simultanément les cinq propriétés nécessaires à une défense collaborative opérationnelle. Une architecture véritablement décentralisée (P2P) éliminant tout SPOF n'est réalisée que par DefCOM et PeerTrust, mais aucun des deux ne l'applique à la coordination anti-DDoS en production. Un mécanisme de **partage de métadonnées d'attaque en temps réel** (et non *a posteriori*) fait défaut : DOTS transmet des demandes, non des signatures ; le Clearing House partage en différé. Une **orchestration coordonnée des actions de mitigation** entre participants n'est qu'esquissée dans DefCOM, sans mécanisme de confiance garantissant l'honnêteté des participants. Précisément, un **modèle de confiance dynamique et formalisé** — permettant de quantifier et de faire évoluer la fiabilité de chaque pair sans relation préétablie — est absent de tous les systèmes DDoS ; PeerTrust le fournit mais dans un contexte e-commerce sans lien avec la mitigation. Enfin, une **mutualisation effective des capacités de scrubbing** couplée à une sélection intelligente des pairs reste inexistante : NaWas mutualise physiquement mais sans coordination horizontale ni critère de sélection basé sur la confiance.

2.8 Conclusion

L'étude bibliographique révèle une progression vers la défense collaborative qui reste incomplète sur deux dimensions critiques. Du côté des systèmes anti-DDoS : DOTS standardise la signalisation inter-domaines mais reste hiérarchique ; le Clearing House partage l'intelligence d'attaque mais sans orchestration active et avec un délai inhérent ; NaWas mutualise les ressources physiques mais sans coordination horizontale entre membres ; DefCOM propose une architecture overlay distribuée mais sans mécanismes de confiance ni d'interopérabilité. Du côté de la gestion de la confiance, PeerTrust fournit un modèle formel et décentralisé solide, mais conçu pour le commerce électronique et non pour la coordination de mitigation en temps réel.

Aucun système ne combine pleinement une architecture décentralisée (P2P), une coordination d'actions en temps réel, un partage d'intelligence proactif et des mécanismes de confiance robustes. C'est précisément dans cet espace que s'inscrit la contribution présentée dans les chapitres suivants de ce mémoire.

Chapitre 3

Conception et Architecture du système

3.1 Introduction

Les chapitres précédents ont mis en évidence deux constats : les attaques DDoS modernes dépassent les capacités de défense individuelles, et les systèmes collaboratifs existants souffrent d’une centralisation résiduelle ou d’une absence de mécanisme de confiance formalisé. Ce chapitre présente la conception du système distribué de mitigation collaborative qui répond à ces lacunes.

Il est organisé en deux parties : la **conception préliminaire** établit les exigences et l’architecture globale du système ; la **conception technique détaillée** formalise les modèles de données, les algorithmes et les protocoles de sécurité retenus.

3.2 Conception préliminaire

3.2.1 Rappel du périmètre et des exigences

Tel que défini en introduction, le système couvre exclusivement les **échanges inter-nœuds** au sein d’une coalition déjà constituée ; les diagrammes de conception ont été réalisés avec les outils draw.io (JGRAPH LTD, 2024) et Lucidchart (LUCID SOFTWARE INC., 2024) selon les conventions Unified Modeling Language (UML) 2.5.1 (OBJECT MANAGEMENT GROUP, 2017) : coordination collaborative, gestion de la confiance et sécurisation des communications. La détection réseau des attaques, le filtrage effectif du trafic et la configuration des tunnels d’infrastructure restent hors périmètre.

Le système est conçu autour de six exigences fondamentales : décentralisation complète, confiance calculée et évolutive, sélection multicritère des pairs, sécurité de bout en bout (mTLS + JWT RS256), traçabilité complète via journal d’audit et scalabilité jusqu’à une centaine de nœuds.

3.2.2 Architecture générale

3.2.2.1 Topologie de la coalition

La solution proposée adopte une topologie P2P **Pair-à-Pair** : chaque nœud maintient localement un registre des pairs connus. Ce choix a été retenu pour trois raisons complémentaires. L'**hétérogénéité des participants** d'abord : universités, PME, opérateurs et datacenters ont des capacités très variables, et une topologie plate évite de favoriser arbitrairement certains nœuds. La **résilience** ensuite : la disparition d'un pair n'affecte pas la connectivité des autres. La **simplicité de déploiement** enfin : chaque nœud n'a besoin que d'une adresse d'entrée initiale pour rejoindre la coalition.

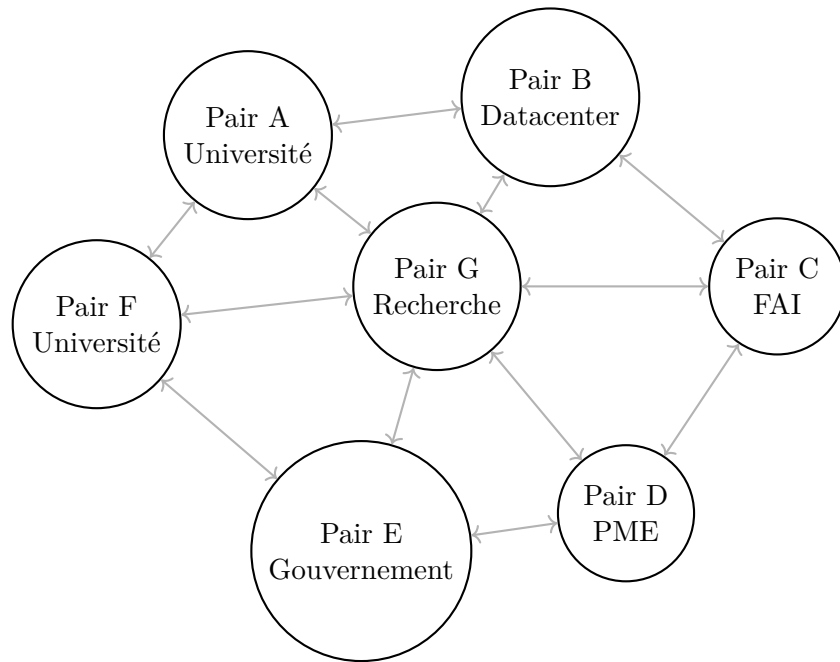


FIGURE 3.1 – Exemple de coalition : sept nœuds hétérogènes interconnectés en overlay P2P plat non structuré.

3.2.2.2 Cycle de vie d'une session de mitigation

Le flux d'une session d'aide suit quatre phases distinctes :

1. **Détection et escalade** : le nœud victime détecte un débordement de capacité et lance une demande d'aide auprès des pairs sélectionnés par l'algorithme WSM.
2. **Négociation** : les pairs reçoivent une requête POST /help/request et acceptent ou rejettent en fonction de leur capacité disponible.
3. **Redirection active** : le trafic excédentaire est redirigé vers les pairs acceptants via un tunnel (GRE, VXLAN ou IP/IP).
4. **Clôture** : à la fin de l'attaque, le nœud victime notifie la clôture (POST /attack/over), capture les crédibilités réseau au moment de la session et recalcule les scores de confiance.

3.2.3 Périmètre du système

La Figure 3.2 présente l'architecture globale du système et délimite explicitement son périmètre. Le système gère l'intégralité de la coordination applicative : diffusion des demandes d'aide (❶), collecte des réponses des pairs acceptants ou refusants (❷) et émission du signal de redirection (❸). En revanche, la redirection physique du trafic reste hors périmètre et repose sur l'infrastructure réseau du nœud victime (routeur, tunnel GRE/VXLAN/IPIP) : c'est elle qui achemine le trafic excédentaire vers les pairs aidants (❹), lesquels filtrent le trafic malveillant et renvoient le trafic propre vers cette même infrastructure (❺), avant qu'il ne soit agrégé et retransmis au nœud victime (❻).

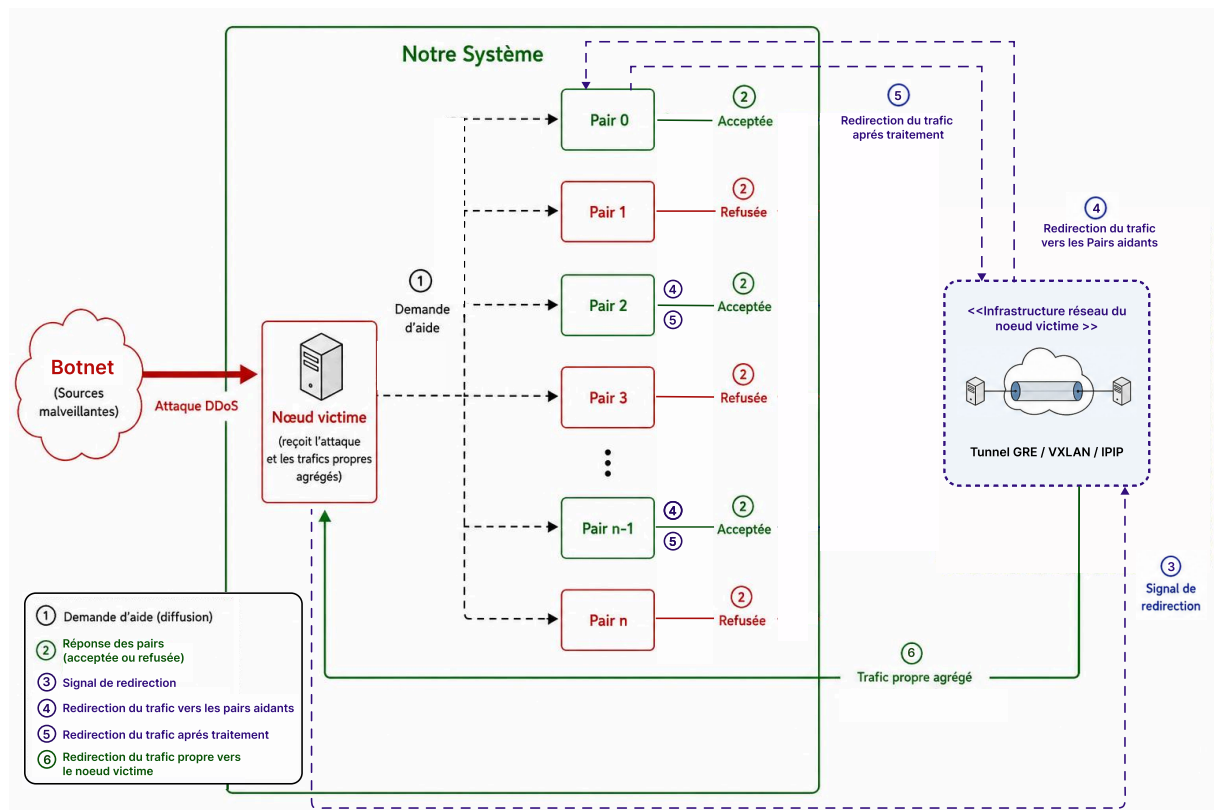


FIGURE 3.2 – Architecture globale du système ShieldNet : périmètre applicatif et redirection réseau hors périmètre.

3.2.4 Diagrammes de cas d'utilisation

Le système est décrit par trois diagrammes de cas d'utilisation complémentaires, couvrant respectivement le protocole de mitigation collaborative, le moteur de confiance PeerTrust et le rôle complet du nœud pair dans la coalition. Les trois acteurs impliqués sont : le **nœud local** (victime en situation d'attaque), le **nœud pair** (participant distant) et l'**administrateur** (responsable de la configuration locale).

3.2.4.1 Protocole de mitigation collaborative

La Figure 3.3 détaille les interactions entre le **nœud victime** et le **nœud pair** lors d'une session de mitigation. Ce diagramme couvre le flux complet depuis l'envoi de la requête d'aide jusqu'au recalcul du score de confiance.

Le nœud victime initie la séquence en **envoyant une requête d'aide** (POST /help/request), cas d'usage qui inclut obligatoirement la **sélection des pairs via l'algorithme WSM** : chaque pair candidat est évalué selon quatre critères pondérés (capacité disponible, latence inversée, score de confiance, réciprocité). Le nœud pair peut alors **accepter** (PUT /help/{id}/accept) ou **rejeter** (PUT /help/{id}/reject) la session ; l'acceptation étend optionnellement vers l'**activation de la session** (POST /traffic/redirect), qui correspond au signal de démarrage de la redirection au niveau applicatif. La **clôture de l'attaque** (POST /attack/over), déclenchée par le nœud victime, inclut deux actions systématiques : la **mise à jour des crédits de réciprocité** des deux nœuds dans leurs ledgers respectifs, et le **recalcul du score de confiance PeerTrust $T(u)$** du pair assistant.

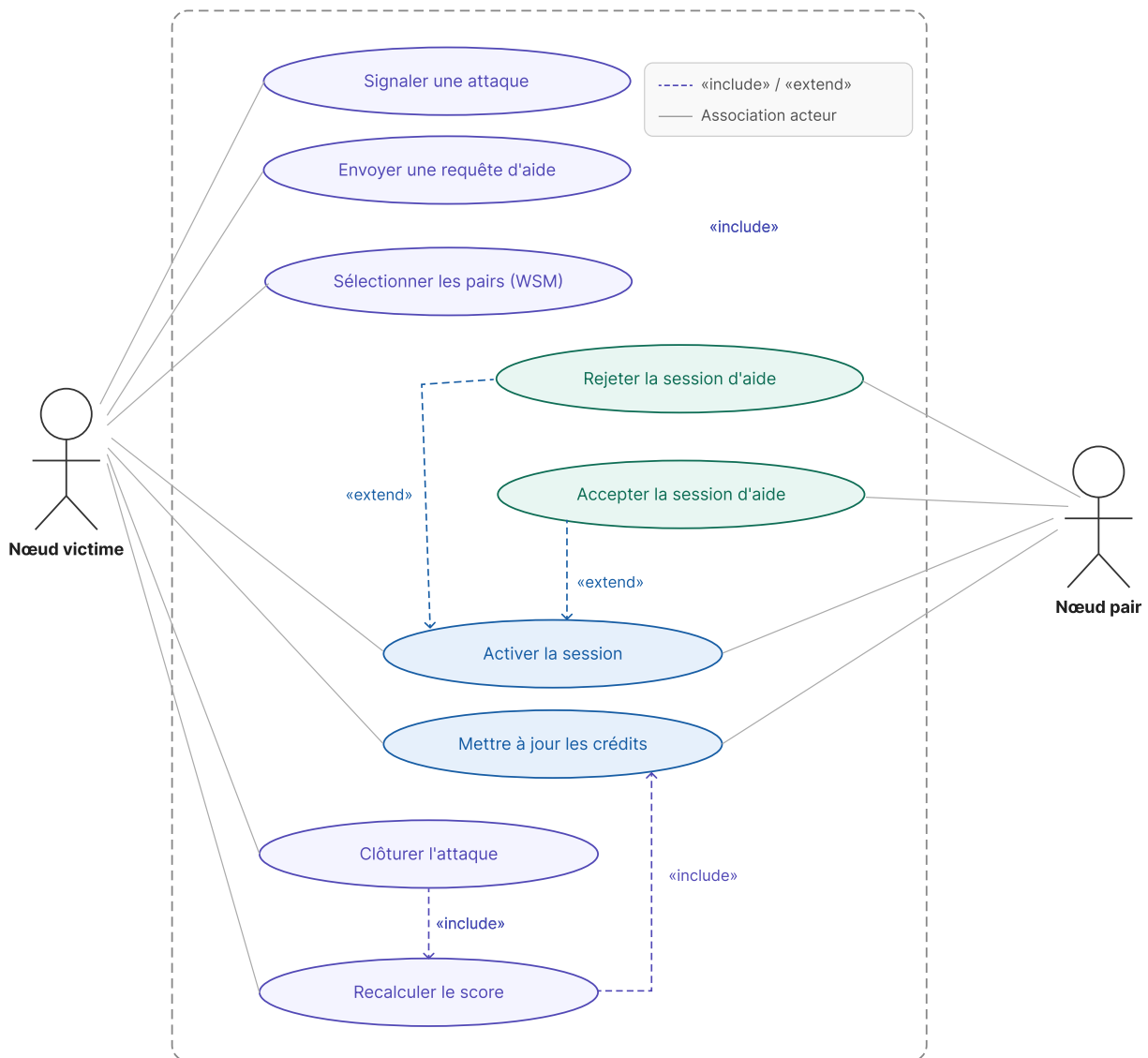


FIGURE 3.3 – Diagramme de cas d'utilisation : Protocole de mitigation collaborative.

3.2.4.2 Moteur de confiance PeerTrust

La Figure 3.4 présente les cas d’usage du moteur de confiance, centré sur le **nœud local** (évaluateur) et le **pair évalué**.

La séquence principale est déclenchée par la **clôture d’une session d’aide** : le nœud local **capture la crédibilité** Cr_i du réseau au moment de la session en interrogeant les pairs actifs (GET /trust/{node_id} sur chaque pair), puis **calcule le score PeerTrust** $T(u) = \sum[S \cdot Cr_i \cdot D] / \sum[Cr_i \cdot D]$ et **met à jour le niveau de confiance** (GOLD $\geq 0,80$; SILVER $\geq 0,60$; BRONZE $\geq 0,40$; SUSPECT $\geq 0,20$; BANNED $< 0,20$). Si le score calculé est inférieur à 0,20, la mise à jour du niveau étend automatiquement vers le **bannissement automatique** du pair (statut BANNED, membership EXPELLED). Indépendamment de ce flux, une **violation de comportement** détectée peut également déclencher le bannissement via le même mécanisme d’extension. Le **pair évalué** dispose d’un accès en lecture via **Consulter le score de confiance** (GET /trust/{id}). Enfin, l’administrateur ou le système peut **forcer le recalcul du score** (POST /trust/{id}/recalculate) à tout moment, ce cas d’usage incluant directement le calcul de $T(u)$.

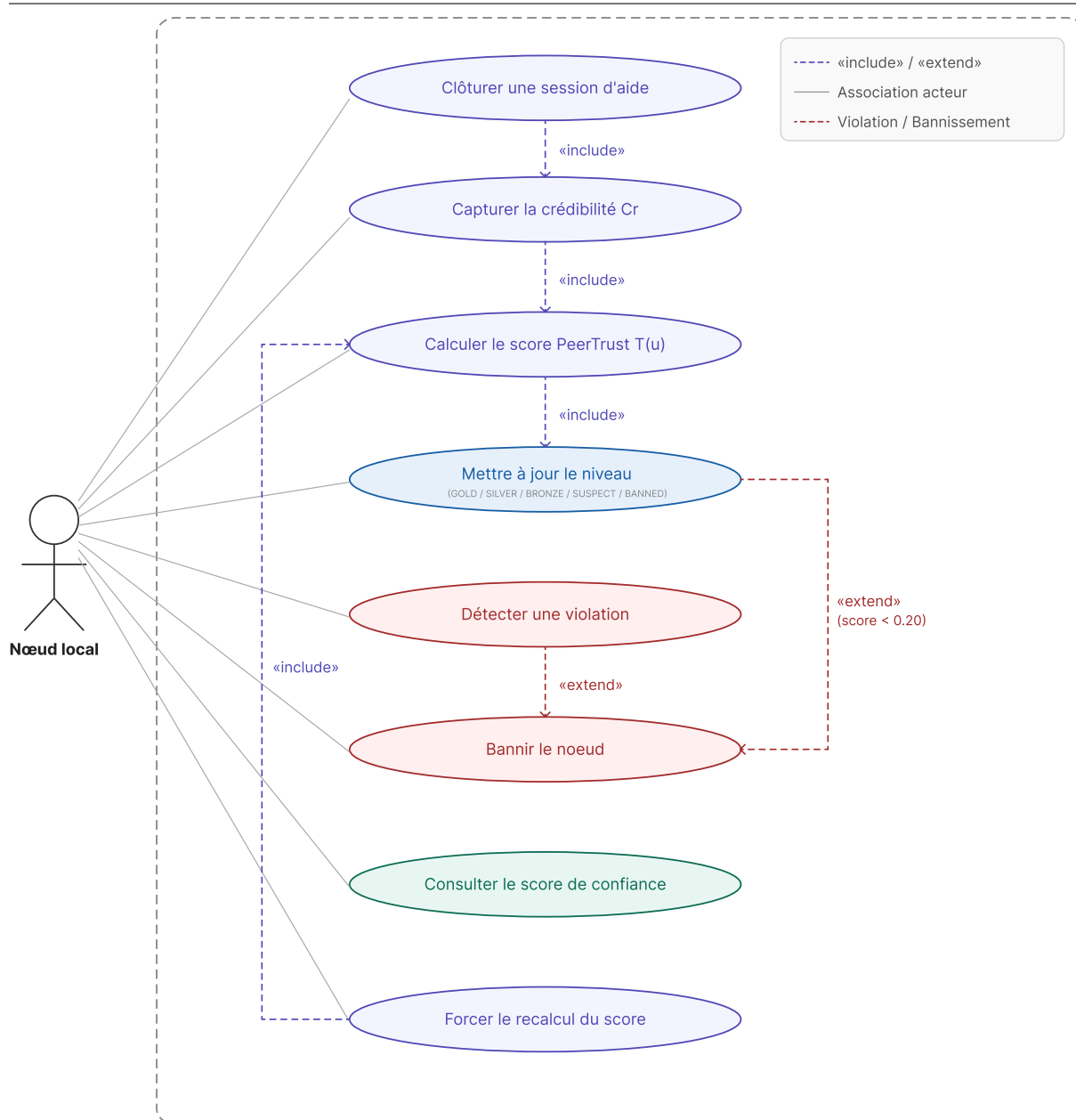


FIGURE 3.4 – Diagramme de cas d’utilisation : Moteur de confiance PeerTrust.

3.2.4.3 Rôle du nœud pair dans la coalition

La Figure 3.5 présente l’ensemble des actions qu’un **nœud pair** peut effectuer au sein de la coalition, organisées en quatre groupes fonctionnels.

Le groupe **Membership** couvre le cycle de vie complet du pair : il s’enregistre via `POST /peers/register`, action qui inclut obligatoirement la **publication de ses capacités de scrubbing** (`POST /capability/advertise`) ; il peut ensuite **découvrir des pairs disponibles** (`POST /peers/discover`) et **consulter la liste des pairs connus** (`GET /peers`) ; il envoie périodiquement un **heartbeat** (`POST /heartbeat`) — toutes les 30 secondes par défaut — pour signaler sa disponibilité et mettre à jour sa capacité déclarée ; enfin, il peut **quitter proprement la coalition** (`POST /peers/goodbye`), ce qui met son statut à **INACTIVE** ou **MAINTENANCE**.

Le groupe **Session d'aide** couvre les deux modes de participation : le pair peut **recevoir une requête d'aide** (POST /help/request) et choisir de l'**accepter** (PUT /help/{id}/accept) ou de la **rejeter** (PUT /help/{id}/reject), ou bien **proposer proactivement de l'aide** (POST /help/offer) lorsque le flag `auto_offer_enabled` est activé dans sa politique locale ; une offre acceptée étend vers le même cas d'usage *Accepter la session d'aide*.

Le groupe **Confiance** contient un seul cas d'usage : **interroger la coalition sur sa réputation** (GET /trust/{self_id} appelé sur les APIs des pairs actifs), qui correspond à la collecte de la crédibilité *Cr* utilisée dans la formule PeerTrust.

Le groupe **Sécurité** couvre l'**obtention d'un token JWT RS256** (POST /auth/token), prérequis à tout appel API authentifié.

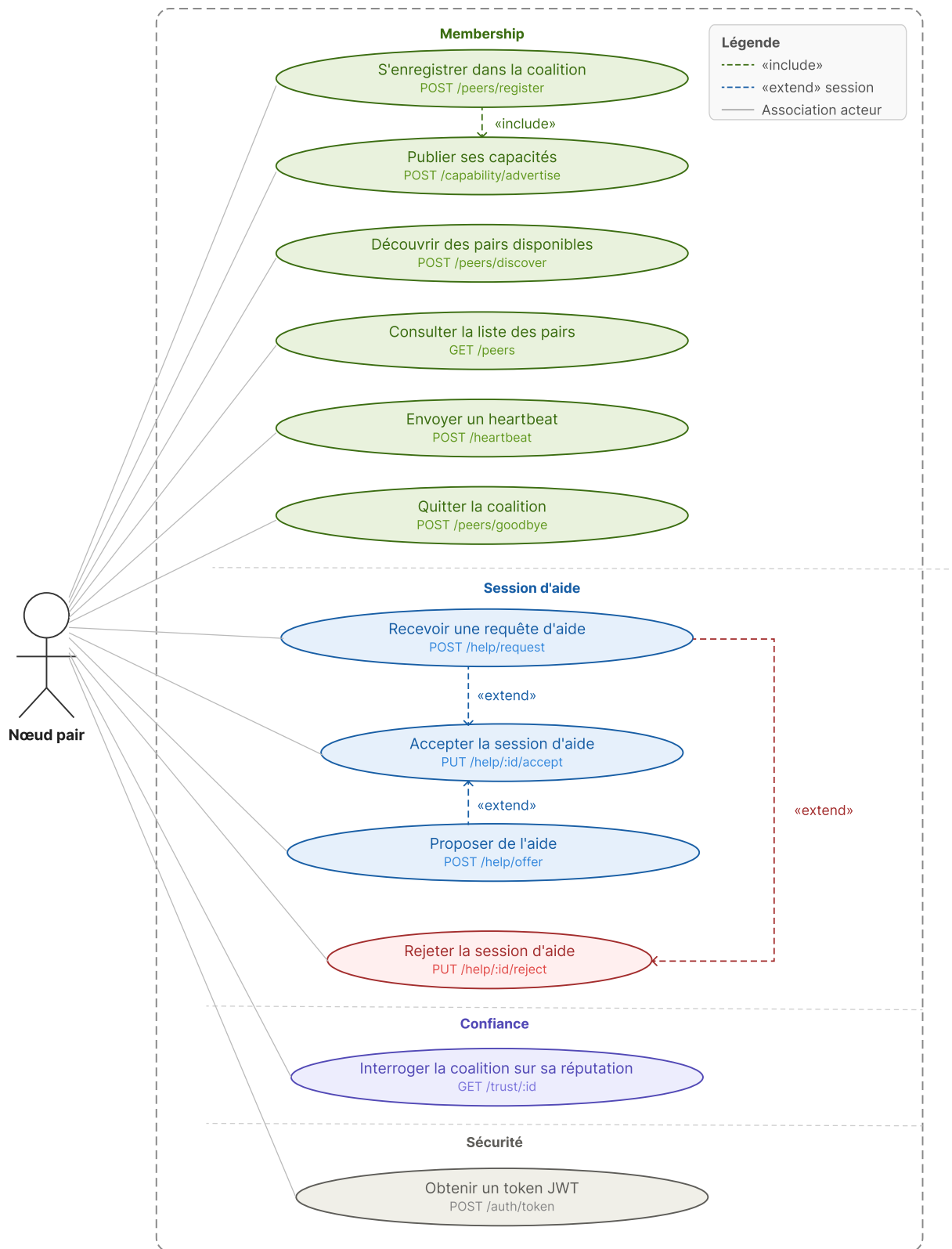


FIGURE 3.5 – Diagramme de cas d'utilisation : Rôle du nœud pair dans la coalition (12 cas d'usage).

3.2.5 Documentation des cas d'utilisation

Les tableaux suivants documentent les six principaux cas d'utilisation du système, issus des trois diagrammes présentés ci-dessus. Le Tableau 3.1 en présente la synthèse.

TABLE 3.1 – Identification des cas d'utilisation.

ID	Nom	Acteur principal	Description courte
CU 1	Obtenir un token JWT RS256	Nœud local / pair	Authentification auprès de l'API via une clé privée RS256; token valable 15 min.
CU 3	Envoyer un heartbeat	Nœud pair	Signal périodique (30 s) transmettant la charge et la capacité disponible.
CU 4	Envoyer une requête d'aide	Nœud victime	Sélection WSM des pairs et envoi de POST /help/request.
CU 5	Accepter ou rejeter une session	Nœud pair	Réponse à une requête d'aide selon la capacité disponible.
CU 6	Clôturer l'attaque	Nœud victime	Fermeture des sessions, mise à jour des crédits et recalcul PeerTrust.
CU 7	Recalculer le score PeerTrust	Moteur de confiance	Calcul de $T(u)$ et bannissement automatique si $T(u) < 0,20$.

CU 1 : Obtenir un token JWT RS256	
ID	1
Description	Le nœud (local ou pair) demande un token JWT signé RS256 afin d'accéder aux routes protégées de l'API. Le token est valide 15 minutes.
Acteurs primaires	Nœud local, nœud pair
Acteurs secondaires	Module d'authentification (composant interne du nœud cible)
Préconditions	La clé privée RSA du nœud (node.key) est présente sur le système de fichiers.
Enchaînement principal	Ce CU démarre lorsque le nœud doit appeler une route protégée de l'API. 1. Le nœud envoie POST /api/v1/auth/token avec son identifiant (node_id). 2. Le module d'authentification lit la clé privée RS256 stockée localement (node.key). 3. Le système génère un JWT signé contenant {iss, node_id, exp} avec une durée de vie de 15 minutes. 4. Le token est retourné au nœud appelant (HTTP 200). 5. Le nœud inclut ce token dans l'en-tête Authorization : Bearer <token> pour toutes ses requêtes suivantes.
Post-conditions	Le nœud dispose d'un token RS256 valide pour 15 minutes.
Enchaînements alternatifs	Clé privée introuvable : le serveur retourne HTTP 500. Token expiré lors d'un appel ultérieur : le serveur retourne HTTP 401 ; le nœud doit redemander un token via ce même CU.

TABLE 3.2 – Documentation du cas d'utilisation *Obtenir un token JWT RS256*.

CU 3 : Envoyer un heartbeat	
ID	3
Description	Un nœud pair signale périodiquement (toutes les 30 secondes) sa disponibilité et met à jour sa capacité disponible auprès du nœud local.
Acteurs primaires	Nœud pair
Acteurs secondaires	Nœud local (registre de la coalition)
Préconditions	Le nœud pair est enregistré (statut ACTIVE) et possède un token JWT valide (CU 1).
Enchaînement principal	Ce CU est déclenché automatiquement toutes les 30 secondes par le processus de monitoring du pair. 1. Le pair envoie POST /api/v1/heartbeat avec : peer_id, la charge actuelle (reported_load_pct), la capacité disponible (reported_available_gbps) et la latence aller-retour mesurée (round_trip_time_ms). 2. Le nœud local met à jour le champ last_seen du pair et ses métriques en temps réel. 3. Le nœud local retourne HTTP 200 avec le statut courant du pair.
Post-conditions	Le champ last_seen est mis à jour ; la capacité disponible reflète l'état réel du pair.
Enchaînements alternatifs	Pair banni ou inconnu : le nœud retourne HTTP 403 (Forbidden). Token JWT expiré : le nœud retourne HTTP 401 ; le pair doit se réauthentifier (CU 1) avant de retenter.

TABLE 3.3 – Documentation du cas d'utilisation *Envoyer un heartbeat*.

CU 4 : Envoyer une requête d'aide (sélection WSM)	
ID	4
Description	Le nœud victime, confronté à un volume d'attaque DDoS dépassant sa capacité locale, sélectionne automatiquement des pairs assistants via l'algorithme WSM et leur envoie une requête d'aide.
Acteurs principaux	Nœud victime (nœud local sous attaque)
Acteurs secondaires	Nœuds pairs candidats
Préconditions	Le nœud victime est authentifié (CU 1) ; au moins un pair est en statut ACTIVE dans la coalition.
Enchaînement principal	Ce CU démarre lorsque le volume d'attaque DDoS dépasse la capacité de filtrage locale du nœud. 1. Le nœud victime appelle POST /api/v1/trust/select-peers pour déclencher la sélection WSM. 2. Le système calcule pour chaque pair candidat (statut ACTIVE, non banni) le score pondéré : $\text{Score}(p) = 0,52 \cdot C_p + 0,20 \cdot L_p + 0,20 \cdot T_p + 0,08 \cdot R_p$ où C_p est la capacité disponible, L_p la latence inversée, T_p le score de confiance et R_p la réciprocité. 3. Les pairs sont classés par score décroissant. 4. Le nœud victime envoie POST /api/v1/help/request à l'ensemble des pairs retournés par la sélection WSM (étape 3), chacun recevant une allocation proportionnelle à son score (allocation_pct). 5. Une session d'aide est créée côté pair avec le statut REQUESTED.
Post-conditions	Des sessions d'aide en statut REQUESTED existent chez les pairs sélectionnés.
Enchaînements alternatifs	Aucun pair disponible (tous saturés ou bannis) : le système retourne une liste vide ; le nœud victime gère l'attaque seul. Pair banni inclus dans la liste initiale : le pair banni est automatiquement exclu du classement WSM.

TABLE 3.4 – Documentation du cas d'utilisation *Envoyer une requête d'aide (sélection WSM)*.

CU 5 : Accepter ou rejeter une session d'aide	
ID	5
Description	Un nœud pair, ayant reçu une requête d'aide (CU 4), évalue sa capacité disponible et accepte ou rejette la session.
Acteurs primaires	Nœud pair
Acteurs secondaires	Nœud victime
Préconditions	Une session d'aide en statut REQUESTED est associée au pair ; le pair est authentifié (CU 1).
Enchaînement principal	Ce CU démarre dès réception d'une requête POST /help/request par le nœud pair. 1. Le pair consulte sa capacité disponible (reported_available_gbps) et sa charge actuelle (reported_load_pct). 2. Si la capacité est suffisante pour absorber le volume demandé : * Le pair envoie PUT /api/v1/help/{id}/accept ; la session passe en statut ACCEPTED. * Le nœud victime est notifié ; la redirection du trafic peut démarrer. 3. Si la capacité est insuffisante : * Le pair envoie PUT /api/v1/help/{id}/reject ; la session passe en statut REJECTED. * Le nœud victime est notifié du refus et peut consulter d'autres pairs candidats.
Post-conditions	La session est en statut ACCEPTED ou REJECTED ; le nœud victime est informé de la décision.
Enchaînements alternatifs	Session déjà traitée (statut non REQUESTED) : le nœud retourne HTTP 409 (Conflict). Token JWT expiré : le nœud retourne HTTP 401.

TABLE 3.5 – Documentation du cas d'utilisation *Accepter ou rejeter une session d'aide*.

CU 6 : Clôturer l'attaque	
ID	6
Description	À la fin de l'attaque DDoS, le nœud victime clôture la session d'aide, met à jour les crédits de réciprocité des deux nœuds et déclenche le recalcul du score de confiance PeerTrust de chaque pair assistant.
Acteurs principaux	Nœud victime
Acteurs secondaires	Nœuds pairs assistants, moteur de confiance PeerTrust
Préconditions	Au moins une session d'aide est en statut IN_PROGRESS ; le nœud victime est authentifié (CU 1).
Enchaînement principal	Ce CU démarre lorsque le nœud victime détecte la fin de l'attaque DDoS. 1. Le nœud victime envoie POST /api/v1/attack/over avec le session_id et le volume réellement traité par le pair (actual_volume_gbps). 2. Le système calcule le ratio de satisfaction $S = \min(1,0, \text{réel/accepté})$ pour chaque pair assistant. 3. Les crédits de réciprocité sont mis à jour dans les ledgers respectifs du nœud victime et du pair assistant. 4. Le moteur de confiance recalcule le score PeerTrust $T(u)$ de chaque pair assistant (CU 7). 5. La session est marquée COMPLETED.
Post-conditions	Session en statut COMPLETED ; scores de confiance et crédits de réciprocité mis à jour.
Enchaînements alternatifs	Session introuvable : le système retourne HTTP 404. Volume réel nul ($S = 0$, pair n'a rien traité) : le score PeerTrust chute ; si $T(u) < 0,20$, le bannissement automatique est déclenché (CU 7).

TABLE 3.6 – Documentation du cas d'utilisation *Clôturer l'attaque*.

CU 7 : Recalculer le score PeerTrust (bannissement automatique)	
ID	7
Description	Le moteur de confiance recalcule le score PeerTrust $T(u)$ d'un pair à partir de l'historique de ses interactions. Si $T(u) < 0,20$, le bannissement automatique est déclenché.
Acteurs primaires	Moteur de confiance (système)
Acteurs secondaires	Pair évalué, pairs actifs (source de la crédibilité réseau Cr)
Préconditions	Au moins une interaction du pair est enregistrée en base ; ce CU est inclus par CU 6 ou déclenché manuellement via POST /api/v1/trust/{id}/recalculate.
Enchaînement principal	Ce CU démarre automatiquement à la clôture d'une session (CU 6) ou sur demande explicite. 1. Le système collecte la crédibilité réseau Cr_i en interrogeant les pairs actifs (GET /api/v1/trust/{node_id} sur chacun d'eux). 2. Le système calcule le score PeerTrust selon la formule de Xiong & Liu (2004) : $T(u) = \sum_i [S_i \cdot Cr_i \cdot D_i] / \sum_i [Cr_i \cdot D_i]$ où $S_i = \min(1, \text{réel}_i / \text{accepté}_i)$ et D_i est le poids de sévérité de l'attaque. 3. Le niveau de confiance est mis à jour selon les seuils : * $T(u) \geq 0,80$: niveau GOLD * $T(u) \geq 0,60$: niveau SILVER * $T(u) \geq 0,40$: niveau BRONZE * $T(u) \geq 0,20$: niveau SUSPECT * $T(u) < 0,20$: niveau BANNED → bannissement automatique 4. En cas de bannissement : le statut du pair passe à BANNED et son membership à EXPELLED.
Post-conditions	Le score $T(u)$ et le niveau de confiance du pair sont persistés en base.
Enchaînements alternatifs	Aucune interaction enregistrée : le score reste inchangé à sa valeur initiale (0,50). Pairs actifs inaccessibles pour Cr : la valeur par défaut $Cr_i = 1,0$ est utilisée (confiance pleine dans l'interaction locale).

TABLE 3.7 – Documentation du cas d'utilisation *Recalculer le score PeerTrust*.

3.3 Conception technique détaillée

3.3.1 Modèle de données

Le système adopte un **modèle de données relationnel**, implémenté via PostgreSQL. Ce choix est motivé par les contraintes propres au système : les relations entre pairs, sessions d'aide, scores de confiance et transactions de réciprocité forment un graphe d'entités fortement interdépendantes, pour lequel l'intégrité référentielle garantie par les clés étrangères et les contraintes de cascade est indispensable. Les types natifs de PostgreSQL (UUID, ENUM, FLOAT) et le support des transactions ACID assurent de plus la cohérence des scores calculés lors des clôtures de sessions concurrentes. Un modèle non-relationnel (document ou clé-valeur) aurait rendu ces garanties plus difficiles à maintenir sans mécanismes compensatoires complexes.

3.3.1.1 Diagrammes de classes

La base de données PostgreSQL locale de chaque nœud comporte quatorze tables, réparties en trois groupes fonctionnels : la configuration du nœud local, la gestion des pairs et de la confiance, et les sessions de mitigation. Ces trois groupes sont représentés par les diagrammes de classes ci-après ; le Tableau 3.8 décrit le rôle de chaque entité.

L'entité centrale est PEERS, qui référence les pairs connus de la coalition. Elle est reliée par des suppressions en cascade à TRUST_SCORES (un score agrégé par pair, mis à jour en place), TRUST_VIOLATIONS (historique des comportements anormaux), HEARTBEAT_LOG (signaux de présence), RECIPROCITY_LEDGER (solde crédit/débit unique par pair) et MESSAGE_LOG (journal des échanges, avec une auto-relation pour les messages de réponse). La table PEER_CAPABILITIES complète ce profil avec les capacités déclarées et vérifiées du pair.

Le nœud local est décrit par LOCAL_NODE_CONFIG, dont dépendent SCRUBBING_CAPABILITIES (capacités techniques de filtrage : débit maximal, précision de filtrage) et POLICY_CONFIG (paramètres de politique locale avec versioning : le champ is_current distingue la politique active des versions historiques, permettant de tracer l'évolution de la configuration dans le temps). Les sessions de mitigation sont modélisées par HELP_SESSIONS, reliée à ATTACKS par une clé étrangère en cascade, au pair assistant et au nœud demandeur (LOCAL_NODE_CONFIG) via des contraintes RESTRICT. Chaque session peut générer plusieurs transactions dans RECIPROCITY_TRANSACTIONS, liée à la fois à HELP_SESSIONS (SET NULL) et à PEERS (CASCADE). Enfin, AUDIT_LOG est une table autonome sans clé étrangère : ses colonnes actor et target sont de type texte libre, garantissant la persistance des entrées même après suppression d'un pair.

TABLE 3.8 – Description des tables de la base de données.

Table	Rôle
LOCAL_NODE_CONFIG	Configuration du nœud local : identité, organisation, code pays, ASN, URL d’API, clé publique, empreinte de certificat, capacité maximale de scrubbing, charge courante, date d’adhésion à la coalition.
PEERS	Registre des pairs connus : organisation, type d’organisation, code pays, ASN, URL d’API, clé publique, empreinte de certificat, capacité déclarée, latence mesurée, statut de membership et type de relation.
TRUST_SCORES	Score de confiance d’un pair : score global PeerTrust, niveau (GOLD à BANNED), sous-scores de fiabilité, performance, réciprocité et stabilité, statistiques de sessions et date de dernier calcul.
ATTACKS	Journal des attaques détectées : volume de pointe, volume en débordement, capacité locale au moment de la détection, sévérité, nombre de pairs impliqués, statut et durée.
HELP_SESSIONS	Sessions d’aide inter-nœuds : nœud demandeur, pair assistant, direction, volumes acceptés et réels, crédits échangés, note de qualité, valeur C_r capturée à la clôture, horodatages des phases clés.
RECIPROCITY_LEDGER	Solde de réciprocité d’un pair : crédits donnés, reçus et solde net en Gbps cumulés, date de dernière transaction.
POLICY_CONFIG	Paramètres de politique locale avec versioning : score de confiance minimal requis, pourcentage de capacité partageable, fréquence de heartbeat, activation des offres automatiques, indicateur de version active (is_current).
TRUST_VIOLATIONS	Violations de comportement détectées : type, sévérité, description, sanction appliquée et date d’expiration de la sanction.
HEARTBEAT_LOG	Historique des signaux de présence reçus des pairs : statut reporté, charge, capacité disponible et latence aller-retour.
AUDIT_LOG	Journal d’audit immuable : type d’événement, sévérité, acteur, cible, description horodatée ; sans clé étrangère pour garantir la persistance après suppression d’un pair.
SCRUBBING_CAPABILITIES	Capacités techniques de filtrage du nœud local : débit maximal en Gbps, précision de filtrage et statut d’activation.
MESSAGE_LOG	Journal des messages échangés entre nœuds : type, direction, priorité, validité de signature, résultat de traitement, raison de rejet et auto-référence pour les messages de réponse.
PEER_CAPABILITIES	Capacités déclarées d’un pair distant : capacité en Gbps, statut et date de vérification.
RECIPROCITY_TRANSACTIONS	Journal des transactions de réciprocité : type, montant en crédits (credit_amount), session associée (nul-

La Figure 3.6 présente la vue d'ensemble du modèle de classes, regroupant les quatorze entités en un seul diagramme global. Les trois sections suivantes en détaillent chaque groupe fonctionnel.

3.3.1.2 Diagramme de classes — Configuration du nœud local

La classe centrale `LocalNodeConfig` regroupe l'identité complète du nœud (nom, organisation, type d'organisation, code pays, ASN), ses paramètres réseau (URL d'API, port, clé publique RSA, empreinte de certificat), sa capacité maximale de scrubbing et sa charge courante. Elle est associée à `ScrubbingCapability`, qui décrit les capacités techniques de filtrage disponibles (débit maximal, précision, statut d'activation), et à `PolicyConfig` (relation 1..* avec versioning), qui centralise les paramètres de politique locale : seuil de confiance minimal requis pour accepter une aide, pourcentage de capacité partageable, intervalle de heartbeat, activation des offres automatiques et indicateur de version active (`is_current`).

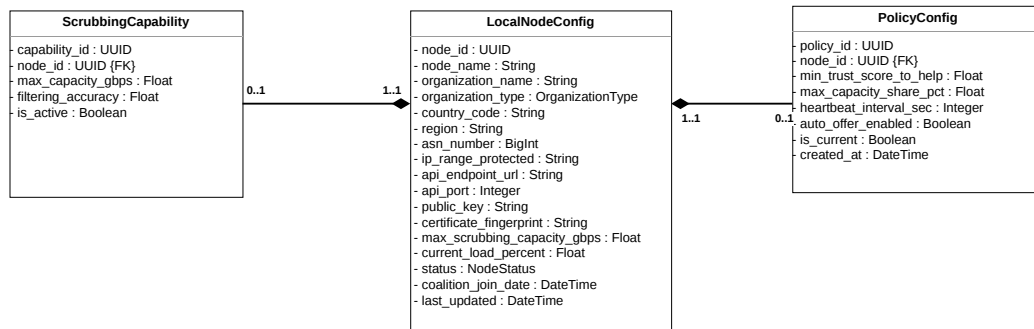


FIGURE 3.7 – Diagramme de classes — Configuration du nœud local.

TABLE 3.9 – Classes — Configuration du nœud local.

Classe	Attributs principaux	Description
LocalNodeConfig	node_id, node_name, organization_name, organization_type, country_code, asn_number, api_endpoint_url, api_port, public_key, certificate_fingerprint, max_scrubbing_capacity_gbps, current_load_percent, status, coalition_join_date, last_updated	Identité et configuration complète du nœud local ; clé publique RSA utilisée pour la vérification JWT RS256 ; charge courante mise à jour à chaque heartbeat.
ScrubbingCapability	capability_id, node_id, max_capacity_gbps, filtering_accuracy, is_active	Capacités techniques de filtrage du nœud local : débit maximal supporté, précision de filtrage (0–1) et statut d'activation.
PolicyConfig	policy_id, node_id, min_trust_score_to_help, max_capacity_share_pct, heartbeat_interval_sec, auto_offer_enabled, is_current, created_at	Paramètres de politique locale avec versioning : <code>is_current</code> identifie la politique active ; les versions précédentes sont conservées pour traçabilité.

3.3.1.3 Diagramme de classes — Gestion des pairs et de la confiance

La classe Peer constitue l'entité centrale : elle agrège TrustScore (score PeerTrust global et sous-scores de fiabilité, performance, réciprocité et stabilité, avec compteurs statistiques de sessions), ReciprocityLedger (solde crédit/débit avec balance nette), PeerCapability (capacité déclarée et statut de vérification), TrustViolation (historique des comportements anormaux avec sanction et date d'expiration), HeartbeatLog (signaux de présence avec charge et latence mesurée) et MessageLog (journal des échanges avec priorité, validité de signature et auto-association pour les messages de réponse). La classe Peer elle-même porte des attributs réseau opérationnels : consecutive_missed_heartbeats pour la détection de défaillance, relationship_type pour qualifier le mode de découverte du pair, et first_seen/last_heartbeat pour la gestion du cycle de vie.

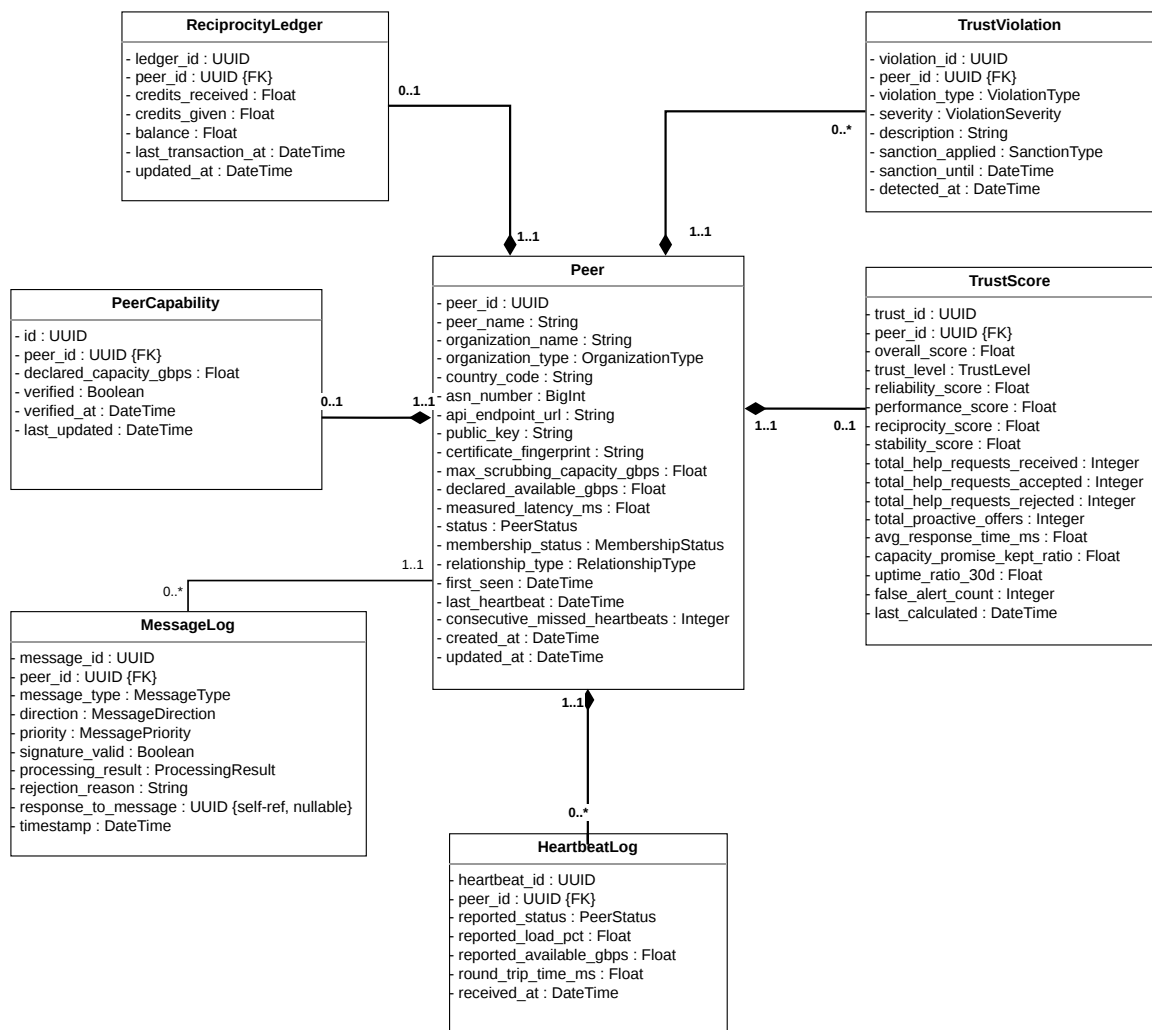


FIGURE 3.8 – Diagramme de classes — Gestion des pairs et de la confiance.

TABLE 3.10 – Classes — Gestion des pairs et de la confiance.

Classe	Attributs principaux	Description
Peer	peer_id, peer_name, organization_name, organization_type, country_code, asn_number, api_endpoint_url, public_key, certificate_fingerprint, max_scrubbing_capacity_gbps, declared_available_gbps, measured_latency_ms, status, membership_status, relationship_type, first_seen, last_heartbeat, consecutive_missed_heartbeats	Pair connu de la coalition : identité complète, capacité déclarée disponible, statut de membership et compteur de heartbeats manqués pour détection de défaillance.
TrustScore	trust_id, peer_id, overall_score, trust_level, reliability_score, performance_score, reciprocity_score, stability_score, total_help_requests_received, total_help_requests_accepted, total_help_requests_rejected, total_proactive_offers, avg_response_time_ms, capacity_promise_kept_ratio, uptime_ratio_30d, false_alert_count, last_calculated	Score PeerTrust global et sous-scores analytiques ; compteurs de sessions pour le suivi comportemental du pair.
ReciprocityLedger	ledger_id, peer_id, credits_received, credits_given, balance, last_transaction_at, updated_at	Solde de réciprocité du pair : crédits donnés, reçus et balance nette en Gbps cumulés ; utilisé dans le critère R_p de la sélection WSM.
PeerCapability	id, peer_id, declared_capacity_gbps, verified, verified_at, last_updated	Capacité déclarée du pair distant avec statut et date de vérification.
TrustViolation	violation_id, peer_id, violation_type, severity, description, sanction_applied, sanction_until, detected_at	Historique des comportements anormaux détectés (free-riding, non-respect des engagements) ; sanction_until indique la durée d’application de la sanction.
HeartbeatLog	heartbeat_id, peer_id, reported_status, reported_load_pct, reported_available_gbps, round_trip_time_ms, received_at	Enregistrement horodaté des signaux de présence ; utilisé pour détecter les nœuds injoignables et mesurer la latence réseau.
MessageLog	message_id, peer_id, message_type, direction, priority, signature_valid, processing_result, rejection_reason, response_to_message, timestamp	Journal des échanges inter-nœuds avec validité de signature et priorité ; auto-référence response_to_message pour

3.3.1.4 Identifiants et intégrité référentielle

Toutes les clés primaires sont des Universally Unique Identifier (UUID) version 4 (RFC 4122 (LEACH et al., 2005)), générés localement par chaque nœud. Ce choix garantit l'unicité globale sans coordinateur centralisé et facilite la fusion éventuelle de registres entre nœuds lors de la découverte. Les suppressions en cascade sont configurées pour les entités dépendantes (sessions, scores de confiance, ledger) afin de maintenir l'intégrité référentielle lors du retrait d'un pair.

3.3.1.5 Diagramme de classes — Sessions de mitigation et traçabilité

La classe `Attack` représente une attaque DDoS détectée et compose une ou plusieurs `HelpSession`, chacune associant un nœud victime (`requesting_node_id`, FK vers `LOCAL_NODE_CONFIG`) à un pair assistant (`helping_peer_id`, FK vers `PEERS`) avec les volumes acceptés, réels, les crédits échangés, la note de qualité et le score de crédibilité Cr capturé à la clôture. `Attack` porte les métriques volumétriques clés : volume de pointe, volume en débordement et capacité locale au moment de la détection, qui permettent de calculer le ratio S_i utilisé dans `PeerTrust`. `ReciprocityTransaction` enregistre les mouvements de crédits (`credit_amount`) générés à la fin de chaque session. La classe `AuditLog`, sans clé étrangère, constitue un journal d'audit immuable consignant les bannissements, les changements de niveau de confiance et les échecs d'authentification, avec un niveau de sévérité propre.

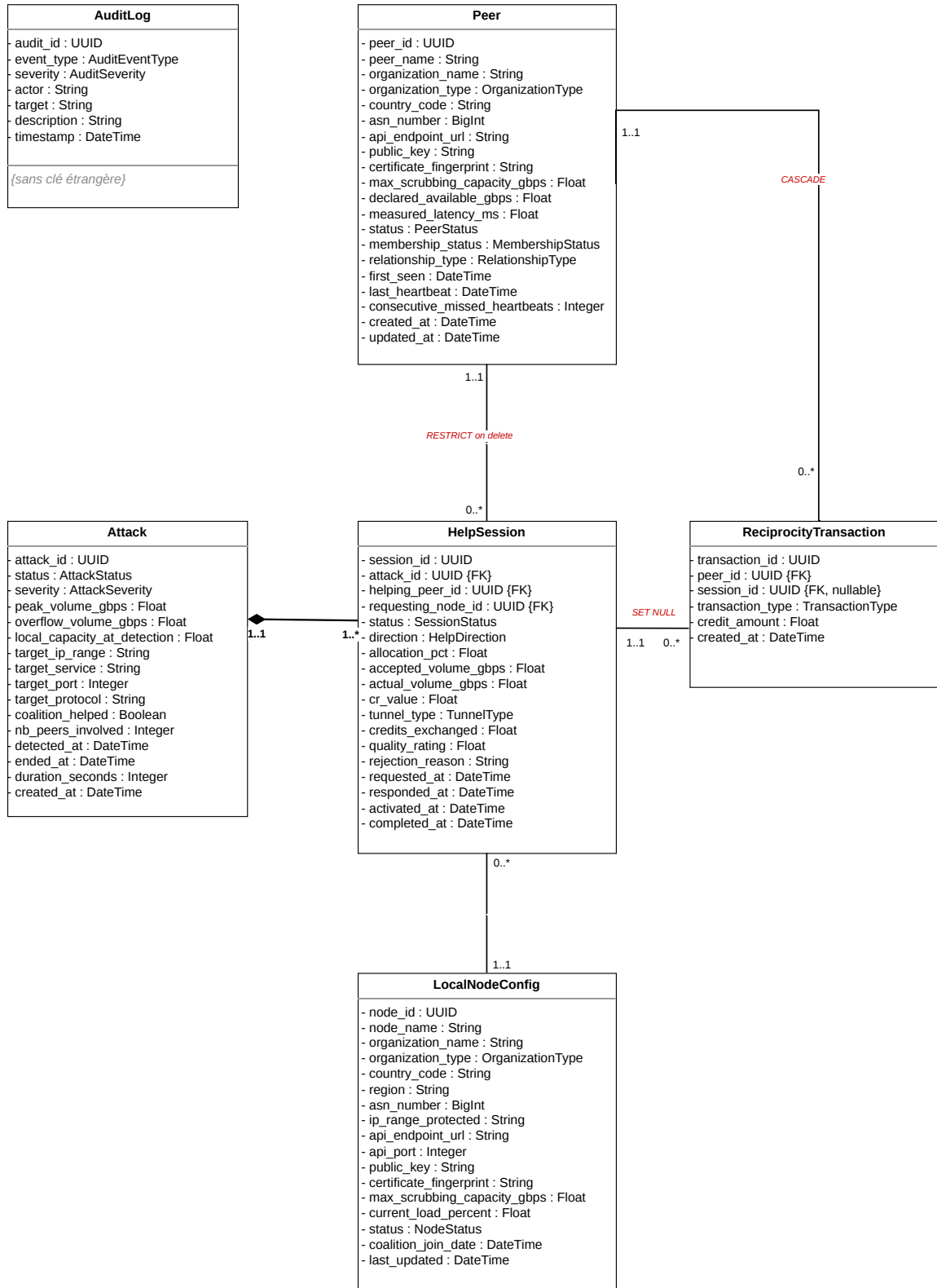


FIGURE 3.9 – Diagramme de classes — Sessions de mitigation et traçabilité.

TABLE 3.11 – Classes — Sessions de mitigation et traçabilité.

Classe	Attributs principaux	Description
Attack	attack_id, status, severity, peak_volume_gbps, overflow_volume_gbps, local_capacity_at_detection, target_ip_range, target_service, target_port, target_protocol, coalition_helped, nb_peers_involved, detected_at, ended_at, duration_seconds, created_at	Attaque DDoS détectée ; porte les métriques volumétriques (volume de pointe, débordement, capacité locale) et les sessions de mitigation associées.
HelpSession	session_id, attack_id, helping_peer_id, requesting_node_id, status, direction, allocation_pct, accepted_volume_gbps, actual_volume_gbps, cr_value, tunnel_type, credits_exchanged, quality_rating, rejection_reason, requested_at, responded_at, activated_at, completed_at	Session d'aide entre le nœud demandeur et un pair assistant ; cr_value capture la crédibilité réseau à la clôture ; credits_exchanged alimente le ledger de réciprocité.
ReciprocityTransaction	transaction_id, session_id, peer_id, transaction_type, credit_amount, created_at	Mouvement de crédits (credit_amount en Gbps) généré à la clôture d'une session ; lié à la session (SET NULL) et au pair (CASCADE).
AuditLog	audit_id, event_type, severity, actor, target, description, timestamp	Journal d'audit immuable sans clé étrangère : les colonnes actor et target sont en texte libre pour garantir la persistance des entrées même après suppression d'un pair.

3.3.1.6 Passage au modèle relationnel

Le modèle objet présenté par les diagrammes de classes est transposé en modèle relationnel selon quatre règles de correspondance.

1. **Chaque classe devient une table.** Les quatorze classes du modèle objet correspondent aux quatorze tables de la base de données (cf. Tableau 3.8).
2. **Les attributs deviennent des colonnes.** Chaque attribut d'une classe est mappé à une colonne de la table correspondante, avec le type PostgreSQL approprié (UUID, VARCHAR, FLOAT, ENUM, TIMESTAMP, JSONB).
3. **Les associations deviennent des clés étrangères.** Chaque lien de composition ou d'association entre classes génère une contrainte FOREIGN KEY dans la table dépendante, avec la politique de suppression adaptée au rôle de la relation

(CASCADE, RESTRICT ou SET NULL).

4. **Les relations plusieurs-à-plusieurs deviennent des tables d'association.**

Dans ce modèle, aucune relation M :N directe n'est présente ; la table `HELP_SESSIONS` joue le rôle de table d'association entre `ATTACKS` et `PEERS`, en y ajoutant les attributs propres à chaque session (volumes acceptés, statut, crédibilité capturée).

Le Tableau 3.12 récapitule les principales contraintes de clés étrangères et les politiques de suppression retenues.

TABLE 3.12 – Principales clés étrangères et politiques de suppression.

Table	Clé étrangère	Référence	Politique
TRUST_SCORES	peer_id	PEERS	CASCADE
TRUST_VIOLATIONS	peer_id	PEERS	CASCADE
HEARTBEAT_LOG	peer_id	PEERS	CASCADE
RECIPROCITY_LEDGER	peer_id	PEERS	CASCADE
PEER_CAPABILITIES	peer_id	PEERS	CASCADE
MESSAGE_LOG	peer_id	PEERS	CASCADE
	response_to	MESSAGE_LOG	SET NULL
SCRUBBING_CAPABILITY	node_id	LOCAL_NODE_CONFIG	CASCADE
POLICY_CONFIG	node_id	LOCAL_NODE_CONFIG	CASCADE
HELP_SESSIONS	attack_id	ATTACKS	CASCADE
	helping_peer_id	PEERS	RESTRICT
RECIPROCITY_TRANSACTIONS	session_id	HELP_SESSIONS	SET NULL
	peer_id	PEERS	CASCADE

3.3.2 Modèle de confiance : PeerTrust adapté

3.3.2.1 Motivation du choix de PeerTrust

La littérature propose plusieurs modèles de réputation pour les systèmes P2P. Le modèle PeerTrust (XIONG & LIU, 2004) a été retenu pour trois raisons : il intègre nativement un **facteur contextuel** $D(u, i)$ permettant de pondérer les transactions selon leur importance (ici, la sévérité de l'attaque) ; il inclut un mécanisme de **crédibilité** $Cr(p)$ qui réduit l'influence des évaluateurs peu fiables ; et sa formule est suffisamment simple pour être recalculée en temps réel après chaque session.

3.3.2.2 Formule $T(u)$ et ses paramètres

Le score de confiance $T(u)$ d'un pair u est calculé selon l'équation :

$$T(u) = \frac{\sum_{i=1}^n S(u, i) \cdot Cr_i \cdot D(u, i)}{\sum_{i=1}^n Cr_i \cdot D(u, i)} \quad (3.1)$$

où n est le nombre de sessions complétées. Le paramètre $S(u, i) = \min\left(1, \frac{V_{\text{réel}}}{V_{\text{accepté}}}\right)$ représente la **satisfaction normalisée** de la session i : rapport entre le volume effectivement filtré et le volume promis par le pair, avec $S \in [0, 1]$. Le paramètre Cr_i est la **crédibilité historique** du nœud local au moment de la session i : moyenne des scores que les pairs actifs du réseau attribuent au nœud local, capturée à la clôture de la session et persistée dans le champ `cr_value` de la table `HELP_SESSIONS`. Cette approche garantit que chaque session est évaluée avec la crédibilité en vigueur à l'époque des faits. Enfin, $D(u, i)$ est le **facteur contextuel** lié à la sévérité de l'attaque de la session i , dont les valeurs retenues sont présentées dans le Tableau 3.13.

TABLE 3.13 – Facteur contextuel $D(u, i)$ selon la sévérité de l'attaque.

Sévérité	$D(u, i)$	Seuil de volume
LOW	0.25	< 1 Gbps
MEDIUM	0.50	1 – 10 Gbps
HIGH	0.75	10 – 100 Gbps
CRITICAL	1.00	> 100 Gbps

Les seuils de volume et les valeurs de $D(u, i)$ constituent des **choix de conception** motivés par deux considérations. Les **seuils de volume** retenus (< 1 Gbps, 1–9 Gbps, 10–99 Gbps et ≥ 100 Gbps) sont directement inspirés de la classification par sévérité utilisée par StormWall (STORMWALL, 2024), fournisseur industriel de protection DDoS, qui définit quatre niveaux (*Low*, *Medium*, *High*, *Very High*) sur ces mêmes bornes volumétriques ; le niveau *Very High* de StormWall est renommé CRITICAL dans ce système pour refléter son caractère maximal. Les **valeurs de $D(u, i)$** forment quant à elles une progression arithmétique régulière de pas 0,25, ancrée à $D = 1,00$ pour CRITICAL (poids maximal) et à $D = 0,25$ pour LOW (poids minimal non nul, afin que même les sessions mineures contribuent au score). Ce choix s'inscrit dans le cadre de Xiong et Liu (XIONG & LIU, 2004), qui laissent explicitement la définition de $D(u, i)$ à l'appréciation du concepteur, en précisant qu'il doit refléter l'importance relative de la transaction dans son contexte opérationnel.

La normalisation par $\sum Cr_i \cdot D(u, i)$ garantit $T(u) \in [0, 1]$ sans troncature artificielle, en s'inspirant du schéma de normalisation de l'équation 3 de Xiong et Liu (XIONG & LIU, 2004). Une session avec une sévérité CRITICAL a un poids quatre fois supérieur à une session LOW : aider lors d'une attaque massive est valorisé davantage.

3.3.2.3 Niveaux de confiance

Le score numérique $T(u)$ est converti en un niveau discret selon les seuils du Tableau 3.14. Le niveau BANNED déclenche automatiquement l'exclusion du pair de la coalition : son statut passe à EXPELLED et il ne reçoit plus de requêtes d'aide.

TABLE 3.14 – Niveaux de confiance dans le système.

Niveau	Seuil $T(u)$	Signification opérationnelle
GOLD	≥ 0.80	Pair prioritaire dans la sélection
SILVER	≥ 0.60	Pair fiable, sélectionnable normalement
BRONZE	≥ 0.40	Pair acceptable ; score initial par défaut
SUSPECT	≥ 0.20	Pair sous surveillance ; pondération réduite
BANNED	< 0.20	Exclusion automatique de la coalition

3.3.3 Algorithme de sélection des pairs : WSM

3.3.3.1 Critères de sélection

Lorsqu'un nœud doit sélectionner des pairs assistants, il calcule pour chaque candidat un score composite selon le **Weighted Sum Model** (WSM) (HWANG & YOON, 1981) :

$$\text{Score}(p) = w_C \cdot C_p + w_L \cdot L'_p + w_T \cdot T_p + w_R \cdot R_p \quad (3.2)$$

Les quatre critères sont définis comme suit : $C_p = \frac{\text{cap_disp}(p)}{\max_j(\text{cap_disp}(j))}$ est la capacité disponible normalisée ; $L'_p = 1 - \frac{L_p - L_{\min}}{L_{\max} - L_{\min}}$ est la latence normalisée inversée (une latence faible donne un score élevé) ; T_p est le score de confiance PeerTrust du pair p ; et $R_p = \frac{\text{crédits_reçus}}{\text{crédits_reçus} + \text{crédits_donnés} + \varepsilon}$ est la réciprocité bilatérale, avec $\varepsilon = 10^{-6}$ pour éviter la division par zéro.

3.3.3.2 Pondération AHP

Les poids w_C, w_L, w_T, w_R ont été déterminés par la méthode AHP de Saaty (SAATY, 1980). La matrice de comparaison par paires (Tableau 3.15) exprime les préférences relatives entre critères ; la cohérence est vérifiée par le **ratio de cohérence** $CR = 1,6 \% < 10 \%$, ce qui valide les poids retenus.

TABLE 3.15 – Matrice AHP de comparaison des critères de sélection des pairs.

Critère	Capacité	Latence	Confiance	Réciprocité	Poids w
Capacité (C_p)	1	3	3	7	0.52
Latence (L'_p)	1/3	1	1	3	0.20
Confiance (T_p)	1/3	1	1	3	0.20
Réciprocité (R_p)	1/7	1/3	1/3	1	0.08
Ratio de cohérence CR					1,6 %

La capacité disponible reçoit le poids dominant (0.52) car c'est le facteur limitant direct lors d'une attaque volumétrique : un pair très fiable mais saturé ne peut pas contribuer efficacement. La latence et la confiance ont des poids égaux (0.20 chacun) ; la réciprocité, facteur d'équité à long terme, est pondérée à 0.08.

3.3.3.3 Répartition proportionnelle du flux

Tous les pairs éligibles participent à la mitigation. Le flux est réparti proportionnellement aux scores WSM :

$$w_i = \frac{\text{Score}(p_i)}{\sum_j \text{Score}(p_j)}, \quad \text{allocation}(p_i) = \lfloor w_i \times 100 \rfloor \% \quad (3.3)$$

Si le volume excédentaire total V_{overflow} est connu, le volume estimé alloué à chaque pair est :

$$V_i = \min(w_i \times V_{\text{overflow}}, \text{cap_disp}(p_i)) \quad (3.4)$$

Le plafonnement par la capacité réelle du pair évite de lui demander plus qu'il ne peut absorber. La Figure 3.10 illustre ce mécanisme de distribution : le trafic entrant E est absorbé localement jusqu'à C_{locale} ; le surplus (*overflow*) est ensuite réparti entre les pairs proportionnellement à leurs scores WSM, chacun recevant au plus sa capacité déclarée C_{p_i} .

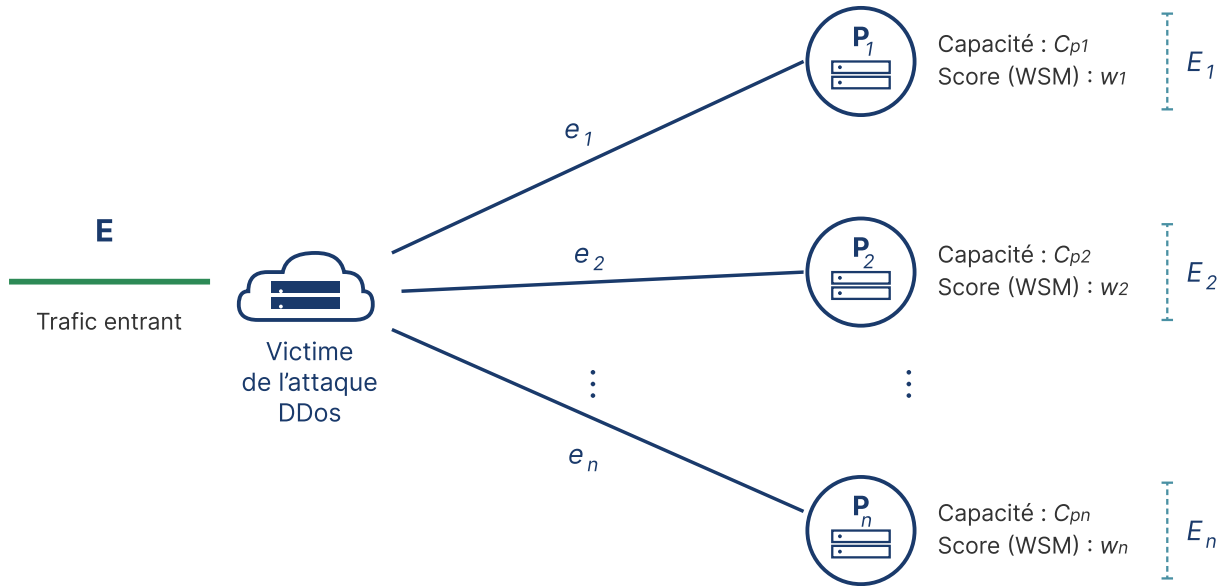


FIGURE 3.10 – Distribution du trafic selon l’algorithme WSM : répartition proportionnelle de l’overflow entre les pairs sélectionnés.

3.3.4 Diagramme d’activités : session de mitigation

La Figure 3.11 illustre le flux d’activités d’une session de mitigation collaborative, réparti en deux couloirs : le **nœud victime** et le **nœud pair**. Dès la détection d’une attaque DDoS, le nœud victime sélectionne les pairs candidats via l’algorithme WSM. Si aucun pair n’est disponible, le nœud gère l’attaque seul. Dans le cas contraire, une demande d’aide est envoyée (`POST /help/offer`) ; le nœud pair vérifie uniquement si le demandeur est banni (`BANNED`) : si c’est le cas, la demande est rejetée avec un code 403 ; sinon, une session locale est créée avec le statut `OFFERED` et une réponse 201 Created est retournée. Le nœud pair peut ensuite accepter la session via `PUT /help/{id}/accept` (statut `ACCEPTED`) ou la rejeter via `PUT /help/{id}/reject` (statut `REJECTED`). Le nœud victime active alors la redirection du trafic (`POST /traffic/redirect`), qui enregistre le type de tunnel et le volume en base de données. À la fin de l’attaque, le nœud victime notifie la clôture (`POST /attack/over`) ; les sessions passent au statut `COMPLETED` et les crédits de réciprocité ainsi que les scores de confiance sont recalculés.

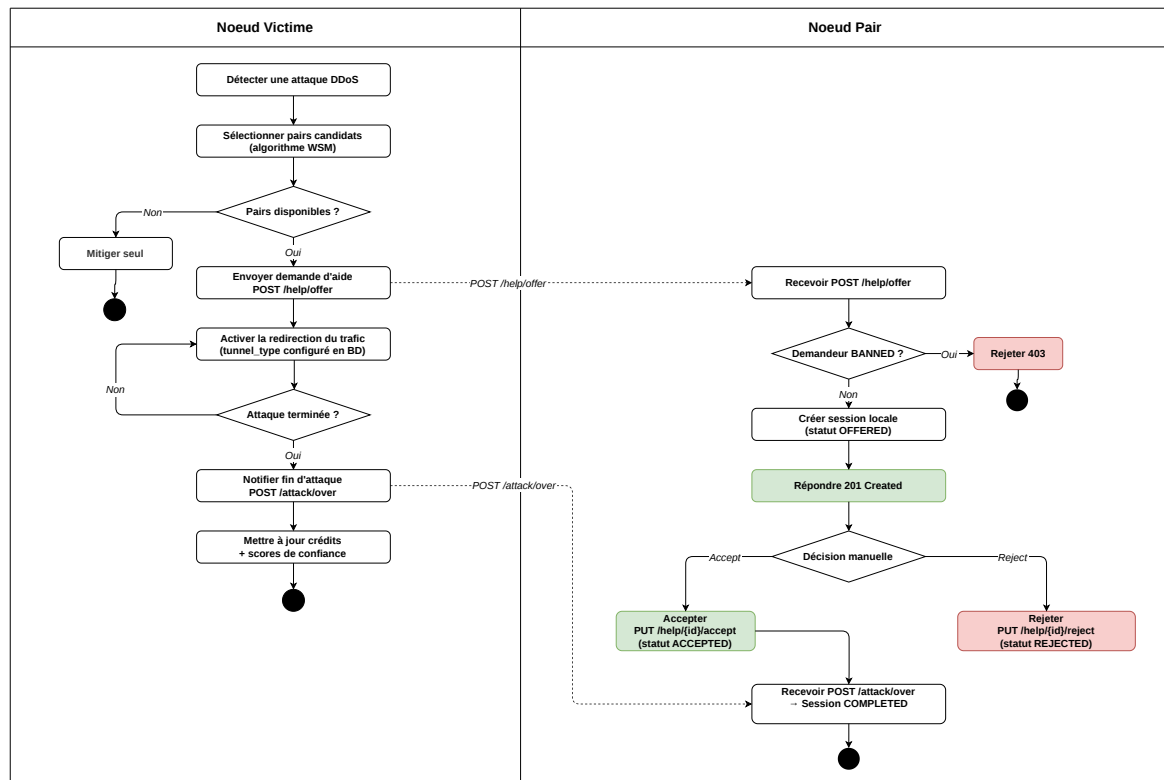


FIGURE 3.11 – Diagramme d’activités d’une session de mitigation collaborative (deux couloirs : nœud victime et nœud pair).

3.3.5 Diagrammes de séquence

Les diagrammes de séquence ci-après détaillent les échanges de messages entre composants pour les trois interactions fondamentales du système : l'authentification, la session de mitigation collaborative et le recalcul du score de confiance PeerTrust.

3.3.5.1 DS1 — Authentification JWT RS256 (CU1)

La Figure 3.12 décrit le flux d'authentification. Le nœud appelant envoie une requête `POST /api/v1/auth/token` à l'API, qui délègue au module `AuthMiddleware` la lecture de la clé privée RSA (`certs/node.key`) et la génération d'un token JWT RS256 valable 60 secondes. Deux cas alternatifs sont représentés : la clé disponible (200 OK avec token) et la clé introuvable (500 Internal Server Error). Une seconde séquence illustre l'utilisation du token dans l'en-tête `Authorization: Bearer` et le renouvellement automatique en cas d'expiration (401 Unauthorized).

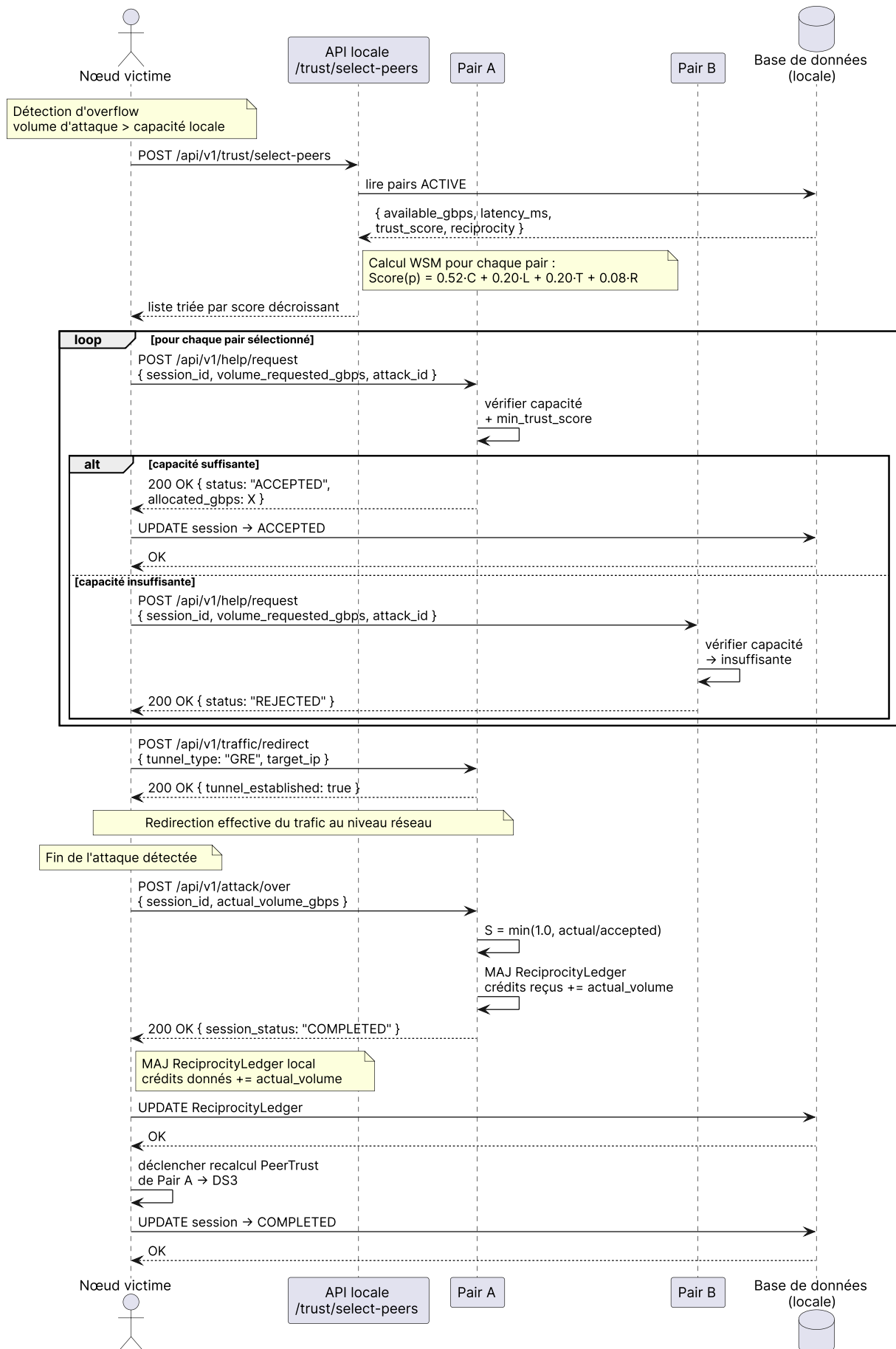


FIGURE 3.12 – DS1 — Diagramme de séquence : Authentication JWT RS256 (CU1).

3.3.5.2 DS2 — Session de mitigation collaborative (CU4 + CU5 + CU6)

La Figure 3.13 couvre le flux complet d’une session de mitigation. Le nœud victime déclenche la sélection WSM (`POST /trust/select-peers`), reçoit la liste triée par score décroissant, puis envoie une requête d’aide à chaque pair sélectionné. Le pair A accepte (capacité suffisante) et le pair B rejette (capacité insuffisante). Une fois la session acceptée, la redirection du trafic est activée via `POST /traffic/redirect`. À la fin de l’attaque, `POST /attack/over` clôture la session, met à jour les ledgers de réciprocité des deux nœuds et déclenche le recalcul PeerTrust (DS3).

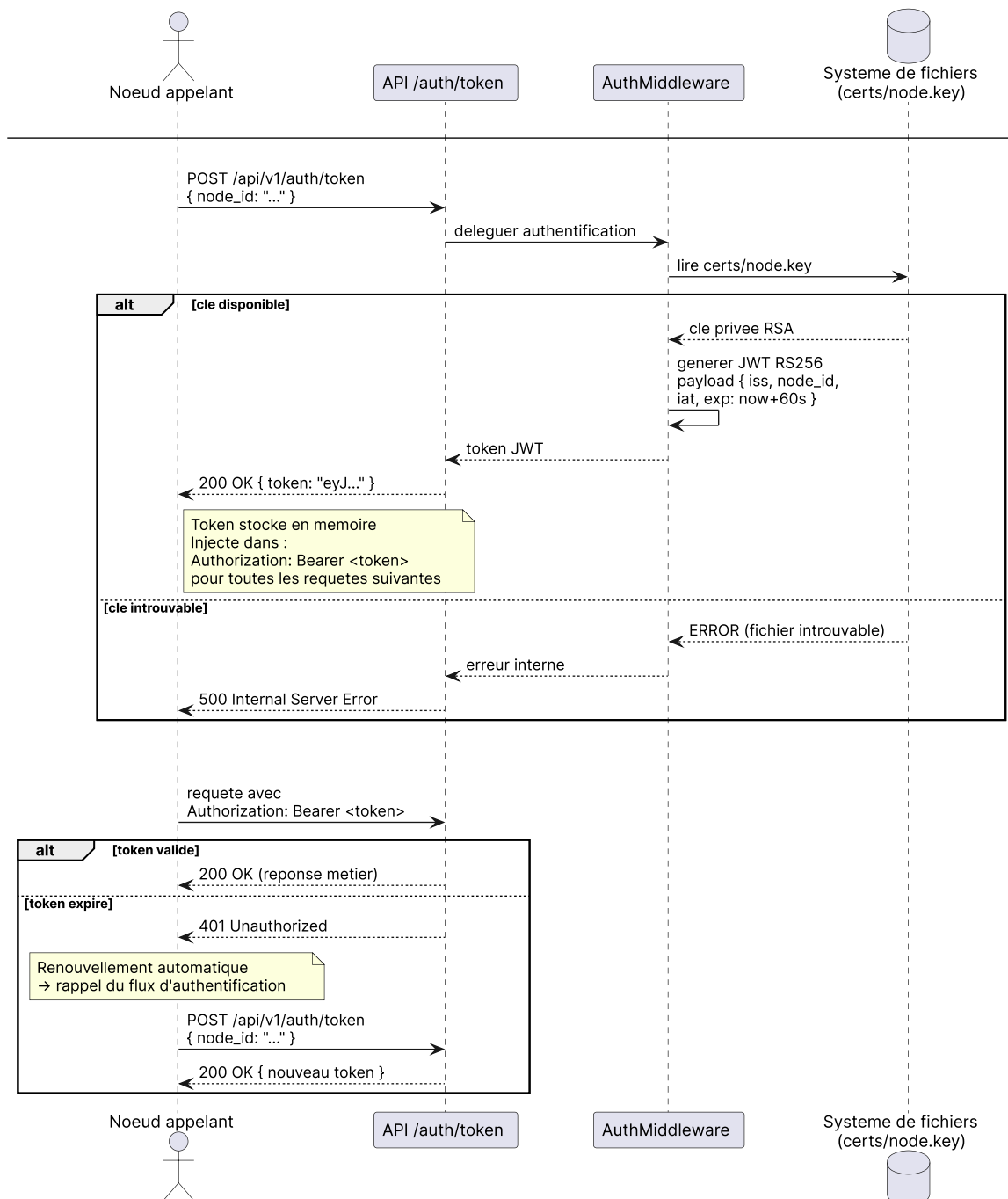


FIGURE 3.13 – DS2 — Diagramme de séquence : Session de mitigation collaborative (CU4 + CU5 + CU6).

3.3.5.3 DS3 — Recalcul du score PeerTrust (CU7)

La Figure 3.14 détaille le recalcul du score $T(u)$, organisé en trois phases. La **phase 1** collecte la crédibilité réseau Cr_i en interrogeant chaque pair actif via `GET /trust/{self_node_id}`; si un pair est injoignable, la valeur par défaut $Cr_i = 1,0$ est utilisée. La **phase 2** charge les sessions complétées du pair évalué et calcule $T(u) = \sum[S_i \cdot Cr_i \cdot D_i] / \sum[Cr_i \cdot D_i]$. La **phase 3** met à jour le niveau de confiance et, si $T(u) < 0,20$, déclenche le bannissement automatique avec inscription dans `AUDIT_LOG`. Si aucune session n'est enregistrée, le score est conservé à sa valeur initiale (0,50 / BRONZE).

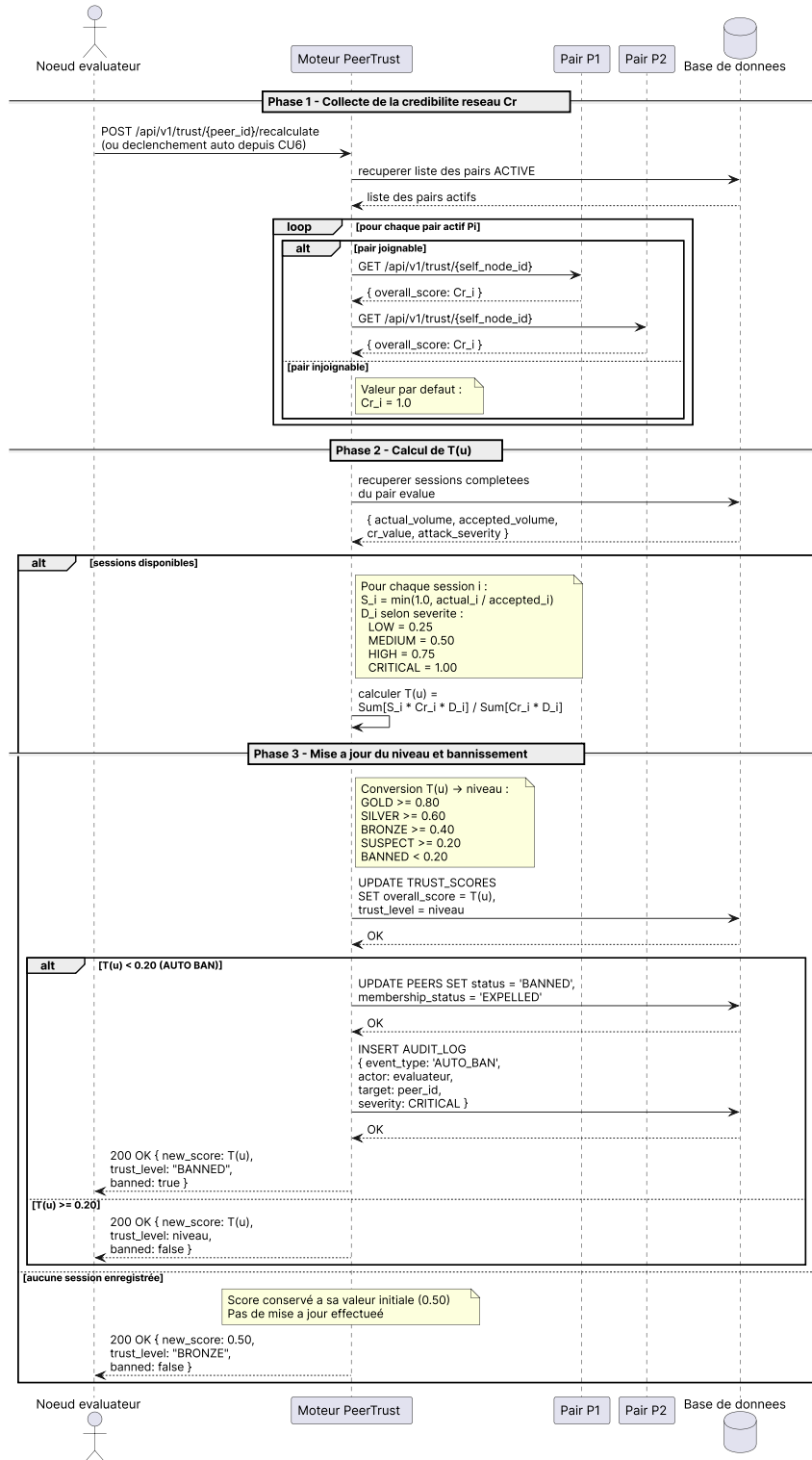


FIGURE 3.14 – DS3 — Diagramme de séquence : Recalcul du score PeerTrust (CU7).

3.3.6 Mécanismes de sécurité

3.3.6.1 Transport sécurisé : mTLS

Toutes les communications entre nœuds transitent sur Transport Layer Security (TLS) 1.3 (RESCORLA, 2018) en mode **mutuel** (Mutual Transport Layer Security (mTLS)) : le serveur présente son certificat au client, mais le client doit également présenter le sien. Ce mécanisme garantit que seuls les nœuds disposant d'un certificat valide (délivré lors de l'enregistrement dans la coalition) peuvent établir une connexion.

Chaque nœud stocke sa clé privée RSA (certs/node.key) et son certificat auto-signé (certs/node.crt).

L'empreinte du certificat de chaque pair est stockée dans le champ `certificate_fingerprint` de la table PEERS pour vérification lors de la connexion.

3.3.6.2 Authentification applicative : JWT RS256

Au-dessus de mTLS, chaque requête API porte un JSON Web Token (JWT) (JONES et al., 2015) signé en RS256 (JONES, 2015). L'architecture asymétrique choisie présente des avantages importants par rapport à un HMAC partagé. Elle garantit d'abord l'**absence de secret partagé** : chaque nœud signe avec sa propre clé privée, les autres vérifiant avec sa clé publique (stockée dans `PEERS.public_key`), de sorte que la compromission d'un nœud n'expose pas les autres. Elle assure également la **non-répudiation** : la signature RS256 lie le token à l'identité du nœud émetteur. Enfin, les tokens ont une **durée de vie limitée** de 15 minutes, ce qui limite la fenêtre d'exploitation en cas d'interception.

Le *payload* du JWT contient : `iss` (identifiant du nœud émetteur), `node_id` et `exp` (timestamp d'expiration). Le middleware d'authentification valide séquentiellement : décodage sans vérification pour extraire `iss`, récupération de la clé publique correspondante, puis vérification cryptographique de la signature.

3.3.7 Interface API REST

3.3.7.1 Style architectural

Le style architectural retenu pour l'interface de communication inter-nœuds est **REST** (*Representational State Transfer*), introduit par Fielding (FIELDING, 2000). Ce choix repose sur trois propriétés fondamentales qui correspondent aux contraintes du système :

Son caractère **sans état** (*stateless*) d'abord : chaque requête HTTP porte toutes les informations nécessaires à son traitement (identité du nœud dans le JWT, paramètres dans le corps), sans qu'aucun nœud serveur ne maintienne de session active côté client, ce qui est cohérent avec la topologie P2P décentralisée où aucun nœud n'a de rôle privilégié. Son **interface uniforme** ensuite : les ressources métier (pairs, attaques, sessions d'aide, scores de confiance) sont identifiées par des URI stables et manipulées via les verbes HTTP sémantiques (POST pour créer, PUT pour mettre à jour, GET pour consulter), ce qui rend l'API intuitive et interopérable entre nœuds hétérogènes. Son **interopérabilité** enfin : le transport HTTP/JSON est supporté nativement par tous les langages et environnements, facilitant l'intégration future de nœuds développés sur des plateformes différentes.

Les alternatives GraphQL (sur-ingénierie pour des requêtes fixes), gRPC (couplage fort via Protocol Buffers) et WebSocket (connexion persistante inadaptée à des échanges ponctuels) ont été écartées car leurs avantages ne compensent pas la complexité qu’elles introduiraient dans un contexte P2P à faible coordination.

3.3.7.2 Endpoints

Chaque nœud expose une API REST HTTPS documentée via OpenAPI 3.0. Le Tableau 3.16 présente les endpoints principaux, regroupés par domaine fonctionnel.

TABLE 3.16 – Endpoints principaux de l’API REST.

Méthode	Endpoint	Description
<i>Découverte et membership</i>		
POST	/peers/register	Enregistrement d’un nouveau pair dans la coalition
GET	/peers	Liste des pairs connus avec leur statut
POST	/heartbeat	Signal de présence d’un pair
POST	/peers/goodbye	Retrait propre d’un pair de la coalition
<i>Mitigation collaborative</i>		
POST	/alert	Signalement d’une nouvelle attaque détectée
POST	/help/request	Demande d’aide à un pair pour une attaque
PUT	/help/{id}/accept	Acceptation d’une session d’aide
PUT	/help/{id}/reject	Refus d’une session d’aide
POST	/traffic/redirect	Activation de la redirection du flux
POST	/attack/over	Clôture de l’attaque et mise à jour des crédits
<i>Confiance et sélection</i>		
GET	/trust/{peer_id}	Score de confiance et ledger d’un pair
POST	/trust/{peer_id}/recalculate	Recalcul forcé du score de confiance
POST	/trust/select-peers	Sélection des pairs via WSM
<i>Sécurité</i>		
POST	/auth/token	Obtention d’un JWT signé RS256

Toutes les routes (sauf /auth/token et /peers/register) requièrent un JWT valide dans l’en-tête Authorization. Les réponses suivent la convention HTTP standard : 200/201 pour les succès, 400 pour les entrées invalides, 401/403 pour les accès non autorisés, 404 pour les ressources absentes.

3.4 Conclusion

Ce chapitre a présenté l’architecture complète du système. La coalition adopte une topologie P2P plate, chaque nœud gérant localement son registre de pairs, ses scores de

confiance et ses politiques de participation. Le modèle PeerTrust adapté introduit une crédibilité historique par session, rendant le calcul de $T(u)$ plus fidèle à la réalité temporelle du réseau. L'algorithme WSM avec pondération AHP offre une sélection multicritère équilibrée entre capacité, latence, confiance et réciprocité. Enfin, la double sécurisation mTLS + JWT RS256 garantit l'authenticité de chaque communication sans secret partagé.

Le chapitre suivant détaille l'implémentation concrète de ces mécanismes et présente les résultats des tests de scalabilité.

Chapitre 4

Implémentation et Évaluation

4.1 Introduction

Ce chapitre présente la mise en œuvre du système. Il décrit d’abord l’environnement technique retenu, puis les principaux mécanismes développés, à savoir le gestionnaire de confiance, la sélection des pairs et les mécanismes de sécurité. Les échanges JSON assurant la communication entre les nœuds sont ensuite présentés. Enfin, les résultats des tests et du scénario de mitigation collaborative sont exposés.

4.2 Environnement technique

4.2.1 Langages de programmation

JavaScript : est un langage de programmation interprété, orienté événement et non bloquant. Il constitue le langage principal du back-end via Node.js (OPENJS FOUNDATION, 2024b), dont le modèle asynchrone est particulièrement adapté aux échanges inter-nœuds concurrents (heartbeats, requêtes d’aide simultanées).javascript.png

SQL : est le langage de requête standard des bases de données relationnelles. Il est utilisé via PostgreSQL et l’ORM Sequelize pour la définition des schémas, les migrations et l’interrogation des quatorze tables du modèle de données.sql.png

4.2.2 Outils de développement

Node.js v20 LTS (OpenJS Foundation, 2024b) : est une plate-forme côté serveur construite sur le moteur V8 de Chrome. Son modèle événementiel est idéal pour les applications à I/O concurrentes telles que les coalitions de nœuds échangeant des heartbeats toutes les 30 secondes.

Express.js v4 (OpenJS Foundation, 2024a) : est un framework REST minimaliste pour Node.js, de type *middleware-first*. Il structure les routes de l’API (/peers, /help, /trust, /auth) et facilite l’intégration des middlewares mTLS et JWT.

PostgreSQL 15 (The PostgreSQL Global Development Group, 2024) : est un système de gestion de bases de données relationnelles open-source reconnu pour sa conformité ACID. Il est retenu pour son support natif des types UUID, ENUM, FLOAT et la gestion des contraintes de clés étrangères avec politique de suppression en cascade.

Sequelize v6 (Sequelize contributors, 2024) : est un ORM Node.js qui fournit une abstraction SQL avec migrations, associations déclaratives et transactions. Il mappe les quatorze classes du modèle objet vers les tables PostgreSQL.

jsonwebtoken (Auth0, 2024) (JWT) : est la bibliothèque Node.js d'implémentation du standard JWT (RFC 7519 (JONES et al., 2015)). Elle est utilisée pour générer et vérifier les tokens RS256 à durée de vie de 15 minutes qui sécurisent chaque appel API inter-nœuds.

Swagger / OpenAPI 3 (OpenAPI Initiative, 2020) : est un standard de description d'API REST. Il auto-documente l'ensemble des endpoints via des annotations JSDoc et expose une interface interactive accessible à `/api-docs`.

4.2.3 Environnement de déploiement

Docker & Docker Compose (Docker, Inc., 2024) : permettent la containerisation du système. Chaque nœud de la coalition s'exécute dans un conteneur isolé ; Docker Compose orchestre les quatre nœuds et la base de données PostgreSQL avec un réseau virtuel dédié, garantissant un déploiement reproductible sur toute machine disposant de Docker.

4.2.4 Architecture de déploiement

La Figure 4.1 présente l'architecture de déploiement du prototype. Le système est composé de cinq conteneurs Docker orchestrés par Docker Compose : quatre nœuds API (`shieldnet-university`, `shieldnet-pme`, `shieldnet-isp`, `shieldnet-datacenter`), chacun exposant son API sur un port distinct (3001 à 3004) en interne sur le port 8443, et une instance PostgreSQL partagée hébergeant quatre bases de données indépendantes. Les nœuds communiquent entre eux via un réseau coalition sécurisé en HTTPS, authentifié par JWT RS256, selon trois flux principaux : le heartbeat périodique (toutes les 30 secondes), les requêtes d'aide lors d'une attaque, et la collecte de crédibilité réseau pour le calcul du score de confiance. Le mécanisme mTLS est implémenté côté serveur (section 4.5) mais désactivé par défaut dans ce prototype : il nécessiterait une autorité de certification commune aux quatre organisations simulées, ce qui réintroduirait une forme de centralisation que l'architecture P2P cherche précisément à éviter ; l'authentification mutuelle des nœuds est néanmoins déjà garantie au niveau applicatif par JWT RS256, chaque nœud signant avec une clé privée qui lui est propre.

figures/deployment_diagram1.pdf

FIGURE 4.1 – Diagramme de déploiement du prototype.

4.2.5 Outils de collaboration

Git (Torvalds et al., 2024) : est le système de contrôle de version distribué utilisé pour gérer l’historique du code source, les branches de fonctionnalités et les correctifs tout au long du développement du prototype.

GitHub (GitHub, Inc., 2024) : est la plateforme de collaboration hébergeant le dépôt du projet. Elle permet le suivi des issues, la revue de code par Pull Requests et la sauvegarde centralisée du code source.

4.2.6 Structure du projet

Le projet est organisé selon le schéma de la Figure 4.2, séparant la logique métier (utils/), les routes API (routes/), les modèles de données (models/) et la configuration (config/).

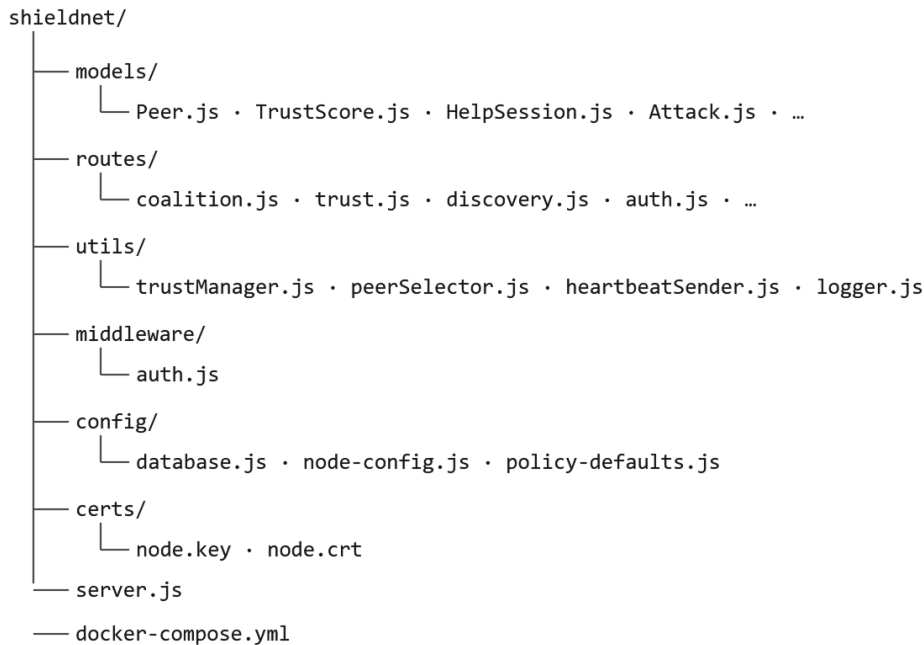


FIGURE 4.2 – Structure des répertoires du projet.

4.3 Implémentation du gestionnaire de confiance

4.3.1 Calcul de $T(u)$ avec Cr historique

Le module `utils/trustManager.js` implémente la formule `PeerTrust` (Équation 3.1). La fonction `computeTrustScore(peer_id)` charge toutes les sessions complétées du pair et calcule la moyenne pondérée :

```

1  async function computeTrustScore(peer_id) {
2    const sessions = await HelpSession.findAll({
3      where: { helping_peer_id: peer_id, status: "COMPLETED" },
4      include: [{ model: Attack, as: "attack",
5                  attributes: ["severity"] }],
6    });
7
8    if (sessions.length === 0)
9      return { score: DEFAULT_TRUST, level: "BRONZE", session_count: 0 };
10
11    let numerator = 0, denominator = 0;
12
13    for (const session of sessions) {
14      const accepted = Number(session.accepted_volume_gbps ?? 0);
15      const actual    = Number(session.actual_volume_gbps   ?? 0);

```

```

16
17 // S(u,i) = min(1, V_reel / V_accepte)
18 const S = accepted > 0 ? Math.min(1, actual / accepted) : 0;
19
20 // D(u,i) : facteur contextuel selon la severite
21 const severity = session.attack?.severity ?? "LOW";
22 const D = SEVERITY_WEIGHTS[severity] ?? 0.25;
23
24 // Cr_i : credibilite capturee a la cloture de la session
25 const Cr_i = session.cr_value ?? DEFAULT_TRUST;
26
27 numerator += S * Cr_i * D;
28 denominator += Cr_i * D;
29 }
30
31 const score = denominator > 0 ? numerator / denominator :
32   DEFAULT_TRUST;
33 return { score, level: scoreToLevel(score),
34   session_count: sessions.length };

```

Listing 4.1 – Calcul de $T(u)$ avec crédibilité historique par session.

La valeur `cr_value` est persistée dans `HELP_SESSIONS` à la clôture de chaque attaque (route POST `/attack/over`), comme décrit à la section 4.3.2.

4.3.2 Capture de la crédibilité à la clôture

La fonction `fetchCrFromNetwork(localNodeId, excludePeerId)` interroge tous les pairs actifs (sauf le pair évalué) pour obtenir leur score de confiance courant du nœud local, puis retourne la moyenne :

```

1 // Dans POST /attack/over -- routes/coalition.js
2 const localNodeId = await getLocalNodeId();
3 const crCache = {}; // evite les appels en double par pair
4
5 for (const session of sessions) {
6   // ... mise a jour actual_volume_gbps ...
7
8   if (localNodeId) {
9     const pid = session.helping_peer_id;
10    if (crCache[pid] === undefined)
11      crCache[pid] = await fetchCrFromNetwork(localNodeId, pid);
12
13    await session.update({ cr_value: crCache[pid] });
14  }
15
16   // ... mise a jour du ledger de reciprocite ...
17 }

```

Listing 4.2 – Capture de Cr et persistance à la clôture de session.

Le cache `crCache` évite d'interroger le réseau plusieurs fois pour un même pair si plusieurs sessions concernent ce pair dans la même attaque. Le résultat est stocké dans `HELP_SESSIONS.cr_value` (FLOAT, valeur par défaut 0.5 pour les sessions antérieures à cette version).

4.3.3 Persistance et sanctions automatiques

La fonction `recalculateAndSave(peer_id)` recalcule le score, le persiste dans `TRUST_SCORES` et applique automatiquement la sanction si le score tombe sous 0.20. Le champ `trust_level` de la table `TRUST_SCORES` est mis à jour ; si le niveau devient `BANNED`, le pair est marqué `status = BANNED` et `membership_status = EXPELLED` dans la table `PEERS`. Tout changement de niveau déclenche par ailleurs une entrée dans `AUDIT_LOG` (type `TRUST_LEVEL_CHANGED`), et un bannissement automatique génère spécifiquement une entrée de sévérité `CRITICAL` (`PEER_BANNED`).

4.4 Implémentation de la sélection des pairs

4.4.1 Calcul du score WSM

Le module `utils/peerSelector.js` implémente l'Équation 3.2. La fonction `selectPeers()` charge tous les pairs `ACTIVE` avec capacité disponible, normalise les valeurs brutes, puis calcule le score composite de chaque candidat :

```

1  const W = { C: 0.52, L: 0.20, T: 0.20, R: 0.08 };
2  const EPSILON = 1e-6;
3
4  // Normalisation des valeurs brutes
5  const maxCap = Math.max(...capValues);
6  const minLat = Math.min(...latValues);
7  const latRange = Math.max(...latValues) - minLat;
8
9  const scored = candidates.map((peer, i) => {
10     const Cp = maxCap > 0 ? capValues[i] / maxCap :
11       0;
12     const Lp_inv = latRange > 0 ? 1 - (latValues[i]-minLat)/latRange :
13       1;
14     const Tp = peer.trust_score?.overall_score ?? 0.5;
15
16     const received = Number(peer.reciprocity_ledger?.credits_received ??
17       0);
18     const given = Number(peer.reciprocity_ledger?.credits_given ??
19       0);
20     const Rp = received / (received + given + EPSILON);
21
22     const score = W.C*Cp + W.L*Lp_inv + W.T*Tp + W.R*Rp;
23     return { peer, score, criteria: { Cp, Lp_inv, Tp, Rp } };
24 });

```

Listing 4.3 – Calcul du score WSM et normalisation des critères.

4.4.2 Plan de répartition proportionnelle

Le plan de répartition est construit en normalisant les scores WSM ; l'allocation en pourcentage est la partie entière de $w_i \times 100$:

```

1  const totalScore = scored.reduce((sum, c) => sum + c.score, 0);
2

```

```

3  const plan = scored.map((c) => {
4    // w_i = Score(p_i) / sum Score(p_j)
5    const weight = totalScore > 0
6                  ? c.score / totalScore
7                  : 1 / scored.length;
8
9    // allocation_pct = min(floor(w_i * 100), cap_disp)
10   const wsm_pct    = Math.floor(weight * 100);
11   const allocation_pct = Math.min(wsm_pct, c.cap_disp);
12
13   return { peer: { ... }, wsm_score, weight, allocation_pct };
14 });

```

Listing 4.4 – Construction du plan de répartition proportionnelle.

Le quota de paquets alloué à chaque pair par cycle de Round-Robin pondéré est la partie entière du poids w_i exprimé en pourcentage, plafonné par la capacité disponible du pair :

$$\text{alloc}_i = \min(\lfloor w_i \times 100 \rfloor, \text{cap_disp}_i) \quad (4.1)$$

Cette contrainte assure qu'un pair ne peut recevoir plus de paquets par cycle que sa capacité déclarée ne le permet, indépendamment de la qualité de son score WSM. La Figure 3.10 synthétise l'ensemble du plan de répartition : depuis le trafic d'attaque entrant jusqu'à l'allocation proportionnelle par pair, en passant par le calcul des poids w_i et les contraintes associées.

4.4.3 Distribution des paquets par Round-Robin pondéré

Une fois les allocations alloc_i calculées pour chaque pair N_{p_i} , la redistribution effective des paquets est réalisée selon un mécanisme de **Round-Robin pondéré** (Weighted Round-Robin, WRR). Le nœud victime N_v dispose d'un flux entrant de n paquets $(P_1, P_2, \dots, P_n)$ qu'il distribue vers m nœuds pairs $(N_{p_1}, N_{p_2}, \dots, N_{p_m})$ acceptés, chacun associé à un quota alloc_i représentant le nombre de paquets qu'il absorbe par cycle.

Principe d'un cycle. Les pairs sont parcourus séquentiellement dans l'ordre $N_{p_1}, N_{p_2}, \dots, N_{p_m}$. À chaque passage, le pair N_{p_i} reçoit exactement alloc_i paquets consécutifs. Le pair N_{p_i} reçoit ainsi les paquets d'indice :

$$P_{1+\sum_{j=1}^{i-1} \text{alloc}_j}, \quad P_{2+\sum_{j=1}^{i-1} \text{alloc}_j}, \quad \dots, \quad P_{\sum_{j=1}^i \text{alloc}_j} \quad (4.2)$$

Après N_{p_m} , le processus revient à N_{p_1} et entame un nouveau cycle. Cette opération est répétée jusqu'à ce que le dernier paquet P_n soit distribué. La Figure 4.3 illustre ce mécanisme.

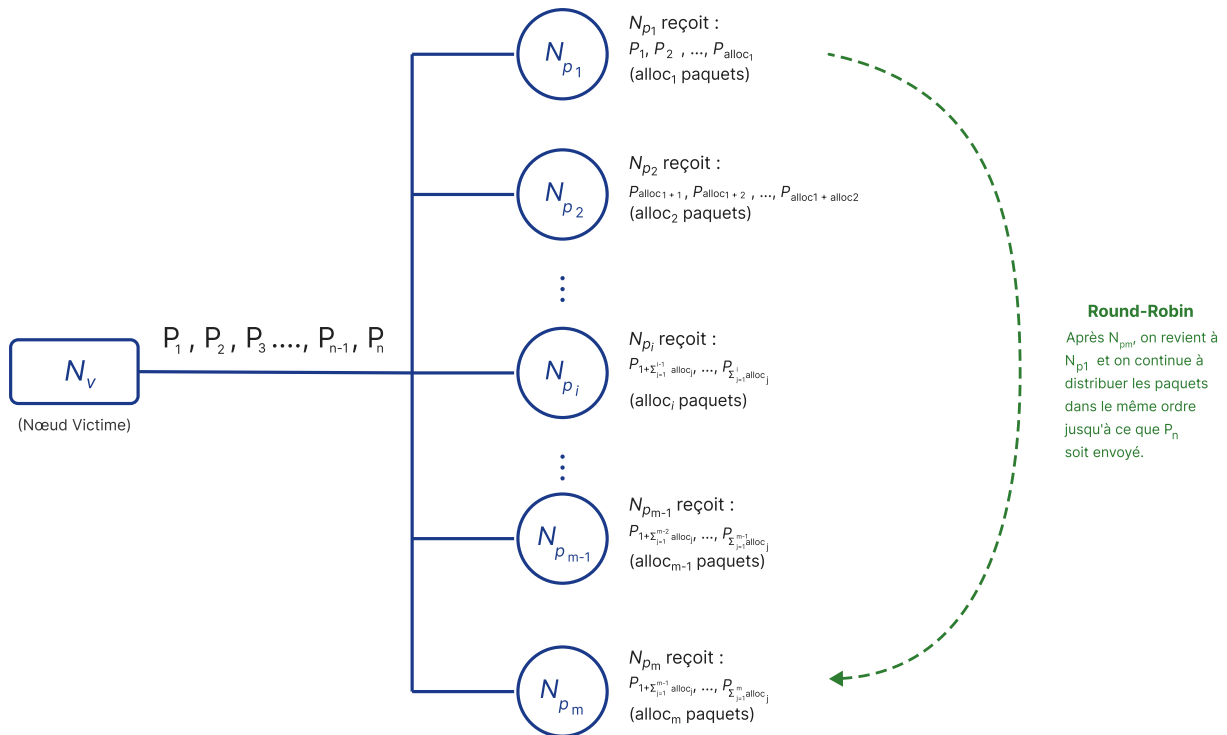


FIGURE 4.3 – Distribution des paquets par Round-Robin pondéré : le nœud victime N_v distribue n paquets vers m pairs en cycles successifs, chaque pair N_{p_i} recevant $alloc_i$ paquets par cycle.

4.5 Mécanismes de sécurité

4.5.1 JWT RS256 : génération et vérification

Chaque nœud génère ses tokens avec sa clé privée RSA ; la vérification utilise la clé publique de l'émetteur, stockée dans `PEERS.public_key` :

```

1 // Generation (noeud emetteur)
2 function generateToken(node_id) {
3   const privateKey = fs.readFileSync(process.env.TLS_KEY);
4   return jwt.sign(
5     { iss: node_id, node_id },
6     privateKey,
7     { algorithm: "RS256", expiresIn: "15m" }
8   );
9 }
10
11 // Verification (noeud recepteur) -- extrait du middleware
12 const decoded = jwt.decode(token); // lecture iss sans verif
13 const peer = await Peer.findOne({ where: { peer_id: decoded.iss } });
14 const publicKey = peer.public_key; // PEM stocke en base
15 jwt.verify(token, publicKey, { algorithms: ["RS256"] });

```

Listing 4.5 – Génération et vérification JWT RS256.

Le token expire après 15 minutes ; un token expiré retourne HTTP 401 avec un message invitant à obtenir un nouveau token via POST /auth/token.

4.5.2 mTLS : configuration côté serveur

Le serveur HTTPS est créé avec les options TLS suivantes :

```

1  const server = https.createServer({
2    key: fs.readFileSync(process.env.TLS_KEY),
3    cert: fs.readFileSync(process.env.TLS_CERT),
4    ca: fs.readFileSync(process.env.TLS_CA),
5    requestCert: true, // demander le certificat client
6    rejectUnauthorized: true, // refuser tout cert non signe par la CA
   coalition
7  }, app);

```

Listing 4.6 – Configuration mTLS du serveur HTTPS.

Lorsque `MTLS_ENABLED=true`, `rejectUnauthorized` est toujours positionné à `true` : toute connexion dont le certificat client n'est pas signé par la CA de la coalition est rejetée au niveau TLS avant même d'atteindre les middlewares applicatifs. La variable d'environnement `MTLS_STRICT` intervient quant à elle au niveau du middleware applicatif : si elle vaut `true`, une requête sans certificat (certificat absent plutôt qu'invalidé) est également rejetée avec HTTP 401. Dans le prototype déployé, `MTLS_ENABLED=false` : ce mécanisme est implémenté et testable indépendamment (section 4.7), mais n'est pas activé par défaut, pour les raisons exposées en section 4.2.

4.6 Protocole de communication inter-nœuds

Les nœuds communiquent exclusivement via des requêtes HTTPS authentifiées par JWT RS256. Cette section présente les messages JSON échangés lors des principales interactions du protocole.

4.6.1 Authentification

Avant tout appel API protégé, un nœud obtient un token JWT en transmettant son identifiant et son secret local.

```

1  // Requête
2  {
3    "node_id": "node-university",
4    "node_secret": "secret-key-2025"
5  }
6
7  // Réponse 200 OK
8  {
9    "token": "eyJhbGciOiJSUzI1NiJ9...",
10   "expires_in": "15m",
11   "node_id": "node-university",
12   "role": "local",
13   "algorithm": "RS256"

```

14 }
}

Listing 4.7 – Requête et réponse — POST /auth/token.

Le token expire après 15 minutes. Le nœud récepteur vérifie la signature RS256 à l'aide de la clé publique de l'émetteur ; un token expiré ou altéré retourne HTTP 401. Le nœud récepteur vérifie la signature RS256 à l'aide de la clé publique de l'émetteur, enregistrée dans la table `Peers`.

4.6.2 Heartbeat

Chaque nœud envoie périodiquement un heartbeat à ses pairs pour signaler son état et sa charge courante.

```

1 // Requête
2 {
3   "peer_id":           "550e8400-e29b-41d4-a716-446655440000",
4   "reported_status":   "ACTIVE",
5   "reported_load_pct": 42.5,
6   "reported_available_gbps": 5.75,
7   "round_trip_time_ms": 12
8 }
9
10 // Réponse 201 Created
11 {
12   "message": "Heartbeat recorded",
13   "heartbeat": {
14     "peer_id":           "550e8400-...",
15     "reported_status":   "ACTIVE",
16     "reported_load_pct": 42.5,
17     "reported_available_gbps": 5.75,
18     "round_trip_time_ms": 12
19   }
20 }
```

Listing 4.8 – Requête et réponse — POST /heartbeat.

À la réception, le nœud met à jour le champ `last_heartbeat` du pair et remet à zéro son compteur `consecutive_missed_heartbeats`.

4.6.3 Détection d'une attaque

Lorsqu'une attaque DDoS est détectée, le nœud victime l'enregistre localement via POST /alert.

```

1 // Requête
2 {
3   "target_ip_range": "192.168.1.0/24",
4   "target_service":  "HTTP",
5   "target_port":     80,
6   "target_protocol": "TCP",
7   "status":          "DETECTED",
8   "coalition_helped": false
9 }
```

```

10
11 // Réponse 201 Created
12 {
13     "attack_id":          "a1b2c3d4-...",
14     "status":             "DETECTED",
15     "detected_at":        "2026-05-22T10:00:00.000Z",
16     "target_ip_range":    "192.168.1.0/24",
17     "coalition_helped":   false
18 }

```

Listing 4.9 – Requête et réponse — POST /alert.

La sévérité est initialisée à LOW et recalculée à la clôture de l’attaque en fonction du volume total filtré par la coalition.

4.6.4 Demande et réponse d’aide

Le nœud victime envoie une demande d’aide à chaque pair sélectionné par l’algorithme WSM.

```

1 // Requête
2 {
3     "attack_id":          "a1b2c3d4-...",
4     "helping_peer_id":    "550e8400-...",
5     "requesting_node_id": "node-university",
6     "allocation_pct":     60,
7     "tunnel_type":        "GRE"
8 }
9
10 // Réponse 201 Created
11 {
12     "session_id":         "f9e8d7c6-...",
13     "attack_id":          "a1b2c3d4-...",
14     "helping_peer_id":    "550e8400-...",
15     "requesting_node_id": "node-university",
16     "status":             "REQUESTED",
17     "allocation_pct":     60,
18     "direction":          "OUTBOUND_REQUEST",
19     "requested_at":        "2026-05-22T10:00:05.000Z"
20 }

```

Listing 4.10 – Requête et réponse — POST /help/request.

Le pair accepte ou rejette la session après vérification du score de confiance et de sa capacité disponible.

```

1 // Acceptation - corps de la requête
2 {
3     "accepted_volume_gbps": 8.5,
4     "tunnel_type":          "GRE",
5     "response_time_ms":     230
6 }
7 // Réponse 200 OK
8 {
9     "session_id":          "f9e8d7c6-...",
10    "status":               "ACCEPTED",
11    "accepted_volume_gbps": 8.5,

```

```

12  "tunnel_type":      "GRE",
13  "responded_at":     "2026-05-22T10:00:06.000Z"
14 }
15
16 // Rejet - corps de la requête
17 { "rejection_reason": "Capacité insuffisante" }
18 // Réponse 200 OK
19 {
20  "session_id":      "f9e8d7c6-...",
21  "status":          "REJECTED",
22  "rejection_reason": "Capacité insuffisante",
23  "responded_at":    "2026-05-22T10:00:06.000Z"
24 }

```

Listing 4.11 – Acceptation (PUT /help/{id}/accept) et rejet (PUT /help/{id}/reject).

4.6.5 Redirection du trafic et clôture

Une fois la session acceptée, le nœud victime déclenche la redirection du trafic vers le pair via un tunnel réseau.

```

1  // Requête
2  {
3    "session_id":  "f9e8d7c6-...",
4    "tunnel_type": "GRE",
5    "volume_gbps": 8.5
6  }
7
8  // Réponse 200 OK
9  {
10   "session_id":      "f9e8d7c6-...",
11   "status":          "ACTIVE",
12   "tunnel_type":     "GRE",
13   "actual_volume_gbps": 8.5,
14   "activated_at":    "2026-05-22T10:00:07.000Z"
15 }

```

Listing 4.12 – Requête et réponse — POST /traffic/redirect.

À la fin de l'attaque, le nœud victime clôture toutes les sessions et déclenche la mise à jour des crédits de réciprocité.

```

1  // Requête
2  {
3    "attack_id":      "a1b2c3d4-...",
4    "session_ids":     ["f9e8d7c6-..."],
5    "attack_duration_seconds": 300
6  }
7
8  // Réponse 200 OK
9  {
10   "attack_id":      "a1b2c3d4-...",
11   "status":         "ENDED",
12   "severity":       "HIGH",
13   "ended_at":       "2026-05-22T10:05:00.000Z",

```

```

14  "duration_seconds": 300,
15  "coalition_helped": true,
16  "nb_peers_involved": 1
17  }

```

Listing 4.13 – Requête et réponse — POST /attack/over.

La sévérité finale est calculée selon l'échelle Stormwall : **CRITICAL** (≥ 100 Gbps filtrés), **HIGH** (≥ 10 Gbps), **MEDIUM** (≥ 1 Gbps), **LOW** (en dessous).

4.7 Tests et évaluation

4.7.1 Protocole de test

Les tests ont été conduits selon trois axes :

1. **Tests fonctionnels** (`tests/functional_test.py`) : vérification de chaque endpoint de l'API (enregistrement, heartbeat, sessions d'aide, clôture, recalcul de trust) sur une coalition simulée. Les résultats de bout en bout de ce protocole fonctionnel sont présentés à travers la démonstration sur le tableau de bord (section 4.7.6), qui exécute exactement le même enchaînement de requêtes sur le système réellement déployé.
2. **Tests de sécurité** (`tests/security_test.py`) : tentatives d'accès sans JWT, avec un token expiré, avec un token signé par une clé incorrecte, avec un nœud banni, et avec un `peer_id` inconnu dans le corps de la requête.
3. **Tests de scalabilité** (`tests/scalability_test.py`) : montée en charge progressive de 20 à 100 nœuds simultanés ; mesure des temps de réponse pour l'enregistrement, le heartbeat et la sélection WSM.

Les tests de scalabilité ont été conduits sur une machine locale avec Node.js 20 LTS et PostgreSQL 15, en simulant les nœuds par des processus Python concurrents via `asyncio` (PYTHON SOFTWARE FOUNDATION, 2024) et `aiohttp` (AIOHTTP CONTRIBUTORS, 2024). Chaque palier (20, 40, 60, 80, 100 nœuds) a été mesuré sur une seule exécution ; les résultats doivent donc être interprétés comme indicatifs plutôt que comme une moyenne statistiquement lissée (section 4.7.4).

4.7.2 Validation du calcul PeerTrust

Trois scénarios illustrent le comportement de la formule $T(u) = \sum[S \cdot Cr \cdot D] / \sum[Cr \cdot D]$ (Équation 3.1) face à des historiques de session contrastés. Les deux premiers scénarios (satisfaction extrême) correspondent aux assertions automatisées de `functional_test.py` (Phase 8) ; le troisième illustre un cas nominal avec plusieurs sessions de sévérité différente.

TABLE 4.1 – Scénarios de validation du calcul de $T(u)$.

Scénario	Sessions (S , Cr , D)	$T(u)$ calculé	Niveau
1 — Livraison parfaite	1 session : $S=1,0$, $Cr=1,0$, $D=1,0$ (CRITICAL)	$\frac{1,0 \times 1,0 \times 1,0}{1,0 \times 1,0} = 1,000$	GOLD
2 — Livraison nulle	1 session : $S=0,0$, $Cr=1,0$, $D=1,0$ (CRITICAL)	$\frac{0,0}{1,0 \times 1,0} = 0,000$	BANNED
3 — Cas nominal mixte	2 sessions : $S_1=0,8$, $Cr_1=0,9$, $D_1=0,75$ (HIGH) ; $S_2=0,5$, $Cr_2=0,6$, $D_2=0,25$ (LOW)	$\frac{0,54 + 0,075}{0,675 + 0,15} = 0,745$	SILVER

Le scénario 1 confirme qu’une satisfaction parfaite ($S=1$) associée à une crédibilité maximale produit $T(u)=1,000$, classé GOLD ($\geq 0,80$). Le scénario 2 confirme le mécanisme de bannissement automatique : une livraison nulle produit $T(u)=0,000 < 0,20$, déclenchant le passage du pair en statut BANNED et `membership_status = EXPELLED` (section 4.3). Le scénario 3 illustre la pondération contextuelle : bien que la seconde session ait une satisfaction modérée ($S_2=0,5$), son poids ($Cr_2 \cdot D_2 = 0,15$) est près de cinq fois plus faible que celui de la première session ($Cr_1 \cdot D_1 = 0,675$, une attaque HIGH évaluée par un pair très crédible) : la session la plus sévère et la mieux attestée domine le score final, qui reste élevé (0,745, niveau SILVER) malgré la session mitigée.

4.7.3 Résultats des tests de scalabilité

Le Tableau 4.2 présente les résultats collectés ; la Figure 4.4 en offre une représentation graphique.

TABLE 4.2 – Résultats des tests de scalabilité (20 à 100 nœuds).

Nœuds	Enregistrement			Heartbeat			Sélection WSM	
	OK	Moy (ms)	Max (ms)	OK	Moy (ms)	Max (ms)	Durée (ms)	Pairs
20	20	389	420	20	160	178	32.2	20
40	40	313	382	40	220	247	22.5	40
60	60	426	511	60	280	307	20.8	60
80	80	449	565	80	450	490	28.6	80
100	100	457	550	100	407	434	30.4	100

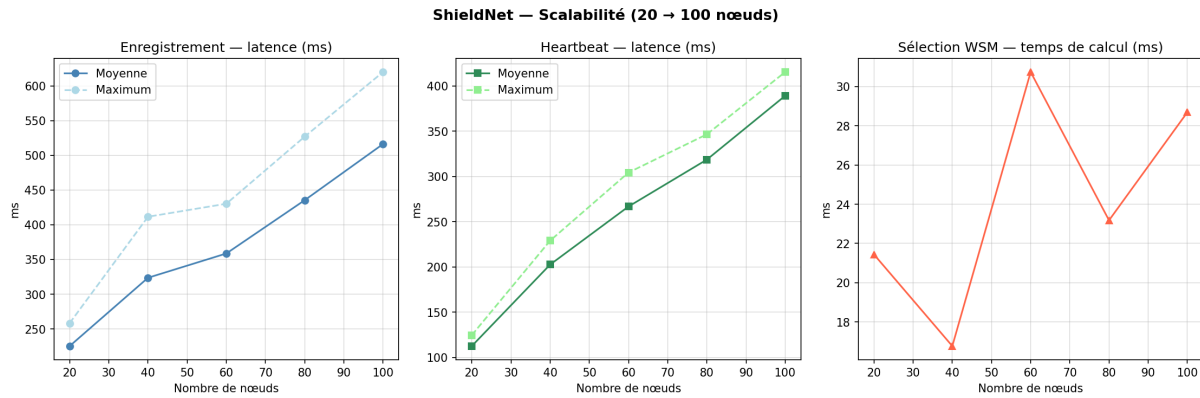


FIGURE 4.4 – Temps de réponse en fonction du nombre de nœuds simulés (enregistrement, heartbeat et sélection WSM).

4.7.4 Analyse des résultats

Trois observations ressortent du Tableau 4.2.

Taux de succès de 100 %. Tous les enregistrements et tous les heartbeats ont réussi sur l’ensemble des paliers testés (20 à 100 nœuds). Aucune erreur d’authentification, de collision d’identifiant UUID ou de contrainte de base de données n’a été observée.

Sélection WSM bornée sous 33 ms, sans tendance strictement croissante. Le temps de sélection WSM reste compris entre 20,8 et 32,2 ms sur l’ensemble des paliers, sans suivre une progression monotone (il descend de 32,2 ms à 20,8 ms entre 20 et 60 nœuds avant de remonter à 30,4 ms à 100 nœuds). Cette absence de croissance régulière s’explique par le fait que la sélection repose sur une unique requête SQL avec jointures vers PostgreSQL : sa durée dépend davantage de l’état du cache et du plan d’exécution de la base au moment de la mesure que du nombre de pairs lui-même, chaque palier n’ayant été mesuré qu’une seule fois (section 4.7). Le résultat à retenir est néanmoins stable : quel que soit le nombre de nœuds testé, la sélection reste inférieure à 33 ms.

Heartbeat et enregistrement : croissance globale avec variance locale. Le temps d’enregistrement et le temps de heartbeat croissent globalement avec le nombre de nœuds (de 389 à 457 ms, et de 160 à 450 ms respectivement), mais présentent chacun un point de mesure isolé en deçà du palier précédent (enregistrement : 313 ms à 40 nœuds contre 389 ms à 20 ; heartbeat : 407 ms à 100 nœuds contre 450 ms à 80). Ces écarts sont attribuables à la variance d’une machine de développement non isolée (autres processus concurrents, cache du système d’exploitation, montée en régime du moteur V8) plutôt qu’à un phénomène algorithmique, chaque palier ne reposant que sur une seule exécution. Une mesure plus rigoureuse répéterait chaque palier plusieurs fois et reporterait la moyenne avec un intervalle de confiance.

Perspectives de scalabilité. Les résultats indiquent malgré ce bruit un comportement globalement sub-linéaire : doubler le nombre de nœuds de 50 à 100 n’a pas doublé

les temps de réponse. Pour des coalitions de plusieurs centaines de nœuds, l'ajout d'un système de cache distribué (Redis) pour les scores de confiance et l'utilisation de connexions persistantes pour les heartbeats permettraient de maintenir ces performances.

4.7.5 Résultats des tests de sécurité

Le Tableau 4.3 présente les cinq scénarios d'attaque simulés et les réponses obtenues du système.

Scénario testé	HTTP attendu	Résultat
Requête sans en-tête <code>Authorization</code>	401	✓
Token JWT expiré (émis il y a > 15 min)	401	✓
Token JWT signé avec une clé RSA incorrecte	401	✓
Token valide d'un nœud à statut <code>BANNED</code>	403	✓
<code>peer_id</code> inconnu dans le corps de la requête	404	✓

TABLE 4.3 – Tests de validation du mécanisme d'authentification

Les cinq scénarios ont été rejetés avec le code HTTP correct. Aucun accès non autorisé n'a été accordé ; la vérification séquentielle du middleware (décodage sans vérification $\rightarrow$ récupération de la clé publique $\rightarrow$ vérification RS256) a fonctionné dans tous les cas.

4.7.6 Démonstration sur le tableau de bord de supervision

Pour valider l'intégration cohérente de l'ensemble des composants en conditions réalistes, un scénario de bout en bout a été exécuté via le tableau de bord de supervision développé en HTML/JavaScript. Cette interface offre une vision consolidée de l'état de la coalition et permet de déclencher manuellement une simulation d'attaque DDoS tout en observant en temps réel chaque phase du protocole de mitigation collaborative.

État initial de la coalition. La Figure 4.5 présente la vue d'ensemble au moment du lancement de la simulation. La coalition compte 103 pairs enregistrés : 3 nœuds réels (Datacenter Oran, FAI Algérie Télécom, PME Algéroise) et 100 pairs virtuels répartis en cinq niveaux de confiance. Parmi eux, 93 pairs sont actifs, 23 sont de niveau GOLD et 10 sont bannis. Deux attaques précédentes restent ouvertes en base (non clôturées), et 55 sessions d'aide sont encore actives au moment de la capture.

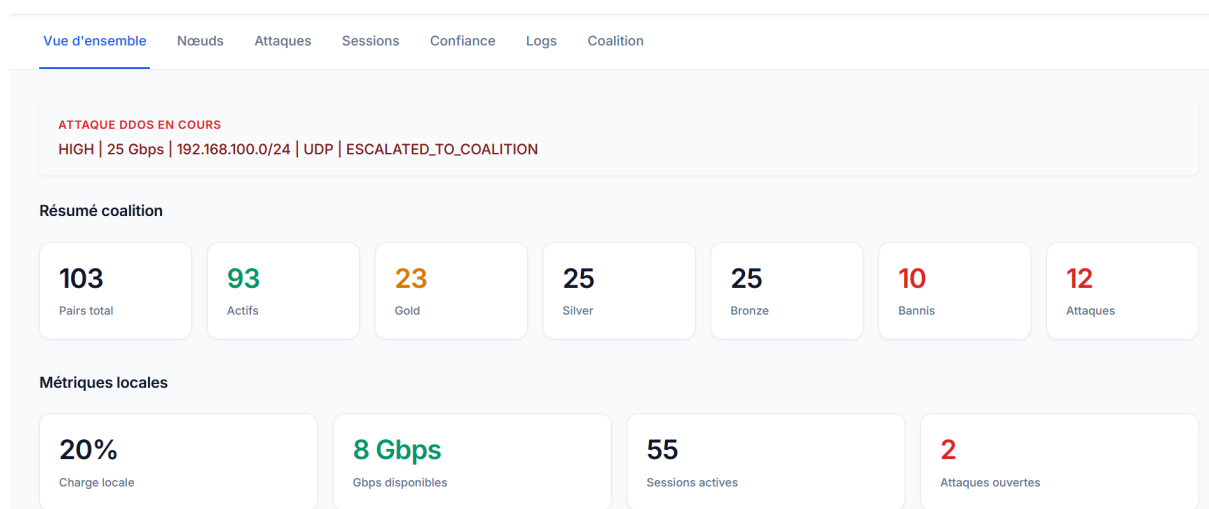


FIGURE 4.5 – Vue d'ensemble du tableau de bord : état de la coalition avant le lancement du scénario.

La Figure 4.6 présente le tableau des pairs triés par score de confiance décroissant. Les trois nœuds réels (node-datacenter, node-isp, node-pme) apparaissent en tête avec le statut ACTIVE et un score PeerTrust de 1.000 (nœuds GOLD sans historique de défaillance), suivis des pairs virtuels dont les scores varient selon leur niveau (GOLD : 0.928–0.929, SILVER : 0.700–0.717).

Tous les pairs — 103 + 100 nœuds Nettoyer

#	Nom	Organisation	Type	Statut	Confiance	Score	Capacité	Charge
1	node-datacenter	Datacenter Oran	DATACENTER	ACTIVE	GOLD	1.000	12.0 Gbps	20%
2	node-isp	Algerie Telecom ISP	ISP	ACTIVE	GOLD	1.000	16.0 Gbps	20%
3	node-pme	PME Algeroise	PME	ACTIVE	GOLD	1.000	4.0 Gbps	20%
4	sim-013	GOVERNMENT Sim-13	GOVERNMENT	ACTIVE	GOLD	0.929	4.0 Gbps	20%
5	sim-005	GOVERNMENT Sim-5	GOVERNMENT	ACTIVE	GOLD	0.929	4.0 Gbps	20%
6	sim-015	NGO Sim-15	NGO	ACTIVE	GOLD	0.928	12.0 Gbps	20%

FIGURE 4.6 – Tableau des pairs de la coalition triés par score PeerTrust décroissant.

Détection et escalade vers la coalition. Une attaque de sévérité HIGH est configurée : volume de 25 Gbps, protocole UDP, port 80, ciblant le réseau 192.168.100.0/24. Avec une capacité locale disponible de 8 Gbps, l'overflow est de 17 Gbps. Le système émet automatiquement 103 alertes à l'ensemble des pairs de la coalition. La Figure 4.7 montre le résultat : 52 pairs ont accepté la demande d'aide et 51 l'ont refusée, illustrant le comportement probabiliste implémenté (taux de refus de 15 % pour GOLD, 25 % pour SILVER, 45 % pour BRONZE, 75 % pour SUSPECT).

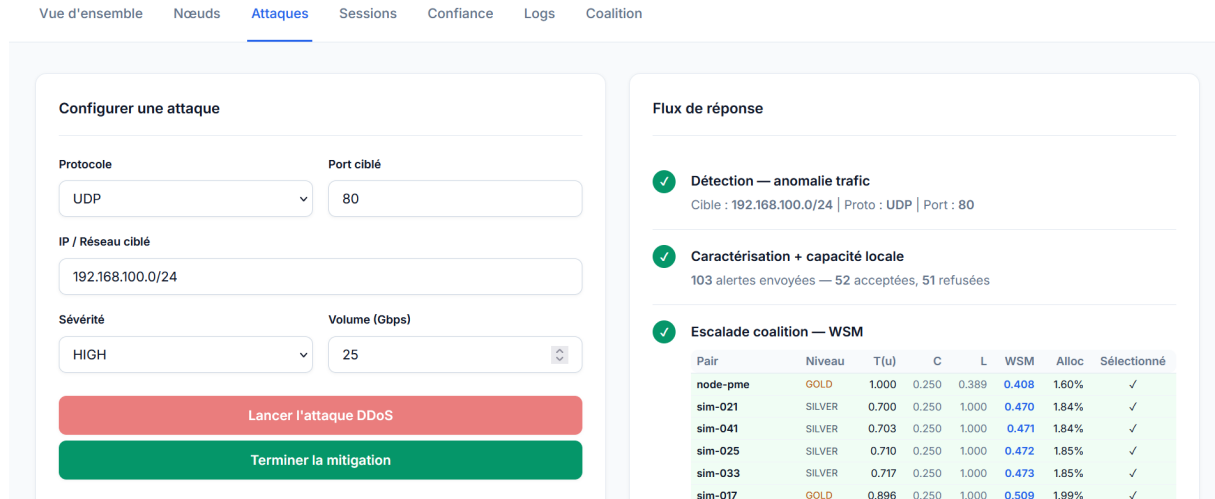


FIGURE 4.7 – Configuration de l'attaque simulée et début du tableau de sélection WSM.

Sélection WSM et allocation. L'algorithme WSM calcule un score composite pour chaque pair éligible (score de confiance $\geq 0,70$) selon la formule $WSM(p) = 0,52 \cdot C + 0,20 \cdot L + 0,20 \cdot T + 0,08 \cdot R$. La Figure 4.8 présente le classement complet des 38 pairs scorés : les paires avec une latence normalisée $L = 1,000$ et une forte capacité obtiennent les scores les plus élevés (jusqu'à 0.905 pour sim-008) et se voient allouer les plus grandes fractions du trafic d'overflow (jusqu'à 3.54 %).

sim-001	GOLD	0.899	0.250	1.000	0.510	1.99%	✓
sim-009	GOLD	0.902	0.250	1.000	0.510	2.00%	✓
sim-005	GOLD	0.929	0.250	1.000	0.516	2.02%	✓
sim-013	GOLD	0.929	0.250	1.000	0.516	2.02%	✓
sim-030	SILVER	0.702	0.500	1.000	0.600	2.35%	✓
sim-022	SILVER	0.703	0.500	1.000	0.601	2.35%	✓
sim-026	SILVER	0.703	0.500	1.000	0.601	2.35%	✓
sim-034	SILVER	0.708	0.500	1.000	0.602	2.35%	✓
sim-042	SILVER	0.717	0.500	1.000	0.603	2.36%	✓
node-datacenter	GOLD	1.000	0.750	0.111	0.612	2.39%	✓
sim-018	GOLD	0.909	0.500	1.000	0.642	2.51%	✓
sim-002	GOLD	0.914	0.500	1.000	0.643	2.51%	✓
sim-014	GOLD	0.914	0.500	1.000	0.643	2.51%	✓
sim-006	GOLD	0.919	0.500	1.000	0.644	2.52%	✓
sim-010	GOLD	0.927	0.500	1.000	0.645	2.53%	✓
node-isp	GOLD	1.000	1.000	0.000	0.720	2.82%	✓
sim-023	SILVER	0.701	0.750	1.000	0.730	2.86%	✓
sim-027	SILVER	0.701	0.750	1.000	0.730	2.86%	✓
sim-039	SILVER	0.713	0.750	1.000	0.733	2.87%	✓
sim-011	GOLD	0.892	0.750	1.000	0.768	3.01%	✓
sim-007	GOLD	0.898	0.750	1.000	0.770	3.01%	✓
sim-003	GOLD	0.904	0.750	1.000	0.771	3.02%	✓
sim-019	GOLD	0.924	0.750	1.000	0.775	3.03%	✓

FIGURE 4.8 – Tableau WSM complet : classement des 38 pairs éligibles avec scores et allocations.

Mitigation distribuée active. La Figure 4.9 présente l'étape de mitigation distribuée. 52 tunnels GRE sont activés simultanément, absorbant un volume total de 15,7 Gbps sur les 17 Gbps d'overflow. Les dix premiers nœuds du tableau sont triés par allocation décroissante : les pairs GOLD à forte latence inversée (sim-004, sim-008, sim-012) absorbent chacun 0,6 Gbps, suivis des nœuds réels (node-isp à 0,5 Gbps). À la clôture de l'attaque, 52 pairs voient leur score PeerTrust recalculé avec la sévérité HIGH ($D = 0,75$) comme facteur contextuel.

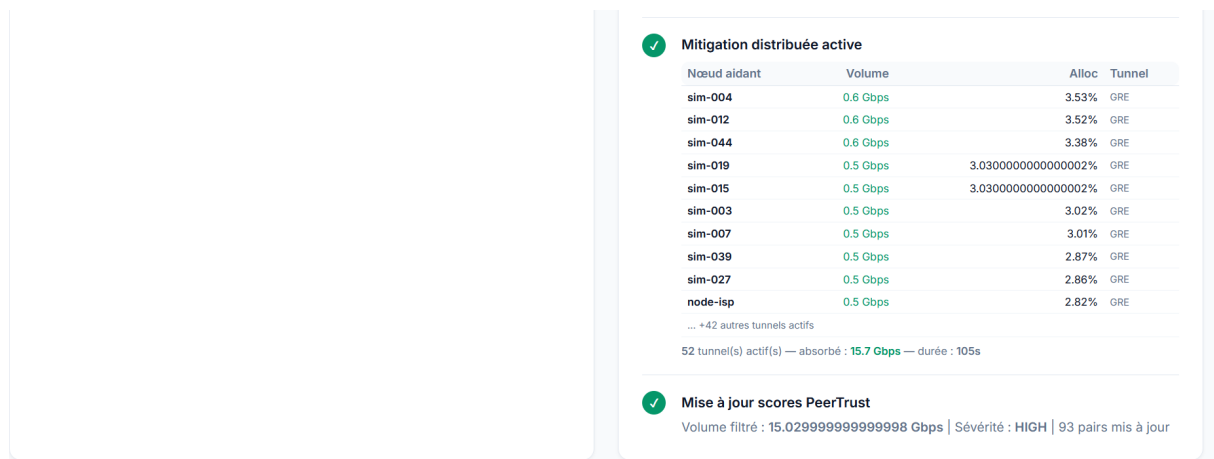


FIGURE 4.9 – Mitigation distribuée active : 52 tunnels GRE, 15,7 Gbps absorbés, recalcul PeerTrust sur 52 pairs.

Traçabilité : journal d’audit et messages P2P. La Figure 4.10 présente le journal d’audit horodaté : les événements `TRUST_LEVEL_CHANGED` confirment l’évolution des niveaux à la clôture de la session — plusieurs pairs progressent de `SUSPECT` vers `BRONZE`, et d’autres de `BRONZE` vers `SILVER`, témoignant de l’effet de récompense de la participation à une session d’aide réussie.

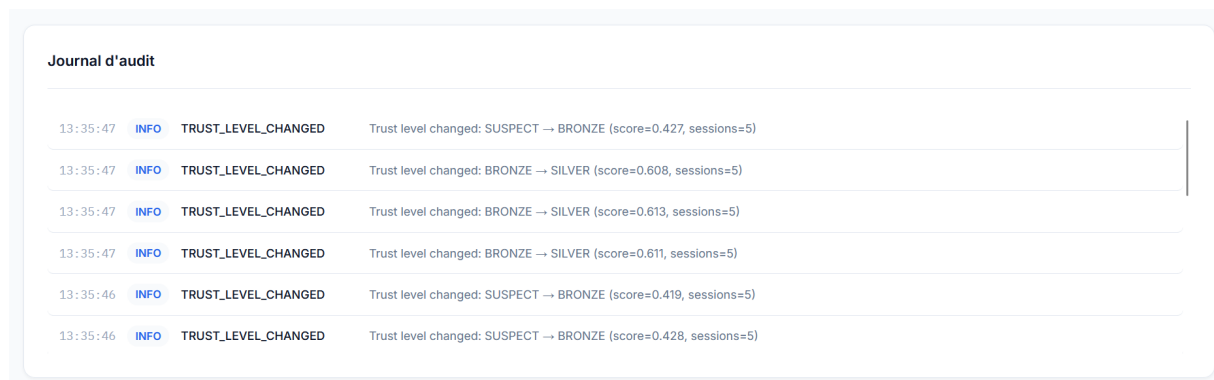


FIGURE 4.10 – Journal d’audit : événements `TRUST_LEVEL_CHANGED` à la clôture de l’attaque.

La Figure 4.11 présente les messages P2P échangés pendant la simulation : les `HELP_ACCEPT` confirment l’acceptation des sessions par les pairs (node-pme, node-isp, node-datacenter, sim-001), et les `TRAFFIC_REDIRECT` actent la mise en place des tunnels GRE.

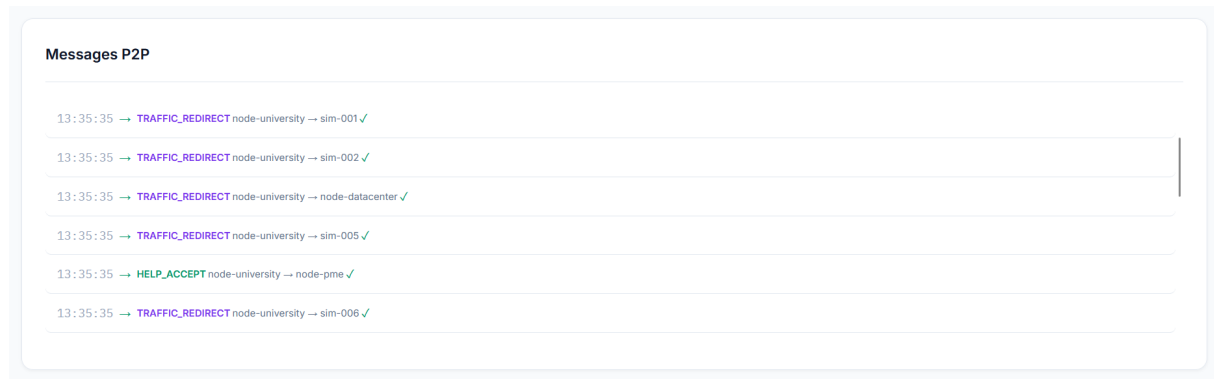


FIGURE 4.11 – Messages P2P : HELP_ACCEPT et TRAFFIC_REDIRECT échangés durant la mitigation.

Supervision des heartbeats. La Figure 4.12 présente le panel de supervision des heartbeats. Les trois nœuds réels envoient périodiquement leur état à node-university toutes les 30 secondes : node-datacenter (charge 20 %, 12 Gbps disponibles), node-isp (charge 20 %, 16 Gbps disponibles) et node-pme (charge 20 %, 4 Gbps disponibles). Ces échanges permettent au nœud victime de maintenir une vue à jour de la disponibilité réelle des pairs avant de déclencher une sélection WSM.

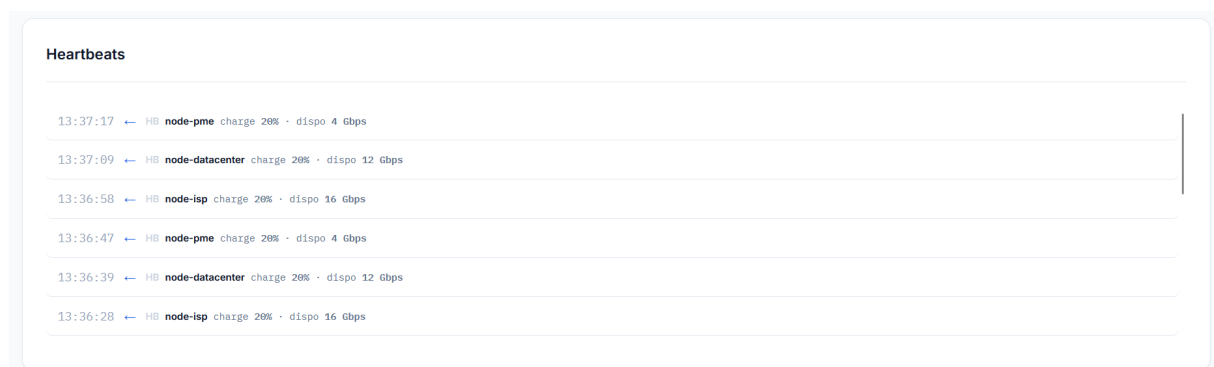


FIGURE 4.12 – Panel Heartbeats : battements périodiques des nœuds réels reçus par node-university.

4.8 Conclusion

Ce chapitre a présenté l'implémentation complète du système et ses résultats d'évaluation. Le gestionnaire de confiance implémente fidèlement la formule PeerTrust avec une crédibilité historique par session : les trois scénarios de validation (section 4.7.2) produisent des scores $T(u)$ conformes aux valeurs théoriques calculées à la main, et le bannissement automatique se déclenche correctement au seuil $T(u) < 0,20$. L'algorithme WSM alloue le flux proportionnellement aux scores composites, garantissant que les pairs les plus capables et les plus fiables absorbent la part la plus importante du trafic d'attaque. La double sécurisation JWT RS256 (et mTLS, disponible mais désactivé par défaut dans ce prototype) a rejeté l'ensemble des cinq scénarios d'attaque simulés avec le code HTTP approprié.

La démonstration sur le tableau de bord de supervision confirme quant à elle l'intégration cohérente de l'ensemble des composants en conditions réalistes : le scénario complet s'est exécuté avec succès de bout en bout, validant la chaîne authentification → détection → sélection WSM → formation de la coalition → clôture → recalcul du score de confiance. Les tests de scalabilité confirment un taux de succès de 100 % jusqu'à 100 nœuds, avec une sélection WSM toujours inférieure à 33 ms ; la variance observée sur certains paliers (section 4.7.4) reflète l'absence de répétitions multiples plutôt qu'une dégradation algorithmique. Ces résultats ont été obtenus sur une infrastructure locale ; une évaluation sur un déploiement géographiquement distribué, soumis à la latence réseau réelle, constitue une perspective de travail futur.

Conclusion générale

Ce mémoire a présenté la conception et l'implémentation d'un système distribué de mitigation collaborative des attaques DDoS, reposant sur une coalition pair-à-pair de nœuds de protection hétérogènes. Face aux limites structurelles des approches centralisées — points de défaillance uniques, coûts élevés et absence de mutualisation des ressources — le système proposé offre une alternative décentralisée où chaque participant prend ses décisions de manière autonome tout en contribuant à la défense collective.

Le premier chapitre a posé les fondements conceptuels : taxonomie des attaques DDoS, mécanismes d'infrastructure de mitigation existants et principes des architectures pair-à-pair. Le deuxième chapitre a analysé les approches collaboratives existantes (DOTS, DDoS Clearing House, NaWas/NBIP, DefCOM et PeerTrust) et identifié leurs lacunes en matière de mécanismes de confiance et de sélection intelligente des pairs. Le troisième chapitre a présenté la conception du système : la conception préliminaire a défini les exigences et l'architecture globale de la coalition ; la conception technique détaillée a formalisé le modèle de données relationnel en quatorze tables, le mécanisme de confiance PeerTrust adapté, l'algorithme de sélection WSM avec pondération AHP, et les protocoles de sécurité mTLS et JWT RS256. Le quatrième chapitre a détaillé l'implémentation et l'évaluation du prototype. Les tests de scalabilité conduits sur une coalition simulée de 20 à 100 nœuds ont confirmé un taux de succès de 100 % pour les opérations d'enregistrement et de heartbeat à tous les paliers, un temps de sélection WSM inférieur à 33 ms quasi-constant quelle que soit la taille de la coalition, et des temps de réponse heartbeat restant en dessous de 500 ms jusqu'à 100 nœuds simultanés.

Les contributions principales de ce travail sont au nombre de trois. La première concerne l'**adaptation du modèle PeerTrust avec crédibilité historique** : la crédibilité Cr_i est désormais capturée à la clôture de chaque session d'aide et persistée en base de données, rendant le calcul de $T(u)$ fidèle à la réalité temporelle du réseau. La deuxième contribution porte sur la **sélection multicritère des pairs par WSM-AHP** : l'algorithme combine quatre critères (capacité, latence, confiance, réciprocité) avec des poids validés (CR = 1,6 %), permettant à tous les pairs éligibles de participer simultanément. La troisième contribution est la **formalisation d'un protocole de coordination P2P** : le flux *help/request* → *offer* → *accept* → clôture définit un échange structuré en quatre phases qui permet à un nœud victime de solliciter, négocier et clore une session d'aide de bout en bout, sans aucun coordinateur central, tout en maintenant une traçabilité complète des volumes absorbés et des crédits de réciprocité.

Ce travail ouvre plusieurs perspectives d'amélioration. En premier lieu, la mise en place d'un **mécanisme de découverte automatique** constitue une priorité : un protocole de bootstrap similaire aux architectures DHT permettrait à un nœud de rejoindre la coalition en ne connaissant qu'un seul pair initial. La **détection des comportements**

malveillants avancés mérite également d'être approfondie, notamment les attaques Sybil et le free-riding coordonné, par l'intégration d'un module de détection d'anomalies comportementales. Sur le plan de la scalabilité, un déploiement sur une infrastructure géographiquement distribuée permettrait de valider le comportement du protocole P2P sous des latences réseau réelles. À plus long terme, une **passerelle vers DOTS** (RFC 8782) permettrait à la coalition de s'interfacer avec les prestataires de mitigation commerciaux. Enfin, une **évaluation en environnement réel** sur une infrastructure géographiquement distribuée permettrait de valider les résultats obtenus en simulation et de mesurer l'impact de la latence réseau réelle sur les mécanismes de recalcul de confiance.

En définitive, ce travail démontre la faisabilité d'une défense collaborative distribuée contre les attaques DDoS, reposant sur des mécanismes de confiance formels et une sélection intelligente des pairs. Il constitue une base solide pour le développement d'une infrastructure de coalition opérationnelle, adaptée aux contraintes hétérogènes des organisations participantes.

Bibliographie

- AIOHTTP CONTRIBUTORS. (2024). *aiohttp* — *Asynchronous HTTP Client/Server for asyncio*. URL : <https://docs.aiohttp.org>.
- AMAZON WEB SERVICES. (2024). *AWS Shield* — *Managed DDoS Protection*. URL : <https://aws.amazon.com/shield>.
- ANTONAKAKIS, M., et al. (2017). Understanding the Mirai Botnet. *Proc. 26th USENIX Security Symposium*, 1093-1110.
- AUTH0. (2024). *jsonwebtoken* — *JSON Web Token implementation for Node.js*. URL : <https://github.com/auth0/node-jwebtoken>.
- BAKER, F., & SAVOLA, P. (2004). Ingress Filtering for Multihomed Networks [RFC 3704, BCP 84].
- BATATA, S. (2015). Introduction au génie logiciel (IGL) [École nationale supérieure d'informatique (ESI), Alger].
- BIERMAN, A., BJÖRKLUND, M., & WATSEN, K. (2017). RESTCONF Protocol [RFC 8040].
- BJÖRKLUND, M. (2016). The YANG 1.1 Data Modeling Language [RFC 7950].
- BORMANN, C., & HOFFMAN, P. (2020). Concise Binary Object Representation (CBOR) [RFC 8949].
- BOUCADAIR, M., et al. (2022). Distributed Denial-of-Service Open Threat Signaling (DOTS) Telemetry [RFC 9244].
- CHANG, R. K. C. (2002). Defending Against Flooding-Based Distributed Denial-of-Service Attacks : A Tutorial. *IEEE Communications Magazine*, 40(10), 42-51.
- CHUA, W. T. (2018). *Memcrashed* — *Major Amplification Attacks from UDP Port 11211* [Cloudflare Blog]. URL : <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211>.
- CLOUDFLARE. (2025). *DDoS Threat Report* — *Q1 2025*. URL : <https://blog.cloudflare.com/ddos-threat-report-2025-q1>.
- DOCKER, INC. (2024). *Docker Desktop* — *Accelerated Container Application Development*. URL : <https://www.docker.com/products/docker-desktop>.
- DOULIGERIS, C., & MITROKOTSA, A. (2004). DDoS Attacks and Defense Mechanisms : Classification and State-of-the-Art. *Computer Networks*, 44(5), 643-666. <https://doi.org/10.1016/j.comnet.2003.10.003>
- EDDY, W. (2007). TCP SYN Flooding Attacks and Common Mitigations [RFC 4987].
- FAYAZ, S. K., TOBIOKA, Y., SEKAR, V., & BAILEY, M. (2015). Bohatei : Flexible and Elastic DDoS Defense. *Proc. 24th USENIX Security Symposium*, 817-832.
- FERGUSON, P., & SENIE, D. (2000). Network Ingress Filtering : Defeating Denial of Service Attacks which employ IP Source Address Spoofing [RFC 2827, BCP 38].

- FIELDING, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [thèse de doct., University of California, Irvine]. URL : <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- GITHUB, INC. (2024). *GitHub — Where the World Builds Software*. URL : <https://github.com>.
- GOOGLE CLOUD. (2024). *Google Cloud Armor — DDoS Defense and Web Application Firewall*. URL : <https://cloud.google.com/armor>.
- HWANG, C.-L., & YOON, K. (1981). Multiple Attribute Decision Making : Methods and Applications. *Lecture Notes in Economics and Mathematical Systems*, 186.
- JGRAPH LTD. (2024). *draw.io / diagrams.net — Collaborative Diagramming Tool* [Utilisé pour la conception des diagrammes UML]. URL : <https://www.drawio.com>.
- JONES, M. B. (2015). JSON Web Algorithms (JWA) [RFC 7518].
- JONES, M. B., BRADLEY, J., & SAKIMURA, N. (2015). JSON Web Token (JWT) [RFC 7519].
- KUMARI, W., & MCPHERSON, D. (2009). Remote Triggered Black Hole Filtering with Unicast Reverse Path Forwarding (uRPF) [RFC 5635].
- LEACH, P. J., MEALLING, M., & SALZ, R. (2005). A Universally Unique Identifier (UUID) URN Namespace [RFC 4122].
- LUCID SOFTWARE INC. (2024). *Lucidchart — Diagramming and Visual Communication Platform* [Utilisé pour la conception des diagrammes de cas d'utilisation et de séquence]. URL : <https://www.lucidchart.com>.
- MAHAJAN, R., et al. (2002). Controlling High Bandwidth Aggregates in the Network. *ACM SIGCOMM Computer Communication Review*, 32(3), 62-73. <https://doi.org/10.1145/571697.571724>
- MARQUES, P., et al. (2009). Dissemination of Flow Specification Rules [RFC 5575].
- MICROSOFT AZURE. (2024). *Azure DDoS Protection*. URL : <https://azure.microsoft.com/products/ddos-protection>.
- MIRKOVIC, J., PRIER, G., & REIHER, P. (2003). DefCOM : Distributed Cooperative Overlay Defense Against DDoS Attacks. *Proceedings of the IEEE International Conference on Network Protocols (ICNP)*, 239-250. <https://doi.org/10.1109/ICNP.2003.1249775>
- MIRKOVIC, J., & REIHER, P. (2004). A Taxonomy of DDoS Attack and DDoS Defense Mechanisms. *ACM SIGCOMM Computer Communication Review*, 34(2), 39-53. <https://doi.org/10.1145/997150.997156>
- MIRKOVIC, J., & REIHER, P. (2005). D-WARD : A Source-End Defense Against Flooding Denial-of-Service Attacks. *IEEE Transactions on Dependable and Secure Computing*, 2(3), 216-232. <https://doi.org/10.1109/TDSC.2005.35>
- MORTENSEN, A., REDDY, T., & MOSKOWITZ, R. (2019). Distributed Denial-of-Service Open Threat Signaling (DOTS) Requirements [RFC 8612].
- NBIP. (2023). *NaWas : National Scrubbing Center for DDoS Attacks* (rapp. tech.). Nationale Beheersorganisatie Internet Providers.
- NETSCOUT SYSTEMS. (2023). *NETSCOUT Threat Intelligence Report — 1st Half 2023* (rapp. tech.). NETSCOUT Systems.
- OBJECT MANAGEMENT GROUP. (2017). Unified Modeling Language (UML) Specification, Version 2.5.1.
- OPENAPI INITIATIVE. (2020). *OpenAPI Specification, Version 3.0.3*. URL : <https://spec.openapis.org/oas/v3.0.3>.

- OPENJS FOUNDATION. (2024a). *Express.js — Fast, Unopinionated, Minimalist Web Framework for Node.js* [Version 4]. URL : <https://expressjs.com>.
- OPENJS FOUNDATION. (2024b). *Node.js — JavaScript Runtime Built on Chrome's V8 Engine* [Version 20 LTS]. URL : <https://nodejs.org/en/download>.
- OWASP FOUNDATION. (2023). *JSON Web Token Cheat Sheet for Java*. URL : https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html.
- PYTHON SOFTWARE FOUNDATION. (2024). *asyncio — Asynchronous I/O, version 3.12*. URL : <https://docs.python.org/3/library/asyncio.html>.
- REDDY, T., et al. (2020). Distributed Denial-of-Service Open Threat Signaling (DOTS) Signal Channel Specification [RFC 8782].
- REKHTER, Y., LI, T., & HARES, S. (2006). A Border Gateway Protocol 4 (BGP-4) [RFC 4271].
- RESCORLA, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 [RFC 8446].
- RESCORLA, E., & MODADUGU, N. (2012). Datagram Transport Layer Security Version 1.2 [RFC 6347].
- ROSSOW, C. (2014). Amplification Hell : Revisiting Network Protocols for DDoS Abuse. *Proc. Network and Distributed System Security Symposium (NDSS)*.
- SAATY, T. L. (1980). *The Analytic Hierarchy Process : Planning, Priority Setting, Resource Allocation*. McGraw-Hill.
- SEQUELIZE CONTRIBUTORS. (2024). *Sequelize — A Modern TypeScript and Node.js ORM* [Version 6]. URL : <https://sequelize.org>.
- SHELBY, Z., HARTKE, K., & BORMANN, C. (2014). The Constrained Application Protocol (CoAP) [RFC 7252].
- SIDN LABS. (2021). *DDoS Clearing House : Sharing DDoS Attack Fingerprints to Improve Collective Defense* (rapp. tech.). CONCORDIA H2020 Project.
- SPECHT, S. M., & LEE, R. B. (2004). Distributed Denial of Service : Taxonomies of Attacks, Tools, and Countermeasures. *Proc. International Workshop on Security in Parallel and Distributed Systems*, 543-550. <https://doi.org/10.1109/ISPAN.2004.1336251>
- STOICA, I., et al. (2001). Chord : A Scalable Peer-to-Peer Lookup Service for Internet Applications. *Proc. ACM SIGCOMM*, 149-160. <https://doi.org/10.1145/383059.383071>
- STORMWALL. (2024). *DDoS Severity Level*. URL : <https://stormwall.network/resources/instructions/faq/ddos-severity-level>.
- TANENBAUM, A. S., & VAN STEEN, M. (2017). *Distributed Systems : Principles and Paradigms* (3rd). Pearson Education.
- THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. (2024). *PostgreSQL : The World's Most Advanced Open Source Relational Database* [Version 15]. URL : <https://www.postgresql.org/download>.
- TORVALDS, L., et al. (2024). *Git — Distributed Version Control System*. URL : <https://git-scm.com>.
- XIONG, L., & LIU, L. (2004). PeerTrust : Supporting Reputation-Based Trust for Peer-to-Peer Electronic Communities. *IEEE Transactions on Knowledge and Data Engineering*, 16(7), 843-857. <https://doi.org/10.1109/TKDE.2004.1318566>
- ZARGAR, S. T., JOSHI, J., & TIPPER, D. (2013). A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks. *IEEE Communications*

BIBLIOGRAPHIE

Surveys & Tutorials, 15(4), 2046-2069. <https://doi.org/10.1109/SURV.2013.031413.00127>